

Block 3: Decision Making

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 7

Probabilistic Reasoning

ഉറപ്പില്ലാത്ത (uncertain) വിവരങ്ങൾ വെച്ച് ഒരു AI system എങ്ങനെ തീരുമാനമെടുക്കുന്നു എന്നതാണ് ഈ unit. Probability-യുടെ അടിസ്ഥാനങ്ങൾ, conditional probability, Bayes' theorem, Bayesian networks, MYCIN-ലെ certainty factors, Dempster-Shafer theory എന്നിവ ഓരോന്നായി worked examples-ഓടെ പഠിക്കാം.

Reasoning with uncertain information

Trials, sample space and events

Complement, union, intersection, mutually exclusive events

Relative frequency and the rules of probability

Conditional probability

Independent events, multiplication rule, total probability

Joint, marginal and conditional probability

Bayes' theorem

Bayes' theorem with many hypotheses (spam filter, factories)

Bayesian networks

Probabilistic inference with Bayesian networks

Other paradigms: MYCIN, certainty factors, heuristic reasoning

Dempster-Shafer theory and types of evidence

Frame of discernment, bpa, belief and plausibility

Dempster's rule of combination

Reasoning with uncertain information

Definition: Probabilistic reasoning is reasoning under uncertainty in which each statement is given a probability (degree of likelihood) between 0 and 1, and conclusions and decisions are based on these values.

Probabilistic reasoning എന്നാൽ ഓരോ statement-നും 0-നും 1-നും ഇടയിലുള്ള ഒരു probability (സാധ്യതയുടെ അളവ്) നൽകി, ആ മൂല്യങ്ങളുടെ അടിസ്ഥാനത്തിൽ നിഗമനങ്ങളും തീരുമാനങ്ങളും എടുക്കുന്ന, uncertainty-യിലുള്ള reasoning ആണ്.

Classical logic works only with statements that are definitely true or false. Real information is often uncertain: it comes from experimental errors, faulty instruments, unreliable sources or incomplete knowledge. Decisions still have to be made, so AI systems use **probabilistic reasoning**: each statement gets a degree of likelihood between 0 and 1, and decisions are based on these numbers.

മലയാളത്തിൽ

ജീവിതത്തിലെ മിക്ക തീരുമാനങ്ങളും ഉറപ്പില്ലാതെയാണ് എടുക്കുന്നത്. "നാളെ മഴ പെയ്യുമോ?" എന്ന് ആർക്കും 100% ഉറപ്പില്ല, പക്ഷേ "70% സാധ്യത" എന്ന് കേട്ടാൽ നമ്മൾ കൂട എടുക്കും. ഇതുപോലെ True/False-ന് പകരം ഒരു probability number വെച്ച് reason ചെയ്യുന്നതാണ് probabilistic reasoning. Sensor തെറ്റു, വിശ്വസിക്കാൻ പറ്റാത്ത source, അപൂർണ്ണമായ അറിവ് ഇവയൊക്കെയാണ് uncertainty-യുടെ കാരണങ്ങൾ.

Probability is a function P mapping events of the sample space S to real numbers, with three axioms:

- (i) $0 \leq P(E) \leq 1$ for every event $E \subseteq S$
- (ii) $P(S) = 1$ (a certain outcome)
- (iii) If $E_1, E_2, \dots$ are mutually exclusive ($E_i \cap E_j = \emptyset$ for $i \neq j$), then $P(E_1 \cup E_2 \cup \dots) = P(E_1) + P(E_2) + \dots$

EXAMPLE

A weather app gives $P(\text{rain}) = 0.7$, $P(\text{no rain}) = 0.3$ and $P(\text{rain and no rain}) = 0.1$ for tomorrow. Is this a valid probability assignment?

"Rain" and "no rain" cannot happen together, so $P(\text{rain} \cap \text{no rain})$ must be $P(\emptyset) = 0$, not 0.1. Also $P(\text{rain}) + P(\text{no rain}) = 1 = P(S)$, which is fine. The value 0.1 breaks the axioms, so the assignment is invalid.

Exam tip: The axioms alone do not tell you any actual probability. You still need data (experiments, records) to estimate the underlying distribution; the unit states this explicitly.

Trials, sample space and events

Trial: one performance of a sub-experiment of a random experiment (e.g. one toss of a coin).

Sample space: the set ζ (or S) of all possible outcomes of a random experiment.

Event: any subset E of the sample space; E occurs if the outcome of the experiment lies in E .

Trial: ഒരു random experiment-ന്റെ ഭാഗമായ ഒരു sub-experiment ഒരു തവണ നടത്തുന്നത് (ഉദാ. coin ഒരു തവണ toss ചെയ്യുന്നത്).

Sample space: ഒരു random experiment-ന്റെ സാധ്യമായ എല്ലാ outcomes-ന്റെയും set ζ (അഥവാ S).

Event: Sample space-ന്റെ ഏതൊരു subset E -യും; experiment-ന്റെ outcome E -ൽ ഉൾപ്പെട്ടാൽ E സംഭവിച്ചു എന്ന് പറയുന്നു.

A random experiment is often made of smaller sub-experiments called **trials** (each toss of a coin, each item picked for checking). The set of all possible outcomes is the **sample space**, written ζ (or S). An **event** is any collection of sample points; "event E occurs" means the outcome of the experiment lies in E .

മലയാളത്തിൽ

മൂന്ന് തവണ coin toss ചെയ്യുന്നത് ഒരു experiment ആണ്; ഓരോ toss-ഉം ഒരു trial. സാധ്യമായ എല്ലാ result-കളുടെയും കൂട്ടമാണ് sample space. അതിൽ നിന്ന് നമുക്ക് താല്പര്യമുള്ള ചില result-കൾ മാത്രം എടുത്താൽ അത് ഒരു event. ഉദാഹരണത്തിന് "കുറഞ്ഞത് ഒരു head" എന്നത് ഒരു event ആണ്.

Three tosses of a coin (the unit's Example 3), $2 \times 2 \times 2 = 8$ outcomes:

$s_1 = HHH, s_2 = HHT, s_3 = HTH, s_4 = THH, s_5 = TTH, s_6 = THT, s_7 = HTT, s_8 = TTT$

EXAMPLE (CHECK YOUR PROGRESS 1, Q1)

Write the sample space: (a) an item is tested for defect; (b) one ball is drawn from an urn of six differently coloured balls; (c) a card from 1 to 10 is drawn, replaced, and a second card drawn.

(a) $\zeta = \{\text{Defective, Non-defective}\}$, 2 points.

(b) $\zeta = \{c_1, c_2, c_3, c_4, c_5, c_6\}$, one point per colour, 6 points.

(c) $\zeta = \{(i, j) : i, j \in \{1, 2, \dots, 10\}\}$. With replacement both draws can be any card, so there are $10 \times 10 = 100$ ordered pairs, including (3, 3) etc.

EXAMPLE (CHECK YOUR PROGRESS 1, Q2)

A die is thrown twice; ζ has 36 ordered pairs (i, j) . Describe the events.

- (i) Maximum score is 6: all pairs with at least one 6: $(6, 1) \dots (6, 6)$ and $(1, 6) \dots (5, 6)$, so $6 + 5 = 11$ points.
- (ii) Total is 9: $\{(3, 6), (4, 5), (5, 4), (6, 3)\}$, 4 points.
- (iii) Both scores even: $i, j \in \{2, 4, 6\}$, so $3 \times 3 = 9$ points.
- (iv) Both even and larger than 2: $\{(4, 4), (4, 6), (6, 4), (6, 6)\}$, 4 points.
- (v) Scores differ by at least 2: $|i - j| \geq 2$. Pairs with difference 0: 6, difference 1: 10, so $36 - 6 - 10 = 20$ points.

Exam tip: The unit's printed list writes $s_5 = (1, T, H)$; it is a typo for (T, T, H) . Always count sample points with the multiplication rule ($2^3 = 8$, $6^2 = 36$) to check your list is complete.

Complement, union, intersection, mutually exclusive events

Complement: E^c is the set of all sample points not in E , so $P(E^c) = 1 - P(E)$.

Union / Intersection: $E \cup F$ occurs when at least one of E, F occurs; $E \cap F$ occurs when both occur.

Mutually exclusive: events E and F with $E \cap F = \emptyset$, so they cannot occur together.

Complement: E -യിൽ ഇല്ലാത്ത എല്ലാ sample points-ന്റെയും set ആണ് E^c ; അതിനാൽ $P(E^c) = 1 - P(E)$.

Union / Intersection: E, F -ൽ ഒന്നെങ്കിലും സംഭവിച്ചാൽ $E \cup F$ സംഭവിക്കുന്നു; രണ്ടും സംഭവിച്ചാൽ $E \cap F$ സംഭവിക്കുന്നു.

Mutually exclusive: $E \cap F = \emptyset$ ആയ events E, F ; അവ ഒരമിച്ച് സംഭവിക്കില്ല.

The **complement** E^c contains all sample points not in E ; E and E^c together make up ζ , and $\zeta^c = \emptyset$ (the null event). The **union** $E \cup F$ occurs when at least one of E, F occurs. The **intersection** $E \cap F$ occurs when both occur. E and F are **mutually exclusive** (disjoint) if $E \cap F = \emptyset$, so they cannot happen together. A Venn diagram (rectangle = ζ , closed curves = events) shows these relations.

മലയാളത്തിൽ

Complement എന്നാൽ "ആ event നടക്കാത്തത്". Union ($\cup$) "ഏതെങ്കിലും ഒന്നെങ്കിലും", intersection ($\cap$) "രണ്ടും ഒരുമിച്ച്". ഒരു coin toss-ൽ head-ഉം tail-ഉം ഒരുമിച്ച് വരില്ല, അതുകൊണ്ട് അവ mutually exclusive. ഒരു ദിവസം "മഴ" എന്നതും "വെയിൽ" എന്നതും mutually exclusive അല്ല; വെയിലത്ത് മഴ (കറുക്കന്റെ കല്യാണം!) നമ്മൾ കണ്ടിട്ടുണ്ടല്ലോ.

$$E \cup E^c = \zeta, E \cap E^c = \emptyset$$

$$P(E^c) = 1 - P(E)$$

$$E, F \text{ mutually exclusive} \Leftrightarrow E \cap F = \emptyset$$

EXAMPLE (FROM THE UNIT)

Three tosses. E = at least one head; F_1 = odd number of heads; F_2 = tails on the first two tosses. Find E^c and $F_1 \cap F_2$.

$E = \{s_1, \dots, s_7\}$, so $E^c = \{s_8\} = \{TTTT\}$, "no head at all".

F_1 : 1 or 3 heads = $\{HHH, TTH, THT, HTT\} = \{s_1, s_5, s_6, s_7\}$. $F_2 = \{TTH, TTT\} = \{s_5, s_8\}$.

$F_1 \cap F_2 = \{s_5\} = \{TTH\}$: only the third toss is a head. The intersection is not empty, so F_1 and F_2 are not mutually exclusive.

Relative frequency and the rules of probability

Relative frequency: $r_n(E) = f_n(E)/n$, the fraction of n repetitions of an experiment in which event E occurs.

Probability (frequency view): the value that $r_n(E)$ settles to as n becomes large; for equally likely outcomes, $P(E) = |E| / |\zeta|$.

Relative frequency: $r_n(E) = f_n(E)/n$; ഒരു experiment n തവണ ആവർത്തിക്കുമ്പോൾ event E സംഭവിക്കുന്ന തവണകളുടെ അനുപാതം.

Probability (frequency view): n വലുതാകുമ്പോൾ $r_n(E)$ എത്തിച്ചേരുന്ന സ്ഥിര മൂല്യം; equally likely outcomes ആണെങ്കിൽ $P(E) = |E| / |\zeta|$.

Repeat an experiment n times under identical conditions. If event E occurs $f_n(E)$ times, then $f_n(E)$ is its **frequency** and $r_n(E) = f_n(E)/n$ its **relative frequency**. As n grows, $r_n(E)$ settles near a fixed number; this long-run value is the (frequency) probability of E . For equally likely outcomes, $P(E) = (\text{number of points in } E) / (\text{number of points in } \zeta)$.

മലയാളത്തിൽ

ഒരു കോയിൻ 5 തവണ എറിഞ്ഞാൽ 3 head കിട്ടാം, അപ്പോൾ relative frequency 3/5. ആയിരം തവണ എറിഞ്ഞാൽ ഇത് 1/2-ന് അടുത്തെത്തും. KSRTC ബസ് 100 ദിവസത്തിൽ 20 ദിവസം late ആയാൽ "late ആകാനുള്ള probability ഏകദേശം 0.2" എന്ന് പറയുന്നതും ഇതേ ആശയമാണ്.

$$r_n(E) = f_n(E)/n$$

Equally likely outcomes: $P(E) = |E| / |\zeta|$

Addition rule: $P(E \cup F) = P(E) + P(F) - P(E \cap F)$

Complement rule: $P(E^c) = 1 - P(E)$

EXAMPLE (THE UNIT'S TABLE 1)

n	$f_n(H)$	$r_n(H)$
1	0	0
2	1	1/2
3	2	2/3
4	3	3/4
5	3	3/5

The fifth toss was a tail (frequency stays 3). The relative frequency jumps around for small n; only for large n does it approach $P(H) = 1/2$.

EXAMPLE

Two dice are thrown. Find $P(\text{total is 9 or both scores even})$.

A = total 9: 4 points. B = both even: 9 points. $A \cap B$: a total of 9 is odd, so it cannot come from two even scores; $A \cap B = \emptyset$.

$$P(A \cup B) = 4/36 + 9/36 - 0 = 13/36.$$

Conditional probability

Definition: The conditional probability of E given F is $P(E|F) = P(E \cap F) / P(F)$, defined when $P(F) > 0$; it is the probability of E once F is known to have occurred.

F നൽകപ്പെട്ടാൽ E-യുടെ conditional probability $P(E|F) = P(E \cap F) / P(F)$ ആണ് ($P(F) > 0$ ആകുമ്പോൾ മാത്രം); F സംഭവിച്ചു എന്ന് അറിഞ്ഞശേഷം E-യുടെ probability ആണ് ഇത്.

Once we learn that event F has occurred, the chance of E must be revised: only outcomes inside F are still possible. The **conditional probability** of E given F is the fraction of F's probability that also lies in E. If $F \subseteq E$, then $P(E|F) = 1$; if E and F are mutually exclusive, $P(E|F) = 0$.

മലയാളത്തിൽ

പുതിയ വിവരം കിട്ടുമ്പോൾ probability മാറും. "ഇന്ന് മഴ പെയ്യുമോ?" എന്നതിന് സാധാരണ ഒരു സാധ്യത ഉണ്ട്; പക്ഷേ "ആകാശം കറുത്തിരുണ്ടിരിക്കുന്നു" എന്ന് അറിഞ്ഞാൽ ആ സാധ്യത കൂടും. $P(E|F)$ എന്നത് "F നടന്നു എന്ന് അറിയാം, ഇനി E-യുടെ സാധ്യത എത്ര" എന്നാണ്. F-നെ പുതിയ sample space ആയി കണക്കാക്കുന്നു എന്ന് ഓർത്താൽ മതി.

$P(E|F) = P(E \cap F) / P(F)$, provided $P(F) > 0$ (undefined when $P(F) = 0$)

Rearranged: $P(E \cap F) = P(E|F) \cdot P(F)$

Frequency view: $P(E|F) \approx \text{No.}(E \cap F) / \text{No.}(F)$ for a large number of repetitions

EXAMPLE (THE UNIT'S EXAMPLE 7)

Two balanced dice. E = sum is 10 or more; F = exactly one of the scores is 5. Find $P(E)$ and $P(E|F)$.

$E = \{(4, 6), (5, 5), (5, 6), (6, 4), (6, 5), (6, 6)\}$, so $P(E) = 6/36 = 1/6$.

$F = \{(1, 5), (2, 5), (3, 5), (4, 5), (6, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 6)\}$, so $P(F) = 10/36$. Note (5, 5) is not in F (both are 5).

$E \cap F = \{(5, 6), (6, 5)\}$, $P(E \cap F) = 2/36$.

$P(E|F) = (2/36)/(10/36) = 2/10 = 1/5$. Knowing F changed the probability of E from $1/6$ to $1/5$.

EXAMPLE (USING THE UNIT'S TABLE 2)

A probability table has $P(E \cap F) = p = 0.1$, $P(E^c \cap F) = q = 0.2$, $P(E \cap F^c) = r = 0.3$, $P(E^c \cap F^c) = s = 0.4$. Find $P(E)$, $P(F)$, $P(E|F)$.

	E	E ^c	Total
F	$p = 0.1$	$q = 0.2$	0.3
F ^c	$r = 0.3$	$s = 0.4$	0.7
Total	0.4	0.6	1

$P(E) = p + r = 0.4$ (column total). $P(F) = p + q = 0.3$ (row total).

$P(E|F) = p/(p + q) = 0.1/0.3 = 1/3$.

Exam tip: The unit writes $P(F) = q + s$ under Table 2. That is a printing error: $q + s = P(E^c)$. From the table, $P(F) = p + q$ (the F row), so $P(E|F) = p/(p + q)$. Also, "Referring back to Example 3" at that point should read Example 7.

Independent events, multiplication rule, total probability

Independent events: E and F are independent if $P(E \cap F) = P(E) \cdot P(F)$, equivalently $P(E|F) = P(E)$.

Multiplication rule: $P(E \cap F) = P(E|F) \cdot P(F) = P(F|E) \cdot P(E)$.

Total probability: if $A_1, \dots, A_n$ are mutually exclusive and exhaustive, $P(B) = \sum_i P(A_i) P(B|A_i)$.

Independent events: $P(E \cap F) = P(E) \cdot P(F)$ ആണെങ്കിൽ (അതായത് $P(E|F) = P(E)$) E, F എന്നിവ independent ആണ്.

Multiplication rule: $P(E \cap F) = P(E|F) \cdot P(F) = P(F|E) \cdot P(E)$.

Total probability: $A_1, \dots, A_n$ mutually exclusive-ഉം exhaustive-ഉം ആണെങ്കിൽ $P(B) = \sum_i P(A_i) P(B|A_i)$.

E and F are **independent** if knowing F tells nothing about E, i.e. $P(E|F) = P(E)$. Then the joint probability is just the product. For any events, the **multiplication rule** gives the joint probability from a conditional one, and the **law of total probability** gets $P(B)$ by splitting over mutually exclusive, exhaustive cases $A_1, \dots, A_n$. Bayes' theorem uses this as its denominator.

മലയാളത്തിൽ

ഒരു event മറ്റേതിനെ ഒട്ടും ബാധിക്കുന്നില്ലെങ്കിൽ അവ independent ആണ്. തിരുവനന്തപുരത്തെ ഇന്നത്തെ lottery result-ഉം കാസർഗോഡ് ഇന്ന് മഴ പെയ്യുന്നതും independent. Total probability എന്നാൽ എല്ലാ വഴികളിലൂടെയും ഉള്ള സാധ്യതകൾ കൂട്ടുന്നത്: "മഴയുള്ള ദിവസം late ആകാനുള്ള സാധ്യത" + "മഴയില്ലാത്ത ദിവസം late ആകാനുള്ള സാധ്യത".

Independent: $P(E \cap F) = P(E) \cdot P(F)$ (equivalently $P(E|F) = P(E)$)

Multiplication rule: $P(E \cap F) = P(E|F) P(F) = P(F|E) P(E)$

Chain rule: $P(A, B, C) = P(A|B, C) P(B|C) P(C)$

Total probability: $P(B) = \sum_i P(A_i) P(B|A_i)$, A_i mutually exclusive and exhaustive

EXAMPLE

Two dice. E = first die even; F = total is 7. Are E and F independent?

$P(E) = 18/36 = 1/2$. $F = \{(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)\}$, $P(F) = 6/36 = 1/6$.

$E \cap F = \{(2, 5), (4, 3), (6, 1)\}$, $P(E \cap F) = 3/36 = 1/12$.

$P(E) P(F) = 1/2 \times 1/6 = 1/12 = P(E \cap F)$. So E and F are independent.

EXAMPLE (TOTAL PROBABILITY, MONSOON)

On a June day in Kochi, $P(\text{heavy rain}) = 0.3$. The bus is late with probability 0.7 on a heavy-rain day and 0.1 otherwise. Find $P(\text{bus late})$.

$P(\text{late}) = P(\text{rain}) P(\text{late}|\text{rain}) + P(\text{no rain}) P(\text{late}|\text{no rain}) = 0.3 \times 0.7 + 0.7 \times 0.1 = 0.21 + 0.07 = 0.28$.

Exam tip: Mutually exclusive is not the same as independent. If $P(E), P(F) > 0$ and $E \cap F = \emptyset$, then $P(E \cap F) = 0 \neq P(E)P(F)$, so mutually exclusive events with non-zero probability are always dependent.

Joint, marginal and conditional probability

Joint probability: $P(A, B) = P(A \cap B)$, the probability that A and B occur together.

Marginal probability: $P(A)$, the probability of A regardless of other variables, obtained by summing the joint probability over them.

Conditional probability: $P(A|B) = P(A, B) / P(B)$, the probability of A given that B has occurred.

Joint probability: $P(A, B) = P(A \cap B)$; A-യും B-യും ഒരുമിച്ച് സംഭവിക്കാനുള്ള probability.

Marginal probability: മറ്റ് variables പരിഗണിക്കാതെ A-യുടെ മാത്രം probability $P(A)$; joint probability-യെ മറ്റ് variables-ന്മേൽ sum ചെയ്താണ് ഇത് കിട്ടുന്നത്.

Conditional probability: $P(A|B) = P(A, B) / P(B)$; B സംഭവിച്ചു എന്ന് അറിയുമ്പോൾ A-യുടെ probability.

Three kinds of probability appear in Bayesian reasoning. **Joint** probability $P(A, B) = P(A \cap B)$ is the chance that A and B happen together. **Marginal** probability $P(A)$ is the chance of A regardless of other variables; it is obtained by summing the joint over the other variable. **Conditional** probability $P(A|B)$ is the chance of A given that B has occurred.

മലയാളത്തിൽ

Joint എന്നാൽ "രണ്ടും ഒരുമിച്ച്" (പനിയും ഉണ്ട്, Covid-ഉം ഉണ്ട്). Marginal എന്നാൽ മറ്റേ variable നോക്കാതെ ഒന്നിന്റെ മാത്രം സാധ്യത (പനി ഉള്ളവർ ആകെ എത്ര).

Conditional എന്നാൽ ഒന്ന് അറിഞ്ഞശേഷം മറ്റേതിന്റെ സാധ്യത (പനി ഉള്ളവരിൽ Covid ഉള്ളവർ എത്ര). ഒരു table-ൽ joint ഉള്ളിലെ cells, marginal row/column totals, conditional ഒരു cell-നെ അതിന്റെ total കൊണ്ട് ഹരിച്ചത്.

Joint: $P(A, B) = P(A \cap B)$

Marginal: $P(A) = \sum_b P(A, B = b)$

Conditional: $P(A|B) = P(A, B)/P(B)$

Hence $P(A|B) = P(B|A) P(A) / P(B)$

EXAMPLE (CHECK YOUR PROGRESS 3, Q1)

In a PHC in Thrissur, 200 patients were recorded:

	Fever	No fever	Total
Dengue	30	10	40
No dengue	50	110	160
Total	80	120	200

Joint: $P(\text{Dengue, Fever}) = 30/200 = 0.15$.

Marginal: $P(\text{Fever}) = 80/200 = 0.40$; $P(\text{Dengue}) = 40/200 = 0.20$.

Conditional: $P(\text{Dengue}|\text{Fever}) = 30/80 = 0.375$; $P(\text{Fever}|\text{Dengue}) = 30/40 = 0.75$.

Check with Bayes: $P(\text{Dengue}|\text{Fever}) = P(\text{Fever}|\text{Dengue}) P(\text{Dengue})/P(\text{Fever}) = 0.75 \times 0.20/0.40 = 0.375$. ✓

Exam tip: $P(A|B)$ and $P(B|A)$ are different numbers (0.375 vs 0.75 above). Confusing them is the most common mistake in Bayes questions.

Bayes' theorem

Definition: Bayes' theorem gives the posterior probability of a hypothesis H given evidence E : $P(H|E) = P(E|H) P(H) / P(E)$, i.e. posterior = likelihood $\times$ prior / evidence.

Evidence E നൽകപ്പെട്ടാൽ hypothesis H -ന്റെ posterior probability Bayes' theorem തരുന്നത്: $P(H|E) = P(E|H) P(H) / P(E)$, അതായത് posterior = likelihood $\times$ prior / evidence.

Bayes' theorem turns one conditional probability around: from $P(\text{evidence}|\text{hypothesis})$, which is easy to measure, it gives $P(\text{hypothesis}|\text{evidence})$, which is what we want to decide. $P(H)$ is the **prior**, $P(E|H)$ the **likelihood**, $P(E)$ the **evidence** (found by total probability) and $P(H|E)$ the **posterior**. It also works where intuition fails, as the medical-test example shows.

മലയാളത്തിൽ

Doctor-ന് അറിയാവുന്നത് "Covid ഉള്ളവരിൽ test positive ആകാനുള്ള സാധ്യത" ആണ്. പക്ഷേ patient-ന് അറിയേണ്ടത് "test positive ആയി, എനിക്ക് ശരിക്കും Covid ഉണ്ടോ?" എന്നാണ്. ഈ തിരിച്ചിലാണ് Bayes' theorem ചെയ്യുന്നത്. Prior (മുൻപത്തെ വിശ്വാസം) $\times$ likelihood (തെളിവ് എത്ര യോജിക്കുന്നു) $\div$ evidence = posterior (പുതുക്കിയ വിശ്വാസം).

$$P(H|E) = P(E|H) P(H) / P(E)$$

$$P(E) = P(E|H) P(H) + P(E|H^c) P(H^c)$$

$$\text{posterior} = \text{likelihood} \times \text{prior} / \text{evidence}$$

EXAMPLE 1: COVID / FEVER TEST

1% of people in a town have Covid. The test detects 95% of real cases (sensitivity) but also gives a positive result for 5% of healthy people (false positive rate). A person tests positive. What is the probability they have Covid?

Let D = has Covid, $+$ = positive test. $P(D) = 0.01$, $P(D^c) = 0.99$, $P(+|D) = 0.95$, $P(+|D^c) = 0.05$.

Step 1, numerator: $P(+|D) P(D) = 0.95 \times 0.01 = 0.0095$.

Step 2, evidence: $P(+) = 0.0095 + 0.05 \times 0.99 = 0.0095 + 0.0495 = 0.059$.

Step 3, posterior: $P(D|+) = 0.0095/0.059 \approx 0.161$, only about 16%.

Why so low? Out of 10,000 people, 100 have Covid and 95 of them test positive; of the 9,900 healthy people, 495 test positive. So only 95 of 590 positives are real cases.

EXAMPLE 2: A SECOND TEST (THE POSTERIOR BECOMES THE NEW PRIOR)

The same person takes a second, independent test, which is also positive. Update the probability.

New prior $P(D) = 0.161$ (more exactly $0.0095/0.059 = 0.16102$).

Numerator = $0.95 \times 0.16102 = 0.15297$. Evidence = $0.15297 + 0.05 \times 0.83898 = 0.15297 + 0.04195 = 0.19492$.

$P(D|+, +) = 0.15297/0.19492 \approx 0.785$, now about 78.5%.

Also, a single negative result is very reassuring: $P(D^c|-) = (0.95 \times 0.99)/(0.95 \times 0.99 + 0.05 \times 0.01) = 0.9405/0.941 \approx 0.9995$.

EXAMPLE 3: MONSOON RAIN FORECAST

In Kerala in early June, it rains heavily on 30% of days. The forecast says "heavy rain" on 90% of the days that actually have heavy rain, and on 20% of the days that do not. Today's forecast says "heavy rain". Find $P(\text{heavy rain} | \text{forecast})$.

$P(R) = 0.3$, $P(F|R) = 0.9$, $P(F|R^c) = 0.2$.

$P(F) = 0.9 \times 0.3 + 0.2 \times 0.7 = 0.27 + 0.14 = 0.41$.

$P(R|F) = 0.27/0.41 \approx 0.659$. The forecast raised the probability from 30% to about 66%.

Exam tip: Always write the four quantities (prior, $P(E|H)$, $P(E|H^C)$, $P(H^C)$) first, then compute the evidence with total probability. In Section 7.6 the unit says the probability of F given E "is the same as" the probability of E given F; that is not true in general. They are related by $P(F|E) = P(E|F) P(F)/P(E)$ and are equal only when $P(E) = P(F)$.

Bayes' theorem with many hypotheses (spam filter, factories)

Bayes' theorem (n hypotheses): for mutually exclusive, exhaustive $A_1, \dots, A_n$, $P(A_k|B) = P(A_k) P(B|A_k) / \sum_i P(A_i) P(B|A_i)$.

Naive Bayes: Bayes' theorem with the pieces of evidence assumed conditionally independent given the hypothesis, so $P(H|w_1, \dots, w_n) \propto P(H) \prod_i P(w_i|H)$.

Bayes' theorem (n hypotheses): Mutually exclusive-ഉം exhaustive-ഉം ആയ $A_1, \dots, A_n$ -ന്,
 $P(A_k|B) = P(A_k) P(B|A_k) / \sum_i P(A_i) P(B|A_i)$.

Naive Bayes: Hypothesis നൽകപ്പെട്ടാൽ evidence-കൾ conditionally independent ആണെന്ന് അനുമാനിച്ചുള്ള Bayes' theorem; അതിനാൽ $P(H|w_1, \dots, w_n) \propto P(H) \prod_i P(w_i|H)$.

If $A_1, \dots, A_n$ are mutually exclusive and exhaustive (exactly one is true) and B is observed with $P(B) > 0$, Bayes' theorem gives the posterior of each A_k . The denominator is the same for every k, so the posteriors always add to 1. With several pieces of evidence that are assumed conditionally independent given the hypothesis, their likelihoods simply multiply; this is the idea behind the **naive Bayes** spam filter.

മലയാളത്തിൽ

ഒരു bulb കേടായി; ഇത് മൂന്ന് factory-കളിൽ ഏതിൽ നിന്നാവാം? ഓരോ factory-ക്കും "prior $\times$ likelihood" കണ്ടുപിടിച്ച്, എല്ലാത്തിന്റെയും ആകെത്തുക കൊണ്ട് ഹരിക്കുക. Spam filter-ഉം ഇങ്ങനെ തന്നെ: "free", "lottery" പോലുള്ള വാക്കുകൾ കാണുമ്പോൾ mail spam ആണെന്ന വിശ്വാസം കൂടുന്നു.

$$P(A_k|B) = P(A_k) P(B|A_k) / [P(A_1) P(B|A_1) + P(A_2) P(B|A_2) + \dots + P(A_n) P(B|A_n)]$$

$$\text{Equivalently } P(A_k|B) = P(A_k \cap B) / [P(A_1 \cap B) + \dots + P(A_n \cap B)]$$

$$\text{Naive Bayes: } P(H|w_1, w_2) \propto P(H) P(w_1|H) P(w_2|H)$$

EXAMPLE 1: SPAM FILTER, ONE WORD

40% of e-mails are spam. The word "free" appears in 60% of spam mails and 5% of genuine (ham) mails. A mail contains "free". Find $P(\text{spam}|\text{free})$.

Spam: $0.4 \times 0.6 = 0.24$. Ham: $0.6 \times 0.05 = 0.03$. $P(\text{free}) = 0.27$.

$P(\text{spam}|\text{free}) = 0.24/0.27 \approx 0.889$.

EXAMPLE 2: SPAM FILTER, TWO WORDS (NAIVE BAYES)

Also "lottery" appears in 30% of spam and 1% of ham. A mail contains both "free" and "lottery". Assume the words are independent given the class.

Spam: $0.4 \times 0.6 \times 0.3 = 0.072$. Ham: $0.6 \times 0.05 \times 0.01 = 0.0003$.

$P(\text{spam}|\text{free, lottery}) = 0.072/(0.072 + 0.0003) = 0.072/0.0723 \approx 0.996$. Mark it as spam.

EXAMPLE 3: THREE FACTORIES

A shop gets bulbs from factories F_1, F_2, F_3 in proportions 50%, 30%, 20%. Their defect rates are 2%, 3%, 5%. A bulb is found defective (D). Which factory most likely made it?

Factory	$P(F_k)$	$P(D F_k)$	$P(F_k)P(D F_k)$	$P(F_k D)$
F_1	0.5	0.02	0.010	$0.010/0.029 \approx 0.345$
F_2	0.3	0.03	0.009	$0.009/0.029 \approx 0.310$
F_3	0.2	0.05	0.010	$0.010/0.029 \approx 0.345$
Total $P(D)$			0.029	1.000

F_1 and F_3 are equally likely (10/29 each); F_2 is 9/29. F_3 makes few bulbs but is the worst, so its high defect rate balances its small share.

Exam tip: Draw the table above in the exam: prior, likelihood, product, posterior. The column of posteriors must add to 1; if it does not, you made an arithmetic error.

Bayesian networks

Bayesian network: a directed acyclic graph whose nodes are random variables, whose edges show direct dependence, and where each node stores a CPT $P(X \mid \text{Parents}(X))$.

Conditional independence: A and C are conditionally independent given B if $P(A|B, C) = P(A|B)$, i.e. $P(A, C|B) = P(A|B) P(C|B)$.

Bayesian network: Nodes random variables-ഉം edges നേരിട്ടുള്ള dependence-ഉം ആയ ഒരു directed acyclic graph; ഓരോ node-ഉം $P(X \mid \text{Parents}(X))$ എന്ന CPT സൂക്ഷിക്കുന്നു.

Conditional independence: $P(A|B, C) = P(A|B)$ ആണെങ്കിൽ (അതായത് $P(A, C|B) = P(A|B) P(C|B)$) B നൽകപ്പെട്ടാൽ A-യും C-യും conditionally independent ആണ്.

A full joint distribution over many variables needs far too many numbers and too much data. A **Bayesian (Bayes') network** is a probabilistic graphical model that uses conditional independence to cut this down. It is a directed acyclic graph (DAG): each **node** is a random variable, each **directed edge** shows a direct (usually causal) dependence, and each node stores a **conditional probability table (CPT)** $P(X \mid \text{parents}(X))$. Its benefits: easy visualisation of the model, clear relationships among variables, and efficient computation of complex probabilities.

മലയാളത്തിൽ

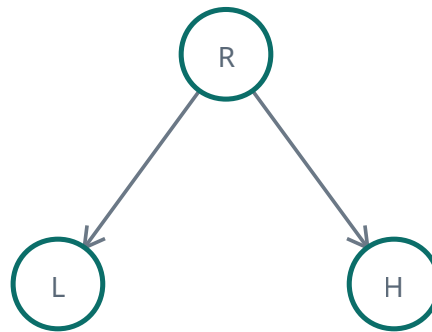
മഴ (R) കാരണം ബസ്‌ലേറ്റ് ആകുന്നു (L), മഴ കാരണം collector അവധി പ്രഖ്യാപിക്കുന്നു (H). ഇത് ഒരു ചെറിയ Bayesian network ആണ്: R-ൽ നിന്ന് L-ലേക്കും H-ലേക്കും arrow. മഴ പെയ്തോ ഇല്ലയോ എന്ന് അറിയാമെങ്കിൽ, ബസ്‌ലേറ്റ് ആയതും അവധി പ്രഖ്യാപിച്ചതും തമ്മിൽ പിന്നെ ബന്ധമില്ല; ഇതാണ് conditional independence. ഓരോ node-ഉം അതിന്റെ parents-നെ മാത്രം ആശ്രയിക്കുന്നതിനാൽ ആവശ്യമുള്ള numbers വളരെ കുറയും.

Joint factorisation: $P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Parents}(X_i))$

Conditional independence of A and C given B: $P(A|B, C) = P(A|B)$ and $P(A, C|B) = P(A|B) P(C|B)$

Unit's Example 8 ($B \rightarrow A, B \rightarrow C$): $P(A, B, C) = P(A|B) \cdot P(C|B) \cdot P(B)$

Numbers needed (Boolean variables): full joint $2^n - 1$; network $\sum_i 2^{(\text{number of parents of } X_i)}$



Same shape as the unit's Fig. 2 ($B \rightarrow A, B \rightarrow C$). R = heavy rain, L = bus late, H = school holiday declared.

EXAMPLE (THE UNIT'S EXAMPLE 8, WITH KERALA NUMBERS)

CPTs for the network above (T = true):

$P(R = T)$	R	$P(L = T R)$	$P(H = T R)$
0.3	T	0.7	0.4
	F	0.1	0.01

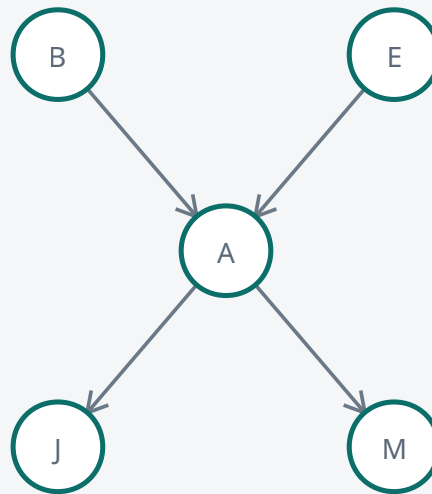
Find $P(R, L, H)$ (all true) and $P(\neg R, L, \neg H)$.

By the factorisation, $P(R, L, H) = P(R) P(L|R) P(H|R) = 0.3 \times 0.7 \times 0.4 = 0.084$.

$P(\neg R, L, \neg H) = P(\neg R) P(L|\neg R) P(\neg H|\neg R) = 0.7 \times 0.1 \times 0.99 = 0.0693$.

Numbers stored: $1 + 2 + 2 = 5$, against $2^3 - 1 = 7$ for the full joint. (All 8 joint entries computed this way add to 1.)

EXAMPLE (CLASSIC BURGLAR ALARM NETWORK)



B = burglary, E = earthquake, A = alarm rings, J = neighbour John calls, M = neighbour Mary calls.

Node	CPT (probability of True)
B	$P(B) = 0.001$
E	$P(E) = 0.002$
A	$P(A B, E) = 0.95, P(A B, \neg E) = 0.94, P(A \neg B, E) = 0.29, P(A \neg B, \neg E) = 0.001$
J	$P(J A) = 0.90, P(J \neg A) = 0.05$
M	$P(M A) = 0.70, P(M \neg A) = 0.01$

Find the probability that the alarm rings and both John and Mary call, but there is no burglary and no earthquake.

$$\begin{aligned} P(J, M, A, \neg B, \neg E) &= P(J|A) P(M|A) P(A|\neg B, \neg E) P(\neg B) P(\neg E) \\ &= 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998 \approx 0.000628. \end{aligned}$$

Numbers stored: $1 + 1 + 4 + 2 + 2 = 10$, against $2^5 - 1 = 31$ for the full joint.

Exam tip: To write the joint of any network, go node by node and write $P(\text{node} \mid \text{its parents})$, then multiply. Root nodes (no parents) contribute their prior. In Example 8 the unit calls $P(A|B, C)$ and $P(C|B, A)$ "conditional independence"; the actual statements are $P(A|B, C) = P(A|B)$ and $P(C|B, A) = P(C|B)$.

Probabilistic inference with Bayesian networks

Probabilistic inference: computing the probability of a query variable Q given observed evidence e , $P(Q | e)$, e.g. $P(\text{cause} | \text{effect})$.

Inference by enumeration: $P(Q | e) = \alpha \sum_y P(Q, e, y)$, where each joint term comes from the CPT factorisation and hidden variables y are summed out.

Probabilistic inference: നിരീക്ഷിച്ച evidence e നൽകപ്പെട്ടാൽ query variable Q -ന്റെ probability $P(Q | e)$ കണക്കാക്കൽ (ഉദാ. $P(\text{cause} | \text{effect})$).

Inference by enumeration: $P(Q | e) = \alpha \sum_y P(Q, e, y)$; ഓരോ joint term-ഉം CPT factorisation-ൽ നിന്ന് കിട്ടുന്നു, hidden variables y -യെ sum ചെയ്ത് ഒഴിവാക്കുന്നു.

Probabilistic inference means computing the probability of a query variable given observed evidence, e.g. $P(\text{cause} | \text{effect})$. In a network, the given probabilities are propagated along the edges to the conclusion node. The simplest method is **inference by enumeration**: write the query as a ratio of joint probabilities, get each joint from the factorisation, and sum out the variables that are neither query nor evidence. To build such a system you must: define the sample space, assign prior and conditional probabilities to all basic facts, choose a method for combining evidence along the inference chain, and finally compare alternative conclusions and pick the most likely one.

മലയാളത്തിൽ

Network ഉണ്ടാക്കിക്കഴിഞ്ഞാൽ ചോദ്യങ്ങൾ ചോദിക്കാം: "ബസ് late ആയി; മഴ പെയ്തിട്ടുണ്ടാകാനുള്ള സാധ്യത എത്ര?" Effect കണ്ടിട്ട് cause കണ്ടുപിടിക്കുന്നത് diagnostic inference, cause-ൽ നിന്ന് effect പ്രവചിക്കുന്നത് causal (predictive) inference. ഉത്തരം കാണാൻ ആവശ്യമായ joint probabilities factorisation വെച്ച് കണക്കാക്കി, അറിയാത്ത variables-നെ sum ചെയ്ത് കളയുന്നു.

$P(Q | e) = P(Q, e) / P(e) = \alpha \cdot \sum_{\text{hidden } y} P(Q, e, y)$, $\alpha = 1/P(e)$
Each $P(Q, e, y)$ = product of CPT entries (the factorisation)

EXAMPLE 1: DIAGNOSTIC INFERENCE (RAIN NETWORK OF THE PREVIOUS TOPIC)

The bus is late. Find $P(R | L)$.

$P(R, L) = P(R) P(L|R) = 0.3 \times 0.7 = 0.21$ (H is summed out: $P(H|R) + P(\neg H|R) = 1$).

$P(\neg R, L) = 0.7 \times 0.1 = 0.07$. So $P(L) = 0.28$.

$P(R|L) = 0.21/0.28 = 0.75$.

EXAMPLE 2: TWO PIECES OF EVIDENCE

The bus is late and a holiday has been declared. Find $P(R \mid L, H)$.

$$P(R, L, H) = 0.3 \times 0.7 \times 0.4 = 0.084. \quad P(\neg R, L, H) = 0.7 \times 0.1 \times 0.01 = 0.0007.$$

$P(R \mid L, H) = 0.084 / (0.084 + 0.0007) = 0.084 / 0.0847 \approx 0.992$. Both effects together make rain almost certain.

EXAMPLE 3: PREDICTING ANOTHER EFFECT THROUGH THE CAUSE

Only the bus is late. Find $P(H \mid L)$.

H and L are linked only through R, so $P(H \mid L) = P(H \mid R) P(R \mid L) + P(H \mid \neg R) P(\neg R \mid L) = 0.4 \times 0.75 + 0.01 \times 0.25 = 0.3 + 0.0025 = 0.3025$.

Compare $P(H) = 0.3 \times 0.4 + 0.7 \times 0.01 = 0.127$. Seeing a late bus more than doubles the chance of a holiday, even though neither causes the other. But once R is known, L gives no more information: $P(H \mid L, R) = P(H \mid R) = 0.4$.

EXAMPLE 4: MARGINAL IN THE ALARM NETWORK

Find $P(A)$, the probability the alarm rings.

$$\begin{aligned} \text{Sum over B and E: } P(A) &= 0.001 \times 0.002 \times 0.95 + 0.001 \times 0.998 \times 0.94 + 0.999 \times 0.002 \times 0.29 + 0.999 \times \\ & 0.998 \times 0.001 \\ &= 0.0000019 + 0.00093812 + 0.00057942 + 0.00099700 \approx 0.002516. \end{aligned}$$

$$\text{Then } P(J) = 0.90 \times 0.002516 + 0.05 \times 0.997484 \approx 0.0521.$$

Exam tip: For a diagnostic question, compute $P(\text{query} = T, \text{evidence})$ and $P(\text{query} = F, \text{evidence})$ separately, then normalise. Show the factorisation line in your answer; that is where most marks are.

Other paradigms: MYCIN, certainty factors, heuristic reasoning

MYCIN: a rule-based expert system for diagnosing meningitis and infectious blood diseases that measures both belief and disbelief in a hypothesis.

Certainty factor: $CF(H, E) = MB(H, E) - MD(H, E)$, from -1 (definitely false) to +1 (definitely true).

Heuristic reasoning: choosing among alternative conclusions by using rules of thumb to weigh positive against negative evidence.

MYCIN: Meningitis-ഉം infectious blood diseases-ഉം diagnose ചെയ്യാനുള്ള ഒരു rule-based expert system; ഒരു hypothesis-ലുള്ള belief-ഉം disbelief-ഉം അത് അളക്കുന്നു.

Certainty factor: $CF(H, E) = MB(H, E) - MD(H, E)$; മൂല്യം -1 (തീർച്ചയായും false) മുതൽ +1 (തീർച്ചയായും true) വരെ.

Heuristic reasoning: അനുകൂലവും പ്രതികൂലവുമായ evidence തൂക്കിനോക്കാൻ rules of thumb ഉപയോഗിച്ച് പല നിഗമനങ്ങളിൽ നിന്ന് ഒന്ന് തിരഞ്ഞെടുക്കൽ.

Many practical systems use **ad hoc** methods without a full probabilistic foundation, because reliable probabilities are hard to obtain and the calculations get complex. The best known is **MYCIN**, an expert system for diagnosing meningitis and infectious blood diseases. It uses IF-THEN rules and measures both **belief** (MB) and **disbelief** (MD) in a hypothesis, combined into a **certainty factor** CF from -1 (definitely false) to +1 (definitely true). A second paradigm is **heuristic reasoning**: domain-specific and general rules of thumb choose among alternative conclusions by weighing positive against negative evidence, presented as justifications or endorsements.

മലയാളത്തിൽ

ഒരു പരിചയസമ്പന്നനായ doctor "ഇത് typhoid ആകാനാണ് സാധ്യത കൂടുതൽ" എന്ന് പറയുന്നത് കൃത്യമായ probability കണക്കാക്കിയല്ല; അനുഭവത്തിൽ നിന്നുള്ള rules വെച്ചാണ്. MYCIN ഇതുപോലെ IF-THEN rules ഉപയോഗിച്ച്, ഓരോ hypothesis-നും അനുകൂലമായ വിശ്വാസവും (MB) പ്രതികൂലമായ അവിശ്വാസവും (MD) അളക്കുന്നു. രണ്ടിന്റെയും വ്യത്യാസമാണ് certainty factor.

$$CF(H, E) = MB(H, E) - MD(H, E), -1 \leq CF \leq 1$$

Rule with premise: $CF(H) = CF(\text{rule}) \times CF(\text{premise})$; for an AND premise use min, for OR use max

Combining two CFs for the same H:

$$\text{both} > 0: CF = CF_1 + CF_2(1 - CF_1)$$

$$\text{both} < 0: CF = CF_1 + CF_2(1 + CF_1)$$

$$\text{opposite signs: } CF = (CF_1 + CF_2) / (1 - \min(|CF_1|, |CF_2|))$$

EXAMPLE

Rule 1: IF fever AND rash THEN measles (CF 0.7). Observed: fever with CF 0.8, rash with CF 0.6. Rule 2 gives independent evidence against measles with CF -0.3. Find the final CF of measles.

Rule 1: $CF = 0.7 \times \min(0.8, 0.6) = 0.7 \times 0.6 = 0.42$.

Combine 0.42 with -0.3 (opposite signs): $(0.42 - 0.3)/(1 - \min(0.42, 0.3)) = 0.12/0.7 \approx 0.171$.

A weak positive belief remains in measles.

EXAMPLE

Two rules support dengue with CF 0.6 and 0.5. Combined CF?

$0.6 + 0.5 \times (1 - 0.6) = 0.6 + 0.2 = 0.8$. Two supporting rules give more confidence than either alone, but never beyond 1.

Exam tip: The CF formulas are not in the IGNOU text (it only says MYCIN "measures both belief and disbelief"); learn them as standard supporting material. In the unit, "heuristic reasoning ... under certainty" should read "under uncertainty".

Dempster-Shafer theory and types of evidence

Dempster-Shafer theory: a theory of evidence that generalises Bayesian probability by assigning belief to sets of hypotheses, not only to single hypotheses.

Evidence: any event related to a hypothesis or a set of hypotheses; it may be consonant, consistent, arbitrary or disjoint.

Dempster-Shafer theory: ഒരു hypothesis-ന്മാത്രമല്ല, hypotheses-ന്റെ sets-നും belief നൽകി Bayesian probability-യെ generalise ചെയ്യുന്ന ഒരു theory of evidence.

Evidence: ഒരു hypothesis-മായോ hypotheses-ന്റെ set-മായോ ബന്ധപ്പെട്ട ഏതൊരു event-ഉം; ഇത് consonant, consistent, arbitrary അല്ലെങ്കിൽ disjoint ആകാം.

Dempster-Shafer (D-S) theory, proposed by Dempster and extended by Shafer in "A Mathematical Theory of Evidence", reasons from evidence alone. It generalises Bayesian probability: belief can be assigned to **sets** of hypotheses, not only to single ones, so "I don't know which" can be expressed directly. The hypotheses in the frame must be exhaustive and mutually exclusive. Independent sources are combined with Dempster's rule, which becomes costly when many sources must be assumed independent. **Evidence** is any event related to a hypothesis or set of hypotheses; the unit lists four types.

മലയാളത്തിൽ

Bayes-ൽ ഓരോ hypothesis-നും കൃത്യമായ probability കൊടുക്കണം. പക്ഷേ ചിലപ്പോൾ നമുക്ക് പറയാനാകുന്നത് "ഇത് dengue-വോ malaria-യോ ആണ്, ഏതാണെന്ന് അറിയില്ല" എന്ന് മാത്രമാണ്. D-S theory-യിൽ ഇങ്ങനെ ഒരു set-ന് തന്നെ belief കൊടുക്കാം. തെളിവുകൾ കൂടുന്തോറും ignorance കുറയുന്നു.

Evidence type	Meaning	Example sets
Consonant	Nested: each subset is inside the next larger one	$\{a\} \subset \{a, b\} \subset \{a, b, c\}$
Consistent	At least one element common to all subsets	$\{a, b\}, \{a, c\}, \{a, d\}$ (a is common)
Arbitrary	No element common to all, but some subsets overlap	$\{a, b\}, \{b, c\}, \{c, d\}$
Disjoint	No subset shares an element with another	$\{a\}, \{b\}, \{c\}$

EXAMPLE (CHECK YOUR PROGRESS 3, Q3)

Three witnesses describe a thief's vehicle (bike b, car c, auto-rickshaw r, van v). Classify their evidence.

"It was a car" / "a car or a van" / "a car, van or bike": $\{c\} \subset \{c, v\} \subset \{b, c, v\}$, nested, so **consonant**.

"car or van" / "car or bike" / "car or auto": c is in every set, so **consistent**.

"bike or car" / "car or auto" / "auto or van": neighbours overlap but nothing is in all three, so **arbitrary**.

"bike" / "car" / "auto": no overlaps, so **disjoint**.

Advantages of D-S (as listed in the unit): uncertainty reduces as information is added; more evidence reduces ignorance; diagnostic hierarchies can be represented. Sources are assumed non-biased, and no extra assumptions are needed to represent information of varying precision.

Frame of discernment, bpa, belief and plausibility

Frame of discernment: $\theta = \{\theta_1, \dots, \theta_n\}$, the set of mutually exclusive, exhaustive possible values (hypotheses).

Basic probability assignment (bpa): a function $m: 2^\theta \rightarrow [0, 1]$ with $m(\emptyset) = 0$ and $\sum_{A \subseteq \theta} m(A) = 1$.

Belief / Plausibility: $\text{Bel}(A) = \sum_{B \subseteq A} m(B)$ is the lower bound; $\text{Pl}(A) = \sum_{B \cap A \neq \emptyset} m(B) = 1 - \text{Bel}(A^c)$ is the upper bound.

Frame of discernment: $\theta = \{\theta_1, \dots, \theta_n\}$; mutually exclusive-ഉം exhaustive-ഉം ആയ സാധ്യമായ മൂല്യങ്ങളുടെ (hypotheses) set.

Basic probability assignment (bpa): $m(\emptyset) = 0$, $\sum_{A \subseteq \theta} m(A) = 1$ എന്നിവ പാലിക്കുന്ന $m: 2^\theta \rightarrow [0, 1]$ എന്ന function.

Belief / Plausibility: $\text{Bel}(A) = \sum_{B \subseteq A} m(B)$ lower bound ആണ്; $\text{Pl}(A) = \sum_{B \cap A \neq \emptyset} m(B) = 1 - \text{Bel}(A^c)$ upper bound ആണ്.

The **frame of discernment** $\theta = \{\theta_1, \dots, \theta_n\}$ is the set of mutually exclusive, exhaustive possible values. Its **power set** 2^θ contains all 2^n subsets. A **basic probability assignment** (bpa, mass function) m gives each subset a mass; subsets with $m(A) > 0$ are **focal elements**. From m we get a lower bound, **belief** $\text{Bel}(A)$ (total mass that surely supports A), and an upper bound, **plausibility** $\text{Pl}(A)$ (total mass that does not contradict A). The gap $\text{Pl}(A) - \text{Bel}(A)$ is the **uncertainty interval**, where the true probability may lie.

മലയാളത്തിൽ

$\text{Bel}(A)$ എന്നാൽ A -യെ ഉറപ്പായും support ചെയ്യുന്ന mass: A -യുടെ ഉള്ളിലുള്ള sets-ന്റെ mass മുഴുവൻ കൂട്ടുക. $\text{Pl}(A)$ എന്നാൽ A -ക്ക് എതിരല്ലാത്ത mass: A -യുമായി എന്തെങ്കിലും common ഉള്ള sets-ന്റെ mass മുഴുവൻ കൂട്ടുക. യഥാർത്ഥ probability ഈ രണ്ടിനും ഇടയിൽ എവിടെയോ ആണ്. ഇടവേള വലുതാണെങ്കിൽ നമ്മുടെ അറിവില്ലായ്മ കൂടുതലാണ്.

$$m: 2^\theta \rightarrow [0, 1], m(\emptyset) = 0, m(A) \geq 0, \sum_{A \subseteq \theta} m(A) = 1$$

$$\text{Bel}(A) = \sum_{B \subseteq A} m(B)$$

$$\text{Pl}(A) = \sum_{B \cap A \neq \emptyset} m(B) = 1 - \text{Bel}(A^c)$$

$$\text{Bel}(A) \leq P(A) \leq \text{Pl}(A); \text{uncertainty interval} = \text{Pl}(A) - \text{Bel}(A)$$

EXAMPLE (THE UNIT'S EXAMPLE 9, CORRECTED)

$\theta = \{a, b, c\}$. The power set has $2^3 = 8$ members: $\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}$. A source gives $m(a) = 0.2, m(c) = 0.1, m(\{a, b\}) = 0.4$.

These add to 0.7, not 1. The remaining mass represents ignorance and goes to the whole frame: $m(\theta) = 0.3$.

Focal elements: $\{a\}, \{c\}, \{a, b\}, \theta$.

A	Bel(A)	Pl(A)	Interval
$\{a\}$	0.2	$0.2 + 0.4 + 0.3 = 0.9$	$[0.2, 0.9]$
$\{b\}$	0	$0.4 + 0.3 = 0.7$	$[0, 0.7]$
$\{c\}$	0.1	$0.1 + 0.3 = 0.4$	$[0.1, 0.4]$
$\{a, b\}$	$0.2 + 0.4 = 0.6$	$0.2 + 0.4 + 0.3 = 0.9$	$[0.6, 0.9]$

Check: $Pl(\{a, b\}) = 1 - Bel(\{c\}) = 1 - 0.1 = 0.9$. ✓

Exam tip: Three corrections to the unit's text. (1) Example 9's masses add to only 0.7; a valid bpa must total 1, so assign the remaining 0.3 to θ . (2) $Bel(A)$ sums $m(B)$ over all subsets $B \subseteq A$, including A itself, not only proper subsets (the unit's own Example 10 uses $Bel(A) = m(A)$). (3) Plausibility measures how much evidence does not contradict A ; the unit's wording "contradicting" is reversed.

Dempster's rule of combination

Definition: Dempster's rule combines two independent bpas: $m_{12}(A) = \sum_{B \cap C = A} m_1(B) m_2(C) / (1 - K)$ for $A \neq \emptyset$, where K is the total mass falling on $\emptyset$ (conflict).

Dempster's rule രണ്ട് independent bpas-നെ ഒന്നിപ്പിക്കുന്നു: $A \neq \emptyset$ -ന് $m_{12}(A) = \sum_{B \cap C = A} m_1(B) m_2(C) / (1 - K)$; ഇവിടെ K എന്നത് $\emptyset$ -ൽ വീഴുന്ന ആകെ mass (conflict) ആണ്.

Two independent sources with masses m_1 and m_2 are combined by a conjunctive (AND) rule: multiply every pair of masses and give the product to the intersection of the two sets. Products that fall on $\emptyset$ are **conflict**, totalled as K , and the rest is renormalised by dividing by $1 - K$. The rule is associative and commutative, but not idempotent or continuous.

മലയാളത്തിൽ

രണ്ട് doctors-ന്റെ അഭിപ്രായം ഒന്നിപ്പിക്കണം എന്ന് കരുതുക. ഒരു table ഉണ്ടാക്കി, ഒരാളുടെ ഓരോ mass-ഉം മറ്റേയാളുടെ ഓരോ mass-ഉം ഗുണിച്ച്, ആ product രണ്ട് sets-ന്റെയും intersection-ന് കൊടുക്കുക. Intersection ശൂന്യമാണെങ്കിൽ (ഒരാൾ "typhoid", മറ്റേയാൾ "dengue") അത് conflict K. അവസാനം ബാക്കിയുള്ളതിനെക്കുറിച്ചെല്ലാം $1 - K$ കൊണ്ട് ഹരിക്കുക.

$$m_{12}(A) = [\sum_{B \cap C = A} m_1(B) m_2(C)] / (1 - K), \text{ for } A \neq \emptyset$$

$$m_{12}(\emptyset) = 0$$

$$K = \sum_{B \cap C = \emptyset} m_1(B) m_2(C)$$

EXAMPLE (THE UNIT'S EXAMPLE 10)

100 candidates; $\theta = \{A, B\}$ (grade A or B). Board 1 saw 60: 40 get A, 20 get B, 40 unassessed. Board 2 saw 60: 30 get A, 30 "A or B", 40 unassessed.

$$m_1(A) = 0.4, m_1(B) = 0.2, m_1(\theta) = 0.4; m_2(A) = 0.3, m_2(\{A, B\}) = 0.3, m_2(\theta) = 0.4.$$

Individual plausibilities: $Pl_1(A) = 0.4 + 0.4 = 0.8$, $Pl_1(B) = 0.2 + 0.4 = 0.6$, $Pl_2(A) = 0.3 + 0.3 + 0.4 = 1.0$ (every focal element of m_2 contains A), $Pl_1(\theta) = Pl_2(\theta) = 1.0$.

$m_2 \setminus m_1$	A: 0.4	B: 0.2	θ : 0.4
A: 0.3	A: 0.12	$\emptyset$: 0.06	A: 0.12
{A, B}: 0.3	A: 0.12	B: 0.06	{A, B}: 0.12
θ : 0.4	A: 0.16	B: 0.08	θ : 0.16

$$K = 0.06, 1 - K = 0.94.$$

$$m_{12}(A) = (0.12 + 0.12 + 0.12 + 0.16)/0.94 = 0.52/0.94 \approx 0.553.$$

$$m_{12}(B) = (0.06 + 0.08)/0.94 = 0.14/0.94 \approx 0.149.$$

$m_{12}(\{A, B\}) = 0.12/0.94 \approx 0.128$; $m_{12}(\theta) = 0.16/0.94 \approx 0.170$. Since $\{A, B\} = \theta$ here, their total 0.298 is the uncommitted mass.

$$Bel(A) = 0.553, Pl(A) = 0.553 + 0.128 + 0.170 = 0.851. Bel(B) = 0.149, Pl(B) = 0.149 + 0.128 + 0.170 = 0.447.$$

Conclusion: about 55 to 85 candidates get grade A and 15 to 45 get grade B.

EXAMPLE (FEVER DIAGNOSIS)

$\theta = \{D, M, T\}$ (dengue, malaria, typhoid). A blood smear suggests a mosquito-borne illness: $m_1(\{D, M\}) = 0.6$, $m_1(\theta) = 0.4$. A doctor's examination gives $m_2(D) = 0.5$, $m_2(T) = 0.2$, $m_2(\theta) = 0.3$. Combine them.

$m_1 \setminus m_2$	D: 0.5	T: 0.2	θ : 0.3
{D, M}: 0.6	D: 0.30	$\emptyset$: 0.12	{D, M}: 0.18
θ : 0.4	D: 0.20	T: 0.08	θ : 0.12

$K = 0.12$, $1 - K = 0.88$.

$m_{12}(D) = 0.50/0.88 \approx 0.568$; $m_{12}(\{D, M\}) = 0.18/0.88 \approx 0.205$; $m_{12}(T) = 0.08/0.88 \approx 0.091$; $m_{12}(\theta) = 0.12/0.88 \approx 0.136$. (Sum = 1.)

$\text{Bel}(D) = 0.568$, $\text{Pl}(D) = 0.568 + 0.205 + 0.136 \approx 0.909$. $\text{Bel}(T) = 0.091$, $\text{Pl}(T) = 0.091 + 0.136 \approx 0.227$.

$\text{Bel}(M) = 0$, $\text{Pl}(M) = 0.205 + 0.136 \approx 0.341$.

Dengue is the strongest diagnosis: belief interval [0.57, 0.91].

Exam tip: In Example 10, $\theta = \{A, B\}$, so $m_2(\{A, B\})$ and $m_2(\theta)$ are really the same set (total 0.7). The unit keeps them as separate columns, which leads it to write $\text{Bel}_{1-2}(A, B) = 0.83$; since $\{A, B\} = \theta$, the correct value is $\text{Bel}(\{A, B\}) = \text{Bel}(\theta) = 1$. The final ranges (55 to 85 for A, 15 to 45 for B) are unaffected. Also the unit's $\text{Pl}_2(A)$ wrongly omits $m_2(\{A, B\})$; it is 1.0, not 0.7.

Quick recap

- Probabilistic reasoning replaces True/False with degrees of likelihood in $[0, 1]$ to decide under uncertainty.
- Axioms: $0 \leq P(E) \leq 1$, $P(S) = 1$, additivity for mutually exclusive events.
- Sample space = all outcomes; event = subset; E^c , $E \cup F$, $E \cap F$; mutually exclusive means $E \cap F = \emptyset$.
- Relative frequency $r_n(E) = f_n(E)/n$ approaches $P(E)$ for large n .
- $P(E|F) = P(E \cap F)/P(F)$, $P(F) > 0$; $P(E \cap F) = P(E|F) P(F)$.
- Independent: $P(E \cap F) = P(E) P(F)$. Mutually exclusive events (non-zero probability) are dependent.
- Total probability: $P(B) = \sum P(A_i) P(B|A_i)$.
- Bayes: $P(H|E) = P(E|H) P(H)/P(E)$; posterior = likelihood $\times$ prior / evidence.
- Low prior means a positive test can still give a low posterior (1% prevalence, 95%/5% test gives about 16%).
- Bayesian network: DAG + CPTs; $P(X_1, \dots, X_n) = \prod P(X_i | \text{Parents}(X_i))$.
- Inference by enumeration: $P(Q|e) = P(Q, e)/P(e)$, summing out hidden variables.
- MYCIN: IF-THEN rules with certainty factors $CF = MB - MD$ in $[-1, 1]$.
- D-S theory: masses on subsets of the frame θ ; $\text{Bel}(A) = \sum_{B \subseteq A} m(B)$, $\text{Pl}(A) = \sum_{B \cap A \neq \emptyset} m(B)$.

- Dempster's rule: multiply masses, assign to intersections, drop conflict K, divide by $1 - K$.

Check yourself

Q. Three students each toss a coin together, and they do this 3 times. How many outcomes does the whole experiment have? In one round, what is the probability of more than 2 heads, of 2 or more heads, and of three tails?

One round (three coins at once) has $2^3 = 8$ outcomes: HHH, HHT, HTH, THH, HTT, THT, TTH, TTT.
Three rounds: $8^3 = 512$ outcomes.

More than 2 heads (i.e. 3 heads): only HHH, so $1/8$. Two or more heads: HHH, HHT, HTH, THH, so $4/8 = 1/2$. Three tails: TTT, so $1/8$.

The unit's question is garbled ("2 more than 2 heads"); both readings are answered here.

Q. In the experiment above, the 9 tosses gave T, H, T, T, T, H, H, T, H in order. Find $r_n(T)$ after each round of three tosses.

Round 1 (T, H, T): $f_3(T) = 2$, $r_3(T) = 2/3$.

Round 2 (T, T, H): $f_6(T) = 4$, $r_6(T) = 4/6 = 2/3$.

Round 3 (H, T, H): $f_9(T) = 5$, $r_9(T) = 5/9 \approx 0.556$. With more tosses it would approach $1/2$.

Q. A card is drawn from a well-shuffled pack of 52. Given that it is a face card (J, Q, K), what is the probability it is a king?

$P(\text{face}) = 12/52$, $P(\text{king} \cap \text{face}) = 4/52$.

$P(\text{king}|\text{face}) = (4/52)/(12/52) = 4/12 = 1/3$.

Q. A disease affects 2 in 1000 people. A test is 99% sensitive and has a 2% false positive rate. Find $P(\text{disease} | \text{positive})$.

$P(D) = 0.002$, $P(+|D) = 0.99$, $P(+|D^c) = 0.02$.

Numerator = $0.99 \times 0.002 = 0.00198$. $P(+)$ = $0.00198 + 0.02 \times 0.998 = 0.00198 + 0.01996 = 0.02194$.

$P(D|+) = 0.00198/0.02194 \approx 0.090$, only about 9%, because the disease is rare.

Q. Differentiate joint, marginal and conditional probability with an example of each.

Joint $P(A, B)$: both together, e.g. $P(\text{it rains and the bus is late}) = 0.3 \times 0.7 = 0.21$.

Marginal $P(A)$: one variable regardless of others, e.g. $P(\text{bus late}) = 0.21 + 0.07 = 0.28$ (summing the joint over rain/no rain).

Conditional $P(A|B)$: one given the other, e.g. $P(\text{rain} | \text{bus late}) = 0.21/0.28 = 0.75$.

Q. Write the joint probability for a network where A has no parents, B and C have parent A, and D has parents B and C. How many numbers does it need (all Boolean)?

$P(A, B, C, D) = P(A) \cdot P(B|A) \cdot P(C|A) \cdot P(D|B, C)$.

Numbers: $1 + 2 + 2 + 4 = 9$, against $2^4 - 1 = 15$ for the full joint.

Q. In the rain network ($P(R) = 0.3$, $P(L|R) = 0.7$, $P(L|\neg R) = 0.1$, $P(H|R) = 0.4$, $P(H|\neg R) = 0.01$), find $P(R|H)$.

$P(R, H) = 0.3 \times 0.4 = 0.12$. $P(\neg R, H) = 0.7 \times 0.01 = 0.007$. $P(H) = 0.127$.

$P(R|H) = 0.12/0.127 \approx 0.945$.

Q. Two MYCIN rules give CF 0.7 and CF 0.4 for the same hypothesis. A third gives CF -0.5. Find the final CF.

Combine the positives: $0.7 + 0.4 \times (1 - 0.7) = 0.7 + 0.12 = 0.82$.

Combine with -0.5 (opposite signs): $(0.82 - 0.5)/(1 - \min(0.82, 0.5)) = 0.32/0.5 = 0.64$.

Q. Explain Dempster-Shafer theory with an example: $\theta = \{x, y\}$, $m_1(x) = 0.6$, $m_1(\theta) = 0.4$; $m_2(y) = 0.5$, $m_2(\theta) = 0.5$. Combine and find Bel and Pl of x .

D-S theory assigns mass to subsets of the frame, gives each set a belief-plausibility interval, and fuses independent sources with Dempster's rule.

Products: $x \cap y = \emptyset$: 0.30 (conflict); $x \cap \theta = x$: 0.30; $\theta \cap y = y$: 0.20; $\theta \cap \theta = \theta$: 0.20. $K = 0.30$, $1 - K = 0.70$.

$m_{12}(x) = 0.30/0.70 \approx 0.429$, $m_{12}(y) = 0.20/0.70 \approx 0.286$, $m_{12}(\theta) = 0.20/0.70 \approx 0.286$.

$\text{Bel}(x) \approx 0.429$, $\text{Pl}(x) = 0.429 + 0.286 \approx 0.714$. So $P(x)$ lies in $[0.43, 0.71]$.

UNIT 8

Decision Making

ഒരു AI agent പല വഴികളിൽ നിന്ന് ഏറ്റവും നല്ലത് എങ്ങനെ തിരഞ്ഞെടുക്കുന്നു എന്നതാണ് ഈ unit. Utility, expected utility, payoff matrix, EMV, Markov Decision Process, value iteration, game theory, Nash equilibrium എന്നിവ ഓരോന്നും കണക്കുകൾ സഹിതം നമ്മൾ പഠിക്കുന്നു.

Utility theory and the utility function

The six axioms of utility theory

Maximum Expected Utility (MEU) principle

Risk attitude, certainty equivalent and risk premium

Decision theory: components and the payoff matrix

Decision under certainty and uncertainty: maximin and maximax

Decision under risk: Expected Monetary Value (EMV)

Decision trees

Do humans follow utility theory? (Tversky-Kahneman)

Sequential decision problems

Markov Decision Process (MDP) and optimal policy

Bellman equation and the value iteration algorithm

Value iteration: the unit's two-room robot (Example 5)

Value iteration with probabilistic transitions

POMDPs, applications and challenges

Game theory: introduction and key concepts

Normal-form and extensive-form games

Nash equilibrium

Sample applications

Utility theory and the utility function

Utility: a number that measures how desirable an outcome (state) is to an agent.

Utility function: a function U that maps each state S of the environment to a real number $U(S)$; the agent prefers states of higher utility.

Utility: ഒരു outcome (state) ഒരു agent-ന് എത്രത്തോളം അഭികാമ്യമാണ് എന്ന് അളക്കുന്ന ഒരു സംഖ്യ.

Utility function: Environment-ലെ ഓരോ state S -നെയും $U(S)$ എന്ന real number-ലേക്ക് map ചെയ്യുന്ന function U ; കൂടുതൽ utility ഉള്ള states-നാണ് agent ഇഷ്ടപ്പെടുന്നത്.

An AI agent must choose between actions whose results are uncertain. **Utility theory** gives a mathematical way to do this: every outcome (state) is given a number, its **utility**, that measures how much the agent likes it. The **utility function** U maps each state S of the environment to a real number $U(S)$. When the agent acts, the state changes, and the agent prefers actions that lead to states of higher utility.

മലയാളത്തിൽ

Utility എന്നാൽ "എനിക്ക് ഇത് എത്രത്തോളം ഇഷ്ടമാണ്" എന്നതിന് ഒരു number കൊടുക്കലാണ്. ഒരേ കാര്യത്തിന് രണ്ടുപേർക്ക് വ്യത്യസ്ത utility ആകാം: ഗൾഫിൽ നിന്ന് വന്ന ഒരാൾക്ക് ₹1000 ചെറിയ തുകയാണ്, ഒരു വിദ്യാർത്ഥിക്ക് അത് വലിയ തുകയാണ്. Self-driving car-ൽ utility function traffic, safety, fuel എന്നിവയെല്ലാം ചേർത്ത് ഒരു score ഉണ്ടാക്കുന്നു, അതനുസരിച്ച് action തിരഞ്ഞെടുക്കുന്നു.

$U(S)$ = a real number measuring how desirable state S is

Utility principle: $U(A) > U(B) \Rightarrow$ the agent prefers A to B

$U(A) = U(B) \Rightarrow$ the agent is indifferent between A and B

EXAMPLE (UNIT'S EXAMPLE 1, FIRST LOOK)

A gambler can take (i) a sure prize of Rs.1000, or (ii) a 50% chance of Rs.1500 and a 50% chance of nothing. Which is better?

If we naively take "utility = the biggest money you can get", move (i) has utility 1000 and move (ii) has utility 1500, so move (ii) looks better. This ignores the 50% chance of getting nothing. The next topics fix this using probabilities (expected utility), which gives 750 for move (ii), so move (i) is actually better.

EXAMPLE

A route planner scores routes from Kochi to Thrissur: $U = 100 - 2 \times (\text{minutes over 60}) - 20 \times (\text{risk level})$.
NH route: 70 minutes, risk 1. Ghat-free inner road: 80 minutes, risk 0. Which route?

$U(\text{NH}) = 100 - 2 \times 10 - 20 \times 1 = 60$. $U(\text{inner}) = 100 - 2 \times 20 - 0 = 60$. $U(\text{NH}) = U(\text{inner})$, so this agent is indifferent. A more risk-averse agent (-30 per risk level) gets $U(\text{NH}) = 50 < 60$ and prefers the inner road: the utility function encodes the agent's risk appetite.

The six axioms of utility theory

Definition: The axioms of utility theory are six constraints that a rational agent's preferences must satisfy: completeness, transitivity, continuity, substitutability, monotonicity and decomposability; if they hold, a utility function exists.

ഒരു rational agent-ന്റെ preferences പാലിക്കേണ്ട ആറ് constraints ആണ് utility theory-യുടെ axioms: completeness, transitivity, continuity, substitutability, monotonicity, decomposability; ഇവ പാലിക്കപ്പെട്ടാൽ ഒരു utility function നിലവിലുണ്ട്.

The axioms are constraints that any rational agent's preferences must satisfy. They do not tell us the utility numbers; they guarantee that if preferences obey them, a utility function exists and the agent behaves as if it maximises expected utility. Write $A \succ B$ for "A is preferred to B", $A \sim B$ for "indifferent", and $[p, A; 1-p, C]$ for a lottery giving A with probability p and C otherwise.

മലയാളത്തിൽ

ഈ ആറ് axioms "ബുദ്ധിയുള്ള ഒരാളുടെ ഇഷ്ടങ്ങൾ എങ്ങനെയിരിക്കണം" എന്ന നിയമങ്ങളാണ്. ഉദാഹരണത്തിന് transitivity: ചായയേക്കാൾ കാപ്പിയും, കാപ്പിയേക്കാൾ ജ്യൂസും ഇഷ്ടമാണെങ്കിൽ ചായയേക്കാൾ ജ്യൂസ് ഇഷ്ടപ്പെടണം. അല്ലെങ്കിൽ ഒരു കച്ചവടക്കാരന് നിങ്ങളെ ചുറ്റിച്ച് പണം തട്ടാം.

- A. Completeness: for any A, B exactly one holds: $A \succ B$, $B \succ A$, or $A \sim B$
- B. Transitivity: $(A \succ B) \wedge (B \succ C) \Rightarrow (A \succ C)$
- C. Continuity: $A \succ B \succ C \Rightarrow \exists p$ such that $[p, A; 1-p, C] \sim B$
- D. Substitutability: $A \sim B \Rightarrow [p, A; 1-p, C] \sim [p, B; 1-p, C]$
- E. Monotonicity: $A \succ B \Rightarrow (p > q \Rightarrow [p, A; 1-p, B] \succ [q, A; 1-q, B])$
- F. Decomposability: $[p, A; 1-p, [q, B; 1-q, C]] \sim [p, A; (1-p)q, B; (1-p)(1-q), C]$

EXAMPLE: WHY TRANSITIVITY MATTERS (MONEY PUMP)

Suppose an agent has cyclic preferences: mango $\succ$ banana, banana $\succ$ jackfruit, jackfruit $\succ$ mango. It holds a mango.

A trader offers: "give me the mango and Re 1, take the jackfruit" (the agent prefers jackfruit to mango, so it pays). Then "jackfruit + Re 1 for banana", then "banana + Re 1 for mango". The agent is back to a mango and has lost Rs.3, and this can repeat forever. Transitivity rules this out.

EXAMPLE: DECOMPOSABILITY

A compound lottery: with probability 0.5 you win Rs.100; otherwise you enter a second lottery giving Rs.50 with probability 0.4 and Rs.0 with probability 0.6. Reduce it.

$P(100) = 0.5$. $P(50) = 0.5 \times 0.4 = 0.2$. $P(0) = 0.5 \times 0.6 = 0.3$. Check: $0.5 + 0.2 + 0.3 = 1$. So the compound lottery $\sim [0.5, \text{Rs.100}; 0.2, \text{Rs.50}; 0.3, \text{Rs.0}]$.

EXAMPLE: CONTINUITY

A = Rs.1000, B = Rs.400, C = Rs.0, and the agent's utility is money itself. Find p with $[p, A; 1-p, C] \sim B$.

$p \times 1000 + (1 - p) \times 0 = 400 \Rightarrow p = 0.4$.

Exam tip: The unit states completeness as "prefers one over the other"; the full axiom also allows indifference ($A \sim B$). The transitivity formula in the unit uses $\cap$; write it with $\wedge$ (logical AND). Learn the six names in order: Completeness, Transitivity, Continuity, Substitutability, Monotonicity, Decomposability.

Maximum Expected Utility (MEU) principle

Expected utility: $EU(A | E) = \sum_i P(\text{Outcome}_i(A) | E) \times U(\text{Outcome}_i(A))$, the probability-weighted average utility of an action's outcomes.

MEU principle: a rational agent chooses the action with the maximum expected utility.

Expected utility: $EU(A | E) = \sum_i P(\text{Outcome}_i(A) | E) \times U(\text{Outcome}_i(A))$; ഒരു action-ന്റെ outcomes-ന്റെ probability-weighted ശരാശരി utility.

MEU principle: ഒരു rational agent ഏറ്റവും കൂടിയ expected utility ഉള്ള action തിരഞ്ഞെടുക്കുന്നു.

Most actions are nondeterministic: action A can lead to several outcomes. Given evidence E, the agent assigns a probability to each outcome and computes the **expected utility**: the probability-weighted average of the outcome utilities. The **MEU principle** says a rational agent chooses the action with the highest expected utility.

മലയാളത്തിൽ

ഓരോ action-ന്റെയും സാധ്യമായ ഓരോ ഫലത്തിനും (probability × utility) കണ്ടുപിടിച്ച് കൂട്ടുക; അതാണ് expected utility. ഏറ്റവും കൂടിയ EU ഉള്ള action എടുക്കുക, അതാണ് MEU. എറണാകുളത്തുനിന്ന് കോഴിക്കോട്ടേക്ക് KSRTC-യോ train-ഓ എന്ന് തീരുമാനിക്കുമ്പോൾ നമ്മൾ മനസ്സിൽ ചെയ്യുന്നതും ഇതുതന്നെ.

$$EU(A | E) = \sum_i P(\text{Outcome}_i(A) | E) \times U(\text{Outcome}_i(A))$$

$$\text{MEU: choose action}^* = \text{argmax}_A EU(A | E)$$

EXAMPLE (UNIT'S EXAMPLE 1, SOLVED PROPERLY)

First move: Rs.1000 for sure. Second move: 50% Rs.1500, 50% Rs.0. Take U = money.

Action	Outcome	P	U	P × U
First	M1: Rs.1000	1	1000	1000
Second	M2: Rs.1500	0.5	1500	750
	M3: Rs.0	0.5	0	0

$EU(\text{First}) = 1000$. $EU(\text{Second}) = 750 + 0 = 750$. By MEU, choose the first move. If the prize in the second move rises to Rs.4000: $EU(\text{Second}) = 0.5 \times 4000 + 0.5 \times 0 = 2000 > 1000$, and the second move becomes better.

EXAMPLE: KSRTC OR TRAIN?

Anu must reach Kozhikode for an 11 am interview. Utilities: on time = 100. Train: on time with probability 0.8; late with probability 0.2 (utility 40). KSRTC: on time with probability 0.6; stuck in a block with probability 0.4 (utility 60, because the bus drops her closer to the venue).

Action	Outcome	P	U	P × U
Train	On time	0.8	100	80
	Late	0.2	40	8
KSRTC	On time	0.6	100	60
	Block	0.4	60	24

$EU(\text{Train}) = 80 + 8 = 88$. $EU(\text{KSRTC}) = 60 + 24 = 84$. $88 > 84$, so she takes the train.

EXAMPLE (CHECK YOUR PROGRESS 1, Q2)

Initial wealth Rs.1,00,000. Invest (I): 0.6 → Rs.1,50,000, 0.4 → Rs.60,000. Keep (K): stay at Rs.1,00,000.
Utility = final wealth.

$$EU(I) = 0.6 \times 1,50,000 + 0.4 \times 60,000 = 90,000 + 24,000 = 1,14,000.$$

$$EU(K) = 1 \times 1,00,000 = 1,00,000.$$

1,14,000 > 1,00,000, so choose Invest.

Extra check with a risk-averse utility $U(w) = \sqrt{w}$: $EU(I) = 0.6 \times \sqrt{150000} + 0.4 \times \sqrt{60000} = 0.6 \times 387.30 + 0.4 \times 244.95 = 232.38 + 97.98 = 330.36$; $U(K) = \sqrt{100000} = 316.23$. Invest still wins.

Exam tip: Always draw the $P \times U$ table and add a check that probabilities of each action sum to 1. In the unit's answer to CYP 1 Q3 ("How does the maximum utility principle help in decision making?") the key talks about the Maximum Entropy principle, which is a different idea. The correct answer: MEU gives a single rule (compute EU of every action, pick the largest), which handles uncertainty through probabilities, handles preferences through utilities, and makes the agent's choices consistent and rational.

Risk attitude, certainty equivalent and risk premium

Certainty equivalent (CE): the sure amount that an agent values equally with a gamble.

Risk premium: EMV of the gamble – CE.

Risk-averse: an agent whose utility of money is concave, so $CE < EMV$.

Certainty equivalent (CE): ഒരു gamble-ന് തുല്യമായി agent വിലമതിക്കുന്ന ഉറപ്പുള്ള തുക.

Risk premium: Gamble-ന്റെ EMV – CE.

Risk-averse: പണത്തിന്റെ utility concave ആയ agent; അതിനാൽ $CE < EMV$.

Utility is not the same as money. For most people the utility of money is **concave** (each extra rupee adds a little less), which makes them **risk-averse**. The **certainty equivalent (CE)** of a gamble is the sure amount the agent values equally with the gamble. The **risk premium** = EMV of the gamble – CE. Risk attitude also depends on circumstances: someone in debt may refuse a 50% win Rs.1500 / 50% lose Rs.1000 gamble that someone with Rs.10,000 savings accepts.

മലയാളത്തിൽ

കടത്തിലുള്ള ഒരാൾ ₹1000 നഷ്ടപ്പെടാൻ സാധ്യതയുള്ള ഒരു പന്തയത്തിൽ ഇറങ്ങില്ല, കാരണം ആ ₹1000 അയാൾക്ക് വളരെ വിലപ്പെട്ടതാണ്. "ഈ പന്തയത്തിന് പകരം ഉറപ്പായി എത്ര കിട്ടിയാൽ ഞാൻ തൃപ്തനാകും" എന്ന തുകയാണ് certainty equivalent. വീടിന് insurance എടുക്കുന്നത് ഈ risk-aversion കാരണമാണ്.

Risk-averse: U concave, $CE < EMV$

Risk-neutral: U linear, $CE = EMV$

Risk-seeking: U convex, $CE > EMV$

Risk premium = $EMV - CE$

EXAMPLE

Gamble: 50% Rs.1500, 50% Rs.0. The agent's utility is $U(x) = \sqrt{x}$. Find EMV, EU, CE and risk premium.

$EMV = 0.5 \times 1500 + 0.5 \times 0 = 750$.

$EU = 0.5 \times \sqrt{1500} + 0.5 \times \sqrt{0} = 0.5 \times 38.730 = 19.365$.

CE is the sure amount x with $\sqrt{x} = 19.365 \Rightarrow x = 19.365^2 = 375$.

Risk premium = $750 - 375 = \text{Rs.}375$. This agent would swap the gamble for any sure amount above Rs.375.

EXAMPLE

The debt gamble: 50% win Rs.1500, 50% lose Rs.1000. EMV?

$EMV = 0.5 \times 1500 + 0.5 \times (-1000) = 750 - 500 = \text{Rs.}250$. A risk-neutral agent accepts ($250 > 0$), but a person in debt, for whom losing Rs.1000 is very painful, may rationally refuse.

Exam tip: The unit says risk-averse agents pay an insurance premium "lower than the expected loss". It is the other way round: a risk-averse agent is willing to pay more than the expected loss (that difference is the risk premium, and it is how insurers make a profit). Also, preferring a sure Rs.1000 to a gamble with EMV Rs.750 does not by itself show risk aversion: even a risk-neutral agent picks the Rs.1000.

Decision theory: components and the payoff matrix

Decision theory: a framework for choosing the best action under uncertainty by listing all alternatives, possible outcomes and their payoffs.

States of nature: possible outcomes outside the decision maker's control; they are mutually exclusive and collectively exhaustive.

Payoff matrix: a table with actions as rows and states as columns, where Payoff_{ij} is the payoff of A_i when S_j occurs.

Decision theory: എല്ലാ alternatives-ഉം സാധ്യമായ outcomes-ഉം അവയുടെ payoffs-ഉം നിരത്തി, uncertainty-യിൽ ഏറ്റവും നല്ല action തിരഞ്ഞെടുക്കാനുള്ള ഒരു framework.

States of nature: Decision maker-ന്റെ നിയന്ത്രണത്തിന് പുറത്തുള്ള സാധ്യമായ outcomes; ഇവ mutually exclusive-ഉം collectively exhaustive-ഉം ആണ്.

Payoff matrix: Actions rows ആയും states columns ആയും ഉള്ള ഒരു table; S_j സംഭവിക്കുമ്പോൾ A_i -യുടെ payoff ആണ് Payoff_{ij} .

Decision theory gives a framework for choosing the best action when the future is uncertain. It forces the decision maker to list all alternatives and all possible outcomes. A standard decision problem has four parts: the decision maker (DM), mutually exclusive **actions** $A_1, \dots, A_n$, **states of nature** $S_1, \dots, S_m$ (outside the DM's control, mutually exclusive and collectively exhaustive), and **payoffs** for each (action, state) pair. These are arranged in a **payoff matrix**: rows = actions, columns = states.

മലയാളത്തിൽ

മഴക്കാലത്തിന് മുമ്പ് എന്ത് കൃഷി ചെയ്യണം എന്ന് ആലോചിക്കുന്ന ഒരു കർഷകനെ ഓർക്കുക. കർഷകനാണ് DM, നെല്ല്/വാഴ/കപ്പ ആണ് actions, മൺസൂൺ എങ്ങനെയാകും എന്നത് states of nature (അത് അയാളുടെ കയ്യിലല്ല), ഓരോ combination-നും കിട്ടുന്ന ലാഭമാണ് payoff. ഇതെല്ലാം ഒരു table-ൽ എഴുതിയാൽ payoff matrix ആയി.

Payoff matrix entry: Payoff_{ij} = payoff of action A_i when state S_j occurs
States must be mutually exclusive and collectively exhaustive

EXAMPLE: THE FARMER'S PAYOFF MATRIX (USED IN THE NEXT TOPICS)

A Palakkad farmer chooses a crop before the monsoon. Profit in thousands of rupees:

Action \ State	S1 : Heavy monsoon	S2 : Normal monsoon	S3 : Weak monsoon
A1: Paddy	80	60	-20
A2: Banana	30	70	40
A3: Tapioca	20	40	50

DM = farmer; actions = {Paddy, Banana, Tapioca}; states = {Heavy, Normal, Weak}; payoff₁₃ = -20 means paddy in a weak monsoon loses Rs.20,000.

Decision under certainty and uncertainty: maximin and maximax

Decision under uncertainty: the states of nature are known but their probabilities are not.

Maximin: the pessimistic rule: choose the action whose minimum payoff is the largest.

Maximax: the optimistic rule: choose the action whose maximum payoff is the largest.

Decision under uncertainty: States of nature അറിയാം, പക്ഷേ അവയുടെ probabilities അറിയില്ല.

Maximin: Pessimistic rule: ഏറ്റവും കുറഞ്ഞ payoff ഏറ്റവും വലുതായ action തിരഞ്ഞെടുക്കുക.

Maximax: Optimistic rule: ഏറ്റവും കൂടിയ payoff ഏറ്റവും വലുതായ action തിരഞ്ഞെടുക്കുക.

Decision environments are classified by what the DM knows about the states. **Certainty:** the state is known, so pick the best payoff in that column. **Uncertainty:** the states are known but no probabilities are available; the choice depends on attitude. The **maximin** (pessimistic) rule takes the minimum payoff of each action and picks the largest minimum. The **maximax** (optimistic) rule takes the maximum of each action and picks the largest maximum. **Risk:** probabilities are known; use EMV (next topic).

മലയാളത്തിൽ

മഴ എങ്ങനെയാകും എന്ന് ഒരു സൂചനയും ഇല്ലെങ്കിൽ, പേടിയുള്ള കർഷകൻ "ഏറ്റവും മോശം വന്നാലും നഷ്ടം കുറവുള്ളത്" തിരഞ്ഞെടുക്കും (maximin). ധൈര്യശാലി "എല്ലാം നന്നായി വന്നാൽ ഏറ്റവും കൂടുതൽ കിട്ടുന്നത്" തിരഞ്ഞെടുക്കും (maximax). IMD forecast കിട്ടിയാൽ probability ഉപയോഗിച്ച് EMV കണക്കാക്കാം.

Maximin: choose $\text{argmax}_i (\min_j \text{Payoff}_{ij})$

Maximax: choose $\text{argmax}_i (\max_j \text{Payoff}_{ij})$

EXAMPLE: FARMER UNDER CERTAINTY

The farmer is told for sure the monsoon will be normal (S2). Best action?

Column S2: Paddy 60, Banana 70, Tapioca 40. Choose Banana (70).

EXAMPLE: FARMER UNDER UNCERTAINTY

Action	Payoffs	Row minimum	Row maximum
Paddy	80, 60, -20	-20	80
Banana	30, 70, 40	30	70
Tapioca	20, 40, 50	20	50

Maximin: $\max(-20, 30, 20) = 30 \Rightarrow$ Banana. Maximax: $\max(80, 70, 50) = 80 \Rightarrow$ Paddy.

EXAMPLE: THE UNIT'S MANUFACTURING TABLE WITHOUT PROBABILITIES

Process A: 200, -50. Process B: 100, 30 (thousand rupees).

Maximin: $\min(A) = -50, \min(B) = 30 \Rightarrow$ Process B. Maximax: $\max(A) = 200, \max(B) = 100 \Rightarrow$ Process A.

Exam tip: Maximin = "max of the row minimums" (pessimist). Maximax = "max of the row maximums" (optimist). The two rules can give different answers, as both examples show; say which rule you used.

Decision under risk: Expected Monetary Value (EMV)

Decision under risk: the probabilities $P(S_j)$ of the states of nature are known.

Expected Monetary Value (EMV): $EMV(A_i) = \sum_j \text{Payoff}(A_i, S_j) \times P(S_j)$; choose the action with the highest EMV.

Decision under risk: States of nature-ന്റെ probabilities $P(S_j)$ അറിയാം.

Expected Monetary Value (EMV): $EMV(A_i) = \sum_j \text{Payoff}(A_i, S_j) \times P(S_j)$; ഏറ്റവും കൂടിയ EMV ഉള്ള action തിരഞ്ഞെടുക്കുക.

When the DM has probabilities $P(S_j)$ for the states (summing to 1), the main criterion is the **Expected Monetary Value**: the probability-weighted average payoff of each action. Choose the action with the highest EMV. EMV is expected utility with utility = money.

മലയാളത്തിൽ

ഓരോ state-ന്റെയും probability അറിയാമെങ്കിൽ, ഓരോ row-യിലെ payoff-നെ അതിന്റെ probability കൊണ്ട് ഗുണിച്ച് കൂട്ടുക. ഏറ്റവും കൂടിയ തുക കിട്ടുന്ന action ആണ് EMV അനുസരിച്ച് നല്ലത്. ഒരു തവണ നഷ്ടം വന്നേക്കാം, പക്ഷേ പല വർഷം ഇങ്ങനെ ചെയ്താൽ ശരാശരി ഏറ്റവും കൂടുതൽ ലാഭം കിട്ടും.

$$EMV(A_i) = \sum_j \text{Payoff}(A_i, S_j) \times P(S_j)$$

$$\sum_j P(S_j) = 1$$

EXAMPLE (UNIT'S EXAMPLE 3: MANUFACTURING CHOICE)

$P(\text{High}) = 0.6$, $P(\text{Low}) = 0.4$. Process A: 200 / -50. Process B: 100 / 30 (thousand rupees).

Process	High: payoff $\times 0.6$	Low: payoff $\times 0.4$	EMV
A	$200 \times 0.6 = 120$	$-50 \times 0.4 = -20$	100
B	$100 \times 0.6 = 60$	$30 \times 0.4 = 12$	72

$100 > 72$, so choose Process A (EMV Rs.1,00,000 against Rs.72,000).

EXAMPLE: THE FARMER WITH AN IMD FORECAST

$P(\text{Heavy}) = 0.3$, $P(\text{Normal}) = 0.5$, $P(\text{Weak}) = 0.2$.

Crop	Heavy $\times 0.3$	Normal $\times 0.5$	Weak $\times 0.2$	EMV
Paddy	$80 \times 0.3 = 24$	$60 \times 0.5 = 30$	$-20 \times 0.2 = -4$	50
Banana	$30 \times 0.3 = 9$	$70 \times 0.5 = 35$	$40 \times 0.2 = 8$	52
Tapioca	$20 \times 0.3 = 6$	$40 \times 0.5 = 20$	$50 \times 0.2 = 10$	36

Highest EMV = 52 $\Rightarrow$ plant Banana (expected profit Rs.52,000).

EXAMPLE (CHECK YOUR PROGRESS 2, Q3)

Action C: Rs.50 for sure. Action G: 80% chance of Rs.100, 20% chance of Rs.0. Choose by EMV.

$$\text{EMV}(C) = 1 \times 50 = 50.$$

$$\text{EMV}(G) = 0.8 \times 100 + 0.2 \times 0 = 80.$$

$80 > 50$, so choose G. (A strongly risk-averse person might still take C; with $U(x) = \sqrt{x}$, $EU(G) = 0.8 \times 10 = 8 > \sqrt{50} = 7.07$, so even this risk-averse agent picks G.)

Exam tip: The unit's answer to CYP 2 Q3 stops after "Action C: Rs.50 for sure" and never finishes. The full answer is $\text{EMV}(G) = 80 > \text{EMV}(C) = 50$, so choose G.

Decision trees

Decision tree: a tree diagram of a decision problem with decision nodes (squares), chance nodes (circles, with branch probabilities) and payoffs at the leaves.

Rolling back: solving the tree from the leaves: take the expected value at chance nodes and the maximum at decision nodes.

EVPI: expected value of perfect information = expected payoff with perfect information – best EMV without it.

Decision tree: Decision nodes (squares), chance nodes (circles, branch probabilities സഹിതം), leaves-ൽ payoffs എന്നിവയുള്ള, ഒരു decision problem-ന്റെ tree diagram.

Rolling back: Leaves-ൽ നിന്ന് tree solve ചെയ്യൽ: chance nodes-ൽ expected value-ഉം decision nodes-ൽ maximum-ഉം എടുക്കുക.

EVPI: Expected value of perfect information = perfect information ഉള്ളപ്പോഴുള്ള expected payoff – അതില്ലാതെയുള്ള ഏറ്റവും നല്ല EMV.

A **decision tree** draws a decision problem step by step: square nodes are decisions, circle nodes are chance events (with probabilities on the branches), and leaves carry payoffs. It is solved by **rolling back** from the leaves: at a chance node take the expected value, at a decision node take the maximum. Trees are especially useful when one decision leads to later decisions.

മലയാളത്തിൽ

Decision tree ഒരു വലിയ payoff matrix-നെ ചിത്രമാക്കുന്നു. ഇലകളിൽ (leaves) നിന്ന് തുടങ്ങി പിന്നോട്ട് വരിക: chance node-ൽ probability $\times$ payoff കൂട്ടുക, decision node-ൽ ഏറ്റവും വലുത് എടുക്കുക. വേരിൽ (root) എത്തുമ്പോൾ ഉത്തരം കിട്ടും.

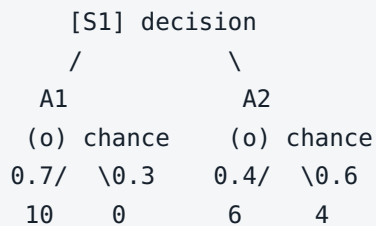
Chance node value = $\sum P(\text{branch}) \times \text{value}(\text{branch})$

Decision node value = max over branches

EVPI = (expected payoff with perfect information) – (best EMV without it)

EXAMPLE (CHECK YOUR PROGRESS 3, Q3 AS A TREE)

From S1: A1 → S2 (0.7, reward 10) or S3 (0.3, reward 0). A2 → S2 (0.4, reward 6) or S3 (0.6, reward 4). S2, S3 terminal.



Chance node under A1: $0.7 \times 10 + 0.3 \times 0 = 7$.

Chance node under A2: $0.4 \times 6 + 0.6 \times 4 = 2.4 + 2.4 = 4.8$.

Decision node: $\max(7, 4.8) = 7 \Rightarrow$ choose A1.

EXAMPLE: IS A PERFECT MONSOON FORECAST WORTH PAYING FOR?

Using the farmer's matrix and probabilities (0.3, 0.5, 0.2), how much is a perfect forecast worth?

With perfect information the farmer picks the best crop for each state: Heavy → Paddy 80, Normal → Banana 70, Weak → Tapioca 50.

Expected payoff with perfect information = $0.3 \times 80 + 0.5 \times 70 + 0.2 \times 50 = 24 + 35 + 10 = 69$.

Best EMV without it = 52 (Banana).

EVPI = $69 - 52 = 17$, so the farmer should pay at most Rs.17,000 for a perfect forecast.

Do humans follow utility theory? (Tversky-Kahneman)

Normative theory: a theory that says how a rational agent should act, not how people actually act.

Regret: the feeling of loss at ending with nothing after giving up a sure gain; it helps explain why people violate utility theory.

Normative theory: ആളുകൾ യഥാർത്ഥത്തിൽ എങ്ങനെ പെരുമാറുന്നു എന്നല്ല, ഒരു rational agent എങ്ങനെ പെരുമാറണം എന്ന് പറയുന്ന theory.

Regret: ഉറപ്പുള്ള നേട്ടം ഉപേക്ഷിച്ച് ഒന്നും കിട്ടാതെ വരുമ്പോഴുള്ള നഷ്ടബോധം; ആളുകൾ utility theory ലംഘിക്കുന്നത് എന്തുകൊണ്ട് എന്ന് വിശദീകരിക്കാൻ ഇത് സഹായിക്കുന്നു.

Decision theory is normative: it says how an agent should act. Experiments by Tversky and Kahneman show that real people systematically violate the axioms. Offered A (80% chance of Rs.4000) or B (Rs.3000 for sure), most people pick B. Offered C (20% chance of Rs.4000) or D (25% chance of Rs.3000), most pick C. No utility function can produce both choices. One explanation is **regret**: choosing A risks a 20% chance of getting nothing after giving up a sure Rs.3000.

മലയാളത്തിൽ

ഉറപ്പായി ₹3000 കിട്ടുമെങ്കിൽ മിക്കവരും അത് എടുക്കും, 80% സാധ്യതയുള്ള ₹4000 വേണ്ടെന്ന് വെക്കും. പക്ഷേ രണ്ടിലും സാധ്യത കുറവാണെങ്കിൽ (20% vs 25%) അതേ ആളുകൾ വലിയ തുക തിരഞ്ഞെടുക്കും. കണക്കനുസരിച്ച് ഇത് പരസ്പരവിരുദ്ധമാണ്. "ഉറപ്പുള്ളത് കളഞ്ഞിട്ട് ഒന്നും കിട്ടിയില്ലെങ്കിൽ" എന്ന ഖേദമാണ് (regret) കാരണം.

$$\begin{aligned} \text{EMV(A)} &= 0.8 \times 4000 = 3200 & \text{EMV(B)} &= 1 \times 3000 = 3000 \\ \text{EMV(C)} &= 0.2 \times 4000 = 800 & \text{EMV(D)} &= 0.25 \times 3000 = 750 \end{aligned}$$

EXAMPLE: PROVING THE CHOICES ARE INCONSISTENT

Show that $B \succ A$ together with $C \succ D$ is impossible for any utility function (take $U(0) = 0$).

$$B \succ A \Rightarrow U(3000) > 0.8 U(4000).$$

$$C \succ D \Rightarrow 0.2 U(4000) > 0.25 U(3000). \text{ Multiply both sides by 4: } 0.8 U(4000) > U(3000).$$

The two lines contradict each other, so no U fits both choices. Note that C and D are just A and B with all winning probabilities multiplied by $1/4$ ($0.8 \times 1/4 = 0.2$, $1 \times 1/4 = 0.25$), so by substitutability/decomposability the preference should not flip.

Exam tip: The unit's text says people prefer D over C and that EMV "reverses" this. In the actual Tversky–Kahneman experiment the majority chose B and C. EMV agrees with C ($800 > 750$) and disagrees only with B; the real puzzle is that B-and-C together fits no utility function, as shown above.

Sequential decision problems

Definition: A sequential decision problem is one in which an agent makes a series of decisions over time, each action affecting future states and options, and seeks a policy that maximises cumulative reward.

ഒരു agent കാലക്രമത്തിൽ തുടർച്ചയായി തീരുമാനങ്ങൾ എടുക്കുകയും ഓരോ action-ഉം ഭാവിയിലെ states-നെയും options-നെയും ബാധിക്കുകയും ചെയ്യുന്ന problem ആണ് sequential decision problem; cumulative reward പരമാവധിയാക്കുന്ന ഒരു policy ആണ് ലക്ഷ്യം.

A payoff matrix handles one-shot decisions. In a **sequential decision problem** the agent makes a series of decisions over time; each action gives an immediate result and also changes the future state and future options (chess is the classic example). The goal is a strategy that optimises a cumulative objective, such as the total reward. If the environment is fully observable, rewards are additive and transitions are Markov, the problem is an **MDP**; if the agent cannot see the state fully, it is a **POMDP**.

മലയാളത്തിൽ

ഇന്നത്തെ തീരുമാനം നാളത്തെ സാഹചര്യം മാറ്റുന്നു. ഈ വർഷം ഭൂമിയിൽ എന്ത് കൃഷി ചെയ്യുന്നു എന്നത് അടുത്ത വർഷം മണ്ണിന്റെ ഗുണത്തെ ബാധിക്കും. അതുകൊണ്ട് ഒരു action അല്ല, ഓരോ സാഹചര്യത്തിലും എന്ത് ചെയ്യണം എന്ന ഒരു plan (policy) ആണ് വേണ്ടത്.

Single-step: choose one action, immediate consequences only

Sequential: choose a policy π (action for every state), maximise cumulative reward

Fully observable + Markov $\Rightarrow$ MDP; partially observable $\Rightarrow$ POMDP

EXAMPLE

Why is a greedy choice wrong in a sequential problem? A fisherman can fish a pond heavily this season (reward 10, pond becomes depleted, next season reward 2) or moderately (reward 7, pond stays healthy, next season reward 7).

Greedy one-step view: $10 > 7$, so fish heavily. Two-season view (no discount): heavy = $10 + 2 = 12$, moderate = $7 + 7 = 14$. Moderate fishing is better over time: the current action changed the future state.

Markov Decision Process (MDP) and optimal policy

MDP: a tuple $\langle S, A, T, R, \gamma \rangle$ modelling sequential decisions in a fully observable, probabilistic environment, with $T(s, a, s') = P(s' | s, a)$.

Markov property: the next state depends only on the current state and action, not on the history.

Optimal policy π^* : a mapping $S \rightarrow A$ that gives an action for every state and maximises the expected total discounted reward.

MDP: Fully observable-ഉം probabilistic-ഉം ആയ environment-ലെ sequential decisions model ചെയ്യുന്ന $\langle S, A, T, R, \gamma \rangle$ എന്ന tuple; ഇവിടെ $T(s, a, s') = P(s' | s, a)$.

Markov property: അടുത്ത state നിലവിലെ state-നെയും action-നെയും മാത്രം ആശ്രയിക്കുന്നു, history-യെ അല്ല.

Optimal policy π^* : ഓരോ state-നും ഒരു action നൽകുകയും expected total discounted reward പരമാവധിയാക്കുകയും ചെയ്യുന്ന $S \rightarrow A$ mapping.

An MDP models an agent acting in a fully observable, probabilistic environment over discrete time steps. It is the tuple $\langle S, A, T, R, \gamma \rangle$: states S , actions A , transition model $T(s, a, s') = P(s' | s, a)$, reward $R(s)$ or $R(s, a)$ or $R(s, a, s')$, and discount factor $\gamma \in [0, 1)$. The **Markov property**: the next state depends only on the current state and action, not on the history. A **policy** $\pi: S \rightarrow A$ gives an action for every state; the **optimal policy** π^* maximises the expected total discounted reward, and $V^*(s)$ is the value (utility) of state s under π^* .

മലയാളത്തിൽ

Markov property എന്നാൽ "ഇപ്പോഴത്തെ അവസ്ഥ മാത്രം നോക്കിയാൽ മതി, പഴയ കഥ വേണ്ട" എന്നാണ്. γ എന്ന discount factor ഭാവിയിലെ reward-ന് എത്ര പ്രാധാന്യം കൊടുക്കണം എന്ന് തീരുമാനിക്കുന്നു: γ 0-ന് അടുത്താണെങ്കിൽ agent "ഇന്ന് കിട്ടുന്നത് മാത്രം" നോക്കും, 1-ന് അടുത്താണെങ്കിൽ ദീർഘവീക്ഷണത്തോടെ ചിന്തിക്കും. ഓരോ state-ലും ഏത് action എന്ന് പറയുന്ന table ആണ് policy.

$$\text{MDP} = \langle S, A, T, R, \gamma \rangle$$

$$T(s, a, s') = P(s' | s, a), \text{ with } \sum_{s'} P(s' | s, a) = 1$$

$$\text{Discounted return} = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$

$$\pi^*: S \rightarrow A$$

$$\pi^*(s) = \operatorname{argmax}_{a \in A} \sum_{s'} T(s, a, s') V^*(s') \quad (8.1, \text{ rewards on states})$$

$$\pi^*(s) = \operatorname{argmax}_{a \in A} [R(s, a) + \gamma \sum_{s'} T(s, a, s') V^*(s')] \quad (\text{rewards on state-action pairs})$$

EXAMPLE: EFFECT OF γ

A reward of 10 every step forever. What is the total discounted reward for $\gamma = 0.9$ and $\gamma = 0.5$?

Sum = $10(1 + \gamma + \gamma^2 + \dots) = 10/(1 - \gamma)$. $\gamma = 0.9$: $10/0.1 = 100$. $\gamma = 0.5$: $10/0.5 = 20$. The smaller γ makes the far future almost worthless (myopic agent).

EXAMPLE: IDENTIFYING THE TUPLE

An autorickshaw can be in Good or Worn condition. Each day the driver can Run it or Repair it. Running a Good auto earns 10 and it stays Good with probability 0.7, becomes Worn with 0.3. Write the MDP.

$S = \{\text{Good, Worn}\}$; $A = \{\text{Run, Repair}\}$; $T(\text{Good, Run, Good}) = 0.7$, $T(\text{Good, Run, Worn}) = 0.3$; $R(\text{Good, Run}) = 10$; γ chosen by the designer (e.g. 0.5). The full model is solved in the value-iteration topic below.

Bellman equation and the value iteration algorithm

Bellman equation: $V(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') V(s')$: a state's value is its reward plus the discounted expected value of the best action.

Value iteration: start with $V_0(s) = 0$ and apply the Bellman update repeatedly until the largest change $\delta < \epsilon$, then extract π^* by argmax.

Bellman equation: $V(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') V(s')$: ഒരു state-ന്റെ value അതിന്റെ reward-ഉം ഏറ്റവും നല്ല action-ന്റെ discounted expected value-ഉം ചേർന്നതാണ്.

Value iteration: $V_0(s) = 0$ -ൽ തുടങ്ങി, ഏറ്റവും വലിയ മാറ്റം $\delta < \epsilon$ ആകുന്നതുവരെ Bellman update ആവർത്തിക്കുക; പിന്നെ argmax വഴി π^* കണ്ടെത്തുക.

The value of a state is its immediate reward plus the discounted value of where the best action leads. This is the **Bellman equation** (Bellman, 1957). With n states there are n such equations in n unknowns, but they are nonlinear because of max, so we solve them iteratively: start with $V_0(s) = 0$ and repeatedly apply the **Bellman update** until the largest change δ between iterations is below a threshold ϵ . Then extract the policy with argmax.

മലയാളത്തിൽ

Value iteration ഒരു "ആവർത്തിച്ച് മെച്ചപ്പെടുത്തൽ" ആണ്. ആദ്യം എല്ലാ state-ന്റെയും value 0 എന്ന് വെക്കുക. ഓരോ round-ലും, ഓരോ action-നും (reward + $\gamma \times$ അടുത്ത state-ന്റെ പഴയ value) കണക്കാക്കി ഏറ്റവും വലുത് പുതിയ value ആക്കുക. മാറ്റം തീരെ ചെറുതാകുമ്പോൾ നിർത്തുക.

Bellman equation: $V(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} T(s, a, s') V(s')$ (8.2)

Bellman update: $V_{i+1}(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} T(s, a, s') V_i(s')$ (8.3)

With $R(s, a)$: $V_{i+1}(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_i(s')]$

VALUE-ITERATION

Step 1 $V_0(s) = 0$ for all s ; choose ϵ (e.g. 0.0001); $\delta = \infty$; $i = 0$

Step 2 repeat while $\delta \geq \epsilon$:

$i = i + 1$; $\delta = 0$

for each state s :

$V_i(s) = \max \text{ over } a \text{ of } [R(s, a) + \gamma \sum P(s' | s, a) V_{i-1}(s')]$

$\delta = \max(\delta, |V_i(s) - V_{i-1}(s)|)$

Step 3 stop: $V_i(s) \approx V^*(s)$; $\pi^*(s) = \text{the argmax action in the last update}$

EXAMPLE: ONE BELLMAN UPDATE

State s has two actions. a1: reward 4, goes to x (0.5) or y (0.5). a2: reward 6, goes to y (1). $\gamma = 0.8$, $V_i(x) = 10$, $V_i(y) = 2$. Find $V_{i+1}(s)$.

a1: $4 + 0.8 \times (0.5 \times 10 + 0.5 \times 2) = 4 + 0.8 \times 6 = 8.8$.

a2: $6 + 0.8 \times (1 \times 2) = 7.6$.

$V_{i+1}(s) = \max(8.8, 7.6) = 8.8$, best action a1.

Exam tip: The unit's algorithm writes "repeat while $\delta > 0$ " but then terminates when $\delta < \epsilon$. The correct loop condition is "repeat while $\delta \geq \epsilon$ ". Also note that equation 8.2 puts the reward on the state, $R(s)$, while Example 5 uses $R(s, a)$; when rewards depend on the action, $R(s, a)$ must go inside the max.

Value iteration: the unit's two-room robot (Example 5)

A robot moves in a hallway with two rooms: s_1 and s_2 . Action a_1 always moves to the other room; a_2 always stays. Rewards $R(s, a)$: $R(s_1, a_1) = 10$, $R(s_1, a_2) = 2$, $R(s_2, a_1) = 3$, $R(s_2, a_2) = 8$. $\gamma = 0.9$. Transitions are deterministic, so each sum $\sum P V$ has one term.

മലയാളത്തിൽ

രണ്ട് മുറിയുള്ള ഒരു ഇടനാഴിയിൽ robot. a_1 എടുത്താൽ മറ്റേ മുറിയിലേക്ക് പോകും, a_2 എടുത്താൽ അവിടെത്തന്നെ നിൽക്കും. ഓരോ iteration-ലും നാല് സംഖ്യകൾ (ഓരോ state-നും ഓരോ action-നും) കണക്കാക്കി ഓരോ state-നും വലുത് എടുക്കുന്നു.

$$V_i(s1) = \max(10 + 0.9 V_{i-1}(s2), 2 + 0.9 V_{i-1}(s1))$$

$$V_i(s2) = \max(3 + 0.9 V_{i-1}(s1), 8 + 0.9 V_{i-1}(s2))$$

EXAMPLE: ITERATIONS 1 AND 2 IN FULL

Iteration 1 ($V_0 = 0$): $V_1(s1) = \max(10 + 0.9 \times 0, 2 + 0.9 \times 0) = \max(10, 2) = 10$. $V_1(s2) = \max(3 + 0.9 \times 0, 8 + 0.9 \times 0) = \max(3, 8) = 8$.

Iteration 2: $V_2(s1) = \max(10 + 0.9 \times 8, 2 + 0.9 \times 10) = \max(17.2, 11) = 17.2$. $V_2(s2) = \max(3 + 0.9 \times 10, 8 + 0.9 \times 8) = \max(12, 15.2) = 15.2$.

EXAMPLE: THE FULL ITERATION TABLE

Q-values are the four candidates inside the max (rounded to 4 decimals). δ = max change over both states.

i	s1, a1: 10 + 0.9 V(s2)	s1, a2: 2 + 0.9 V(s1)	s2, a1: 3 + 0.9 V(s1)	s2, a2: 8 + 0.9 V(s2)	$V_i(s1)$	$V_i(s2)$	δ
0	–	–	–	–	0	0	–
1	10	2	3	8	10	8	10
2	17.2	11	12	15.2	17.2	15.2	7.2
3	23.68	17.48	18.48	21.68	23.68	21.68	6.48
4	29.512	23.312	24.312	27.512	29.512	27.512	5.832
5	34.7608	28.5608	29.5608	32.7608	34.7608	32.7608	5.2488
6	39.4847	33.2847	34.2847	37.4847	39.4847	37.4847	4.7239
7	43.7362	37.5362	38.5362	41.7362	43.7362	41.7362	4.2515
8	47.5626	41.3626	42.3626	45.5626	47.5626	45.5626	3.8264
9	51.0064	44.8064	45.8064	49.0064	51.0064	49.0064	3.4437
10	54.1057	47.9057	48.9057	52.1057	54.1057	52.1057	3.0994
20	72.2739	66.0739	67.0739	70.2739	72.2739	70.2739	1.0807
30	78.6087	72.4087	73.4087	76.6087	78.6087	76.6087	0.3768
50	81.5877	75.3877	76.3877	79.5877	81.5877	79.5877	0.0458
100	81.9979	75.7979	76.7979	79.9979	81.9979	79.9979	0.0002
109	81.9992	75.7992	76.7992	79.9992	81.9992	79.9992	< 0.0001

Rows 1–10 are every iteration; after that the pattern is identical and only δ shrinks (by a factor 0.9 each time), so selected rows are shown. The winning action never changes: a1 in s1 and a2 in s2.

Closed form (check): $V_i(s2) = 80(1 - 0.9^i)$ and $V_i(s1) = 82 - 72 \times 0.9^{i-1}$. For $i = 2$: $80 \times 0.19 = 15.2$ and $82 - 64.8 = 17.2 \checkmark$. The change in s1 at iteration i is $7.2 \times 0.9^{i-2}$, which first drops below $\epsilon = 0.0001$ at $i = 109$.

Exact values: with $\pi = (s1 \rightarrow a1, s2 \rightarrow a2)$, $V^*(s2) = 8 + 0.9 V^*(s2) \Rightarrow V^*(s2) = 8/0.1 = 80$, and $V^*(s1) = 10 + 0.9 \times 80 = 82$.

Policy check: in s1, a1 gives $10 + 0.9 \times 80 = 82$ against a2: $2 + 0.9 \times 82 = 75.8 \Rightarrow a1$. In s2, a2 gives $8 + 0.9 \times$

$80 = 80$ against $a1$: $3 + 0.9 \times 82 = 76.8 \Rightarrow a2$.

$\pi^*(s1) = a1$ (go to $s2$), $\pi^*(s2) = a2$ (stay in $s2$ and collect 8 forever).

Exam tip: The unit's reward matrix gives $R(s2, a1) = 3$, but its worked iterations use 5 ("max($5 + 0.9 (1 \times 0)$, $8 \dots$)" and " $5 + 0.9 (1 \times 10) = 14$ "). With the stated $R = 3$ the numbers are 3 and 12, not 5 and 14. The final values $V_1(s2) = 8$ and $V_2(s2) = 15.2$ happen to be the same either way because $a2$ wins, so the unit's final answers are right but its intermediate steps are not.

Value iteration with probabilistic transitions

Definition: The Q-value of action a in state s is $Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s')$, and $V(s) = \max_a Q(s, a)$.

State s -ൽ action a -യുടെ Q-value $Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s')$ ആണ്; $V(s) = \max_a Q(s, a)$.

When an action can lead to several next states, each Q-value uses a weighted sum $\sum P(s' | s, a) V(s')$. The procedure is otherwise the same. This example also shows that the best action in a state can change during the iterations before settling.

മലയാളത്തിൽ

ഒരു auto-rickshaw ഡ്രൈവറെ ഓർക്കുക. Auto നല്ല നിലയിലാണെങ്കിൽ ഓടിക്കാം, പക്ഷേ തേയ്മാനം വരാൻ സാധ്യതയുണ്ട്. തേഞ്ഞ auto ഓടിച്ചാൽ കുറച്ച് മാത്രം കിട്ടും; repair ചെയ്താൽ ഇന്ന് നഷ്ടം, പക്ഷേ നാളെ മുതൽ നല്ല വരുമാനം. ഏതാണ് നല്ലത് എന്ന് value iteration പറഞ്ഞുതരും.

EXAMPLE: THE AUTORICKSHAW MDP

$S = \{G \text{ (good)}, W \text{ (worn)}\}$, $A = \{\text{Run, Rep (repair)}\}$, $\gamma = 0.5$, stop when $\delta < 0.01$.

State , action	Reward $R(s, a)$	Transitions
G, Run	10	G 0.7, W 0.3
G, Rep	4	G 1
W, Run	3	W 1
W, Rep	-2	G 1

Update rules:

$$Q(G, \text{Run}) = 10 + 0.5 (0.7 V(G) + 0.3 V(W))$$

$$Q(G, \text{Rep}) = 4 + 0.5 V(G)$$

$$Q(W, \text{Run}) = 3 + 0.5 V(W)$$

$$Q(W, \text{Rep}) = -2 + 0.5 V(G)$$

Iteration 2 worked by hand ($V_1(G) = 10$, $V_1(W) = 3$):

$$Q(G, \text{Run}) = 10 + 0.5 (0.7 \times 10 + 0.3 \times 3) = 10 + 0.5 \times 7.9 = 13.95$$

$$Q(G, \text{Rep}) = 4 + 0.5 \times 10 = 9$$

$$Q(W, \text{Run}) = 3 + 0.5 \times 3 = 4.5$$

$$Q(W, \text{Rep}) = -2 + 0.5 \times 10 = 3$$

$$V_2(G) = 13.95, V_2(W) = 4.5, \delta = \max(3.95, 1.5) = 3.95.$$

i	Q(G, Run)	Q(G, Rep)	Q(W, Run)	Q(W, Rep)	V _i (G)	V _i (W)	Best in W	δ
0	–	–	–	–	0	0	–	–
1	10	4	3	–2	10	3	Run	10
2	13.95	9	4.5	3	13.95	4.5	Run	3.95
3	15.5575	10.975	5.25	4.975	15.5575	5.25	Run	1.6075
4	16.2326	11.7787	5.625	5.7787	16.2326	5.7787	Rep	0.6751
5	16.5482	12.1163	5.8894	6.1163	16.5482	6.1163	Rep	0.3376
6	16.7093	12.2741	6.0582	6.2741	16.7093	6.2741	Rep	0.1611
7	16.7894	12.3547	6.1371	6.3547	16.7894	6.3547	Rep	0.0805
8	16.8295	12.3947	6.1773	6.3947	16.8295	6.3947	Rep	0.0401
9	16.8495	12.4147	6.1973	6.4147	16.8495	6.4147	Rep	0.0201
10	16.8595	12.4248	6.2074	6.4248	16.8595	6.4248	Rep	0.0100
11	16.8646	12.4298	6.2124	6.4298	16.8646	6.4298	Rep	0.0050

At iteration 10, δ is 0.01002, just above 0.01, so one more round is needed. At iteration 11, $\delta = 0.0050 < 0.01$: stop. Best action in G is always Run. In W, Run wins for the first three iterations (the future is not yet "visible"), then Repair wins from iteration 4 onward.

Policy: $\pi^*(G) = \text{Run}$, $\pi^*(W) = \text{Repair}$. Exact check: solve $V(G) = 10 + 0.5 (0.7 V(G) + 0.3 V(W))$ and $V(W) = -2 + 0.5 V(G)$. From these, $0.65 V(G) - 0.15 V(W) = 10$ and $V(W) = -2 + 0.5 V(G) \Rightarrow 0.575 V(G) = 9.7 \Rightarrow V(G) = 16.870$, $V(W) = 6.435$. The iterates are approaching these values. (Always-Run gives $V(W) = 3/0.5 = 6 < 6.435$, confirming Repair is better in W.)

POMDPs, applications and challenges

POMDP: an MDP in which the agent cannot observe the state exactly; it adds observations and an observation model $P(o | s)$, and the agent acts on a belief (probability distribution over states).

Curse of dimensionality: computational cost grows exponentially with the number of states and actions.

POMDP: Agent-ന് state കൃത്യമായി കാണാൻ കഴിയാത്ത MDP; ഇതിൽ observations-ഉം $P(o | s)$ എന്ന observation model-ഉം ചേർക്കുന്നു, agent ഒരു belief (states-ന്മേലുള്ള probability distribution) അനുസരിച്ച് പ്രവർത്തിക്കുന്നു.

Curse of dimensionality: States-ന്റെയും actions-ന്റെയും എണ്ണം കൂടുമ്പോൾ computational cost exponential ആയി വർധിക്കുന്നു.

In a **Partially Observable MDP** the agent does not know the exact current state. POMDPs add hidden states and an **observation model** (the probability of each observation given the state), so the agent must reason about a probability distribution over states (a belief). Sequential decision problems appear in robotics (navigation, path planning, manipulation), finance (portfolio management over time), healthcare (treatment plans for chronic diseases) and autonomous systems (cars, drones). Main challenges: the **curse of dimensionality** (cost grows exponentially with states and actions), the **exploration-exploitation** trade-off under uncertainty, and **scalability**, often handled by approximation such as deep reinforcement learning.

മലയാളത്തിൽ

MDP-ൽ agent-ന് എല്ലാം കാണാം. POMDP-ൽ ഡോക്ടർ രോഗിയെ നോക്കുന്നതുപോലെയാണ്: രോഗം (state) നേരിട്ട് കാണാനാകില്ല, ലക്ഷണങ്ങൾ (observations) മാത്രം. പുതിയ ഹോട്ടലിൽ പോകണോ (exploration) സ്ഥിരം പോകുന്ന ഹോട്ടലിൽ പോകണോ (exploitation) എന്ന ചോദ്യമാണ് exploration-exploitation dilemma.

MDP: $\langle S, A, T, R, \gamma \rangle$, state fully observed

POMDP: $\langle S, A, T, R, \gamma \rangle + \text{observations } O + \text{observation model } P(o | s)$, state hidden

EXAMPLE: MDP OR POMDP?

Classify: (a) chess, (b) a robot vacuum with a noisy sensor that cannot tell which room it is in, (c) a doctor treating a patient whose disease is not yet confirmed.

(a) MDP (game-like, full board visible). (b) POMDP: the location is hidden, only noisy readings are seen. (c) POMDP: the true disease is hidden, only symptoms and test results are observed.

Game theory: introduction and key concepts

Game theory: the study of situations in which several decision makers (players) interact and each one's payoff depends on the actions of the others.

Zero-sum game: a game in which one player's gain is the other's loss, so the payoffs in every outcome add to 0.

Game theory: പല decision makers (players) പരസ്പരം ഇടപെടുകയും ഓരോരുത്തരുടെയും payoff മറ്റുള്ളവരുടെ actions-നെ ആശ്രയിക്കുകയും ചെയ്യുന്ന സാഹചര്യങ്ങളുടെ പഠനം.

Zero-sum game: ഒരു player-ന്റെ നേട്ടം മറ്റേയാളുടെ നഷ്ടമായ game; അതിനാൽ ഓരോ outcome-ലും payoffs-ന്റെ തുക 0 ആണ്.

Game theory models situations where several decision makers (players or agents) interact and each one's outcome depends on the actions of the others. Modern AI is full of such settings: autonomous vehicles on the road, trading agents, warehouse robots, ad auctions, security systems against attackers. A game has three components: **players**, **actions (strategies)** and **payoffs (utilities)**. Games are classified as cooperative vs non-cooperative, zero-sum vs general-sum, simultaneous vs sequential, and one-shot vs repeated.

മലയാളത്തിൽ

ഇതുവരെ ഒറ്റ agent മാത്രമായിരുന്നു. ഇനി പല agents ഒരുമിച്ച്, ഒരാളുടെ ഫലം മറ്റുള്ളവരുടെ തീരുമാനത്തെയും ആശ്രയിക്കുന്നു. ഒരേ റൂട്ടിൽ ഓടുന്ന രണ്ട് private bus-കൾ സമയം പിടിക്കാൻ മത്സരിക്കുന്നത് ഒരു game ആണ്. ഒരാളുടെ ലാഭം കൃത്യമായി മറ്റേയാളുടെ നഷ്ടമാണെങ്കിൽ zero-sum.

Classification	Meaning	Example
Cooperative / Non-cooperative	Binding agreements and coalitions allowed / each acts alone	Swarm robots / competing sellers
Zero-sum / General-sum	One's gain = other's loss / interests can be mixed	Chess, matching pennies / traffic coordination
Simultaneous / Sequential	Choose at the same time / take turns	Rock-paper-scissors / chess
One-shot / Repeated	Played once / played many times (reputation, learning)	A single auction / daily ad auctions

EXAMPLE: IS IT ZERO-SUM?

Check whether each game is zero-sum: (a) Table 4 below, with cells (2, 3), (0, 0), (1, 1), (3, 2); (b) matching pennies with cells (1, -1), (-1, 1).

Zero-sum means the two payoffs in every cell add to 0. (a) $2 + 3 = 5 \neq 0 \Rightarrow$ general-sum. (b) $1 + (-1) = 0$ in every cell $\Rightarrow$ zero-sum.

Normal-form and extensive-form games

Normal-form game: a game given as a payoff matrix of players' strategies (rows and columns), suited to simultaneous moves.

Extensive-form game: a game given as a game tree (decision nodes, action edges, payoff leaves), showing the order of moves and what each player knows.

Backward induction: solving an extensive-form game from the last decision back to the root.

Normal-form game: Players-ന്റെ strategies rows-ഉം columns-ഉം ആയ payoff matrix ആയി നൽകുന്ന game; simultaneous moves-ന് അനുയോജ്യം.

Extensive-form game: Game tree ആയി (decision nodes, action edges, payoff leaves) നൽകുന്ന game; moves-ന്റെ ക്രമവും ഓരോ player-നും എന്ത് അറിയാം എന്നതും ഇത് കാണിക്കുന്നു.

Backward induction: അവസാന decision-ൽ നിന്ന് root-ലേക്ക് പിന്നോട്ട് പോയി extensive-form game solve ചെയ്യൽ.

A **normal-form** (strategic-form) game is a payoff matrix: Player 1's strategies are rows, Player 2's are columns, and each cell holds (payoff to Player 1, payoff to Player 2). It suits simultaneous moves and ignores timing. An **extensive-form** game is a game tree: nodes are decision points, edges are actions, leaves are payoffs. It shows the order of moves and what each player knows, and it is solved by **backward induction** (like minimax): start at the last decision and work back to the root.

മലയാളത്തിൽ

Normal form ഒരു table ആണ്, രണ്ടുപേരും ഒരേസമയം തീരുമാനിക്കുന്നു. Extensive form ഒരു മരം (tree) ആണ്, ആര് ആദ്യം, ആര് പിന്നെ എന്ന ക്രമം അതിൽ കാണാം. മരം solve ചെയ്യാൻ അവസാനത്തെ കളിക്കാരൻ എന്ത് ചെയ്യും എന്ന് ആദ്യം നോക്കി പിന്നോട്ട് വരുന്നു.

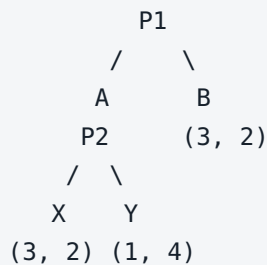
EXAMPLE (TABLE 4: NORMAL FORM)

Player 1 \ Player 2	C1	C2
R1	(2, 3)	(0, 0)
R2	(1, 1)	(3, 2)

Reading a cell: (R1, C1) = (2, 3) means Player 1 gets 2 and Player 2 gets 3. (R2, C2) = (3, 2): Player 1 gets 3, Player 2 gets 2.

EXAMPLE (FIGURE 1: EXTENSIVE FORM, SOLVED)

Player 1 chooses A or B. After A, Player 2 chooses X → (3, 2) or Y → (1, 4). B ends the game with (3, 2). What happens?



Step 1 (last mover, Player 2 after A): X gives P2 2, Y gives P2 4 ⇒ P2 picks Y, so A leads to (1, 4).

Step 2 (Player 1 at the root): A → 1, B → 3 ⇒ P1 picks B.

Outcome: (3, 2). Player 1 foresees that Player 2 would pick Y after A, so she avoids A.

Exam tip: For "distinguish normal form and extensive form" write: normal form = payoff matrix, simultaneous moves, strategies and payoffs only (e.g. Prisoner's Dilemma); extensive form = game tree, sequential moves, shows order and information (e.g. chess, bargaining).

Nash equilibrium

Nash equilibrium: a strategy profile in which no player can increase their payoff by changing only their own strategy while the others keep theirs fixed.

Mixed strategy: a probability distribution over a player's actions (a pure strategy plays one action with probability 1).

Nash equilibrium: മറ്റുള്ളവർ അവരുടെ strategy മാറ്റാതിരിക്കുമ്പോൾ, സ്വന്തം strategy മാത്രം മാറ്റി ഒരു player-നും payoff കൂട്ടാൻ കഴിയാത്ത strategy profile.

Mixed strategy: ഒരു player-ന്റെ actions-ന്മേലുള്ള probability distribution (pure strategy ഒരു action-നെ probability 1 കൊണ്ട് കളിക്കുന്നു).

A **Nash equilibrium** is a strategy profile (one strategy per player) in which no player can increase their payoff by changing only their own strategy while the others keep theirs fixed. Every player is playing a **best response** to the others. Equilibria can be **pure** (one action with probability 1) or **mixed** (a probability distribution over actions); in a mixed equilibrium each player's mix makes the other player indifferent between the actions they randomise over. Some games have no pure equilibrium but every finite game has at least one mixed one.

മലയാളത്തിൽ

Nash equilibrium എന്നാൽ "മറ്റുള്ളവർ മാറ്റുന്നില്ലെങ്കിൽ ഞാൻ മാറിയിട്ട് ഒരു ഗുണവുമില്ല" എന്ന അവസ്ഥ, എല്ലാ കളിക്കാർക്കും ഒരുപോലെ. ഒരു ജംഗ്ഷനിൽ രണ്ട് കാറുകൾ എത്തുമ്പോൾ ഒരാൾ പോകുകയും മറ്റേയാൾ കാത്തുനിൽക്കുകയും ചെയ്യുന്നതാണ് സ്ഥിരതയുള്ള അവസ്ഥ. ഓരോ cell-ലും "ഈ കളിക്കാരന് മാറിയാൽ കൂടുതൽ കിട്ടുമോ" എന്ന് രണ്ടുപേർക്കും check ചെയ്യാൽ മതി.

$s^* = (s_1^*, \dots, s_n^*)$ is a Nash equilibrium if for every player i and every strategy s_i :

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$$

Pure NE test for a cell: Player 1's payoff is the best in its column AND Player 2's payoff is the best in its row

EXAMPLE (UNIT'S EXAMPLE 6: TWO CARS AT A JUNCTION)

Car A \ Car B	G (Go)	W (Wait)
G	(-10, -10)	(5, 3)
W	(3, 5)	(2, 2)

Best responses of Car A: if B plays G, A compares -10 (G) and 3 (W) $\Rightarrow$ W. If B plays W, A compares 5 (G) and 2 (W) $\Rightarrow$ G. By symmetry Car B: against G $\Rightarrow$ W, against W $\Rightarrow$ G.

Cell	Can A gain by switching?	Can B gain by switching?	NE?
(G, G)	Yes: -10 $\rightarrow$ 3	Yes: -10 $\rightarrow$ 3	No
(G, W)	No: 5 vs 3	No: 3 vs -10	Yes
(W, G)	No: 3 vs -10	No: 5 vs 3	Yes
(W, W)	Yes: 2 $\rightarrow$ 5	Yes: 2 $\rightarrow$ 5	No

Two pure Nash equilibria: (G, W) and (W, G): one car goes, the other waits. A convention or communication is needed so both cars pick the same equilibrium.

Mixed equilibrium: let Car A go with probability p . Car B is indifferent when its expected payoffs are equal: Go: $-10p + 5(1 - p) = 5 - 15p$; Wait: $3p + 2(1 - p) = 2 + p$. So $5 - 15p = 2 + p \Rightarrow 16p = 3 \Rightarrow p = 3/16$. By symmetry Car B also goes with probability $3/16$. Each car's expected payoff $= 2 + 3/16 = 35/16 \approx 2.19$, and a crash (G, G) still happens with probability $(3/16)^2 \approx 0.035$: another reason AI cars need a coordination rule.

EXAMPLE (TABLE 4)

Find all Nash equilibria of the Table 4 game.

(R1, C1) = (2, 3): P1 switching to R2 gives $1 < 2$; P2 switching to C2 gives $0 < 3 \Rightarrow$ NE.

(R1, C2) = (0, 0): P1 switching to R2 gives $3 > 0 \Rightarrow$ not NE.

(R2, C1) = (1, 1): P1 switching to R1 gives $2 > 1 \Rightarrow$ not NE.

(R2, C2) = (3, 2): P1 switching gives $0 < 3$; P2 switching gives $1 < 2 \Rightarrow$ NE.

Mixed: P1 plays R1 with p so that P2 is indifferent: $3p + 1(1 - p) = 0p + 2(1 - p) \Rightarrow 1 + 2p = 2 - 2p \Rightarrow p = 1/4$. P2 plays C1 with q so that P1 is indifferent: $2q = q + 3(1 - q) \Rightarrow 2q = 3 - 2q \Rightarrow q = 3/4$. Expected payoff = 1.5 for each player.

EXAMPLE: PRISONER'S DILEMMA

Two accused are questioned separately. Payoffs are (negative) years in jail: both stay silent (S, S) = (-1, -1); one confesses (C) while the other is silent: confessor 0, silent one -10; both confess (C, C) = (-5, -5). Find the NE.

For Player 1: if P2 is silent, C gives $0 > -1$; if P2 confesses, C gives $-5 > -10$. So C is best whatever P2 does (a dominant strategy); the same holds for P2. The only NE is (C, C) = (-5, -5), even though (S, S) = (-1, -1) is better for both. A Nash equilibrium need not be the best joint outcome.

Exam tip: Always check both players for every cell, and state the unilateral deviation payoffs. Importance in AI: it predicts stable outcomes of multi-agent systems, guides multi-agent reinforcement learning, security games and auction design.

Sample applications

Security game: a game in which a defender with limited resources uses an unpredictable mixed strategy to protect targets from an attacker.

Mechanism design: designing the rules of a game, such as an auction, so that self-interested players produce a desired outcome.

Security game: പരിമിതമായ resources ഉള്ള ഒരു defender, attacker-ൽ നിന്ന് targets സംരക്ഷിക്കാൻ പ്രവചിക്കാനാകാത്ത mixed strategy ഉപയോഗിക്കുന്ന game.

Mechanism design: സ്വന്തം താല്പര്യം നോക്കുന്ന players ആഗ്രഹിച്ച outcome ഉണ്ടാക്കുന്ന രീതിയിൽ ഒരു game-ന്റെ (ഉദാ. auction) rules രൂപകൽപ്പന ചെയ്യൽ.

Utility theory, sequential decisions and game theory together let AI agents act well alone and among others. **Self-driving cars** give routes utilities from time, fuel, safety and comfort and pick the highest expected utility.

Recommendation systems give higher utility to films matching a user's genres, actors or directors and rank by it. **Multi-agent RL** (independent Q-learning, Nash Q-learning, policy gradients) has rewards that depend on other agents. **Security games** compute unpredictable mixed strategies for defenders with limited resources.

Auctions and mechanism design model ad bidding. **Cooperative systems** (swarm robots, sensor networks) use game theory for fair rewards and to stop free-riding.

മലയാളത്തിൽ

YouTube-ൽ നിങ്ങൾക്ക് ഇഷ്ടമുള്ള മലയാളം സിനിമകൾ ആദ്യം വരുന്നത് utility ഉയർന്നതുകൊണ്ടാണ്. Google Maps കുറഞ്ഞ സമയമുള്ള വഴി കാണിക്കുന്നത് expected utility കണക്കാക്കിയാണ്. ശബരിമല സീസണിൽ പോലീസ് പരിമിതമായ സേനയെ എവിടെ വിന്യസിക്കണം എന്ന് തീരുമാനിക്കുന്നത് ഒരു security game പോലെയാണ്.

EXAMPLE: RECOMMENDER UTILITY

A system scores films as $U = 3 \times (\text{genre match}) + 2 \times (\text{favourite actor}) + 1 \times (\text{favourite director})$, each 0 or 1. Film P matches genre and actor; film Q matches actor and director. Which is shown first?
 $U(P) = 3 + 2 + 0 = 5$. $U(Q) = 0 + 2 + 1 = 3$. P is ranked above Q.

EXAMPLE: SECURITY GAME

A defender with one guard protects two targets: T1 worth 10, T2 worth 5. The attacker hits whichever target gives it more expected gain (it gains the target's value only if that target is unguarded). The defender guards T1 with probability g . Find g that makes the attacker indifferent.

Attacker's gain at T1 = $10(1 - g)$; at T2 = $5g$. Set equal: $10 - 10g = 5g \Rightarrow g = 2/3$. Expected loss for the defender = $10 \times 1/3 = 10/3 \approx 3.33$ whichever target is hit. A fixed (pure) choice would lose 5 (guard T1 always) or 10 (guard T2 always), so randomising helps.

Quick recap

- Utility $U(S)$ is a number for how much the agent likes state S ; $U(A) > U(B) \Rightarrow$ prefer A.
- Six axioms: completeness, transitivity, continuity, substitutability, monotonicity, decomposability.
- $EU(A | E) = \sum P(\text{outcome} | E) \times U(\text{outcome})$; MEU: pick the action with the largest EU.
- Utility $\neq$ money: concave utility $\Rightarrow$ risk-averse; risk premium = EMV - certainty equivalent.
- Decision problem = DM, actions, states of nature, payoffs, arranged in a payoff matrix.
- Certainty: best in the known column. Uncertainty: maximin (pessimist) or maximax (optimist). Risk: EMV.
- $EMV(A_i) = \sum \text{Payoff}(A_i, S_j) \times P(S_j)$.
- Decision trees roll back: expected value at chance nodes, max at decision nodes.
- Tversky-Kahneman: most people choose B and C, which no utility function can explain (regret).
- Sequential problems need a policy; fully observable + Markov $\Rightarrow$ MDP $\langle S, A, T, R, \gamma \rangle$, else POMDP.
- Bellman update: $V_{i+1}(s) = \max_a [R(s, a) + \gamma \sum P(s' | s, a) V_i(s')]$, repeat until $\delta < \epsilon$.
- Example 5 converges to $V^*(s_1) = 82$, $V^*(s_2) = 80$, $\pi^* = (a_1, a_2)$.
- Game = players, strategies, payoffs; normal form = matrix, extensive form = tree (backward induction).

- Nash equilibrium: nobody gains by changing alone; the junction game has pure NE (G, W) and (W, G).

Check yourself

Q. 1. List two advantages and two disadvantages of using a utility function in AI.

Advantages: (i) it lets an agent act rationally by maximising expected utility; (ii) combined with probabilities it handles uncertainty and risk in one framework (also used for sequential decisions and multi-agent games).

Disadvantages: (i) it is hard to design; a badly chosen utility can cause unintended or unsafe behaviour; (ii) utilities are subjective and cannot easily encode ethics, emotions or social norms. It also assumes perfect rationality and can be expensive to compute in large state spaces.

Q. 2. Wealth Rs.2,00,000. Invest: 0.5 → Rs.3,00,000, 0.5 → Rs.1,20,000. Keep: Rs.2,00,000. Utility = final wealth. Which action?

$$EU(\text{Invest}) = 0.5 \times 3,00,000 + 0.5 \times 1,20,000 = 1,50,000 + 60,000 = 2,10,000.$$

$$EU(\text{Keep}) = 2,00,000.$$

$$2,10,000 > 2,00,000 \Rightarrow \text{Invest.}$$

Q. 3. Name the three decision environments and the criterion used in each.

Certainty (state known): choose the highest payoff in that state's column. Uncertainty (no probabilities): maximin (pessimistic) or maximax (optimistic). Risk (probabilities known): Expected Monetary Value. (The unit's answer key instead lists agent environments: fully/partially observable, deterministic/stochastic, single/multi-agent; mention these too if the question asks about "environmental scenarios" in general.)

Q. 4. Payoffs (Rs. thousand): Shop X: 50, 20, -10; Shop Y: 30, 25, 15 for Boom, Normal, Slump with P = 0.2, 0.5, 0.3. Find the maximin, maximax and EMV choices.

$$\text{Maximin: } \min X = -10, \min Y = 15 \Rightarrow Y.$$

$$\text{Maximax: } \max X = 50, \max Y = 30 \Rightarrow X.$$

$$EMV(X) = 50 \times 0.2 + 20 \times 0.5 + (-10) \times 0.3 = 10 + 10 - 3 = 17.$$

$$EMV(Y) = 30 \times 0.2 + 25 \times 0.5 + 15 \times 0.3 = 6 + 12.5 + 4.5 = 23.$$

$$EMV \Rightarrow Y.$$

Q. 5. Action C: Rs.50 for sure. Action G: 80% chance of Rs.100, 20% nothing. Choose by EMV.

$$EMV(C) = 50. EMV(G) = 0.8 \times 100 + 0.2 \times 0 = 80. \text{ Choose G.}$$

Q. 6. What is a sequential decision problem? How does it differ from a single-step decision problem, and how is it represented?

The agent makes a series of decisions over time; each action affects the future state and future options, so it needs a policy that maximises long-term cumulative reward. A single-step problem chooses one action with immediate consequences only. Sequential problems are represented as MDPs: states S , actions A , transition model $T = P(s' | s, a)$, reward R , discount γ , solved for a policy π ; POMDPs when the state is only partially observable.

Q. 7. From S_1 : $A_1 \rightarrow S_2$ (0.7, reward 10) or S_3 (0.3, reward 0); $A_2 \rightarrow S_2$ (0.4, reward 6) or S_3 (0.6, reward 4). S_2, S_3 terminal. Which action?

$EU(A_1) = 0.7 \times 10 + 0.3 \times 0 = 7$. $EU(A_2) = 0.4 \times 6 + 0.6 \times 4 = 2.4 + 2.4 = 4.8$. $7 > 4.8 \Rightarrow A_1$.

Q. 8. In Example 5 (two rooms, $\gamma = 0.9$), compute $V_3(s_1)$ and $V_3(s_2)$ from $V_2 = (17.2, 15.2)$.

$V_3(s_1) = \max(10 + 0.9 \times 15.2, 2 + 0.9 \times 17.2) = \max(23.68, 17.48) = 23.68$.

$V_3(s_2) = \max(3 + 0.9 \times 17.2, 8 + 0.9 \times 15.2) = \max(18.48, 21.68) = 21.68$.

Q. 9. Define Nash equilibrium and find all pure equilibria of: $(U, L) = (4, 1)$, $(U, R) = (0, 3)$, $(D, L) = (2, 2)$, $(D, R) = (1, 0)$.

A Nash equilibrium is a strategy profile where no player can increase their payoff by changing only their own strategy. It matters in AI because it predicts stable outcomes of multi-agent systems and guides MARL, security games and auctions.

(U, L) : P2 switches to R: $3 > 1 \Rightarrow$ no. (U, R) : P1 switches to D: $1 > 0 \Rightarrow$ no. (D, L) : P1 switches to U: $4 > 2 \Rightarrow$ no. (D, R) : P2 switches to L: $2 > 0 \Rightarrow$ no. There is no pure Nash equilibrium; the players must randomise.

Mixed: P1 plays U with p so P2 is indifferent: $1p + 2(1 - p) = 3p + 0 \Rightarrow 2 - p = 3p \Rightarrow p = 1/2$. P2 plays L with q so P1 is indifferent: $4q + 0 = 2q + 1(1 - q) \Rightarrow 4q = 1 + q \Rightarrow q = 1/3$.

Q. 10. Distinguish normal-form and extensive-form representations, and solve the tree: P1 chooses A or B; after A, P2 chooses X (3, 2) or Y (1, 4); B gives (3, 2).

Normal form: payoff matrix, simultaneous moves, shows only strategies and payoffs. Extensive form: game tree, sequential moves, shows order of play and information.

Backward induction: after A, P2 picks Y ($4 > 2$), giving (1, 4). P1 compares $A \rightarrow 1$ and $B \rightarrow 3$, picks B. Outcome (3, 2).

UNIT 9

Expert Systems

ഒരു വിദഗ്ദ്ധന്റെ (expert) അറിവും ചിന്താശീലിയും ഒരു computer program-ലേക്ക് പകർത്തുന്നതാണ് expert system. ഈ unit-ൽ expert system എന്താണ്, അറിവ് എങ്ങനെ represent ചെയ്യുന്നു (logic, rules), MYCIN, COMPASS പോലുള്ള പ്രശസ്ത systems, expert system shell-ന്റെ ഭാഗങ്ങൾ, EMYCIN, applications എന്നിവ പഠിക്കാം.

What is an expert system?

Knowledge-based systems, task domain and knowledge engineering

Planning and using domain knowledge

Knowledge representation schemes

Propositional logic and its rules of inference

First order predicate logic (FOPL)

Shortcomings of predicate calculus

MYCIN

COMPASS

Other well-known expert systems

Expert system building tools

Expert system shell and its architecture

Knowledge base

Knowledge acquisition subsystem (COMPASS)

Inference engine

Explanation subsystem (car example, MYCIN)

User interface and EMYCIN

Application areas of expert systems

Advantages and limitations

Case study: a paddy-disease expert system for Kerala farmers

What is an expert system?

Definition: An expert system is a computer program that possesses or represents knowledge in a particular domain and can reason with this knowledge to solve a problem or achieve a specific goal.

ഒരു പ്രത്യേക domain-ലെ knowledge ഉൾക്കൊള്ളുകയും, ഒരു problem solve ചെയ്യാനോ ഒരു നിശ്ചിത goal നേടാനോ ആ knowledge ഉപയോഗിച്ച് reason ചെയ്യുകയും ചെയ്യുന്ന computer program ആണ് expert system.

Human experts are scarce in health, education, agriculture, finance and many other fields. An **expert system** is a computer program (or set of programs) that contains the knowledge and some of the reasoning ability of an expert, usually a human expert, in one particular domain. Just having an algorithm is not enough: the program must hold **domain knowledge** and reason with it to solve problems. The domain must be narrow, because no human (and no program) can be an expert in everything.

മലയാളത്തിൽ

നമ്മുടെ നാട്ടിൽ ഒരു നല്ല cardiologist-നെ കാണാൻ ചിലപ്പോൾ ജില്ല വിട്ട് പോകേണ്ടി വരും, കാരണം experts കുറവാണ്. ആ doctor-ന്റെ അറിവും ചിന്താശീലിയും ഒരു program-ലേക്ക് പകർത്തിയാൽ അതാണ് expert system. ഇത് ഒരു പ്രത്യേക domain-ൽ മാത്രമേ പ്രവർത്തിക്കൂ; heart രോഗത്തിന്റെ expert system-നോട് തെങ്ങിന്റെ രോഗം ചോദിച്ചിട്ട് കാര്യമില്ല. Expert-നെ പൂർണ്ണമായി replace ചെയ്യണമെന്നില്ല, വേഗത്തിൽ തീരുമാനമെടുക്കാൻ സഹായിച്ചാൽ മതി.

Definition (IGNOU): An Expert System is a computer program that possesses or represents knowledge in a particular domain, and has the capability of processing/manipulating or reasoning with this knowledge with a view to solving a problem or achieving some specific goal.

Basic properties:

1. It simulates human reasoning about a domain, not the domain itself.
2. It reasons over acquired knowledge, rather than only calculating or retrieving data.
3. It can use heuristic or approximate models, which (unlike algorithms) are not guaranteed to succeed.

Conventional program	Expert system
Algorithm + data	Knowledge base + inference engine
Knowledge is mixed into the code	Knowledge (rules) is kept separate from the reasoning code
Always gives the exact answer if the algorithm is correct	Uses heuristics (rules of thumb); may be wrong, like a human expert
Cannot explain its answer	Can explain why it asks a question and how it reached a conclusion
Changing behaviour means reprogramming	Add or edit rules; the engine stays the same

EXAMPLE

Which of these is an expert system? (a) A program that computes income tax from a fixed formula. (b) A program that asks a farmer about leaf symptoms and suggests which disease his paddy has.

(a) is a conventional program: it only calculates using a fixed algorithm. (b) is an expert system: it holds an agricultural officer's knowledge (symptom → disease rules), reasons with the farmer's answers, and its conclusion is a heuristic judgement in a narrow domain.

Exam tip: "Define expert system and list its properties" is a very common 4-5 mark question. Write the IGNOU definition word-for-word, then the three properties.

Knowledge-based systems, task domain and knowledge engineering

Knowledge-based system (KBS): a system that performs a task by applying rules of thumb to a symbolic representation of knowledge, rather than mainly algorithmic or statistical methods.

Task domain: the area captured in an expert system: the task is the goal-oriented problem-solving activity, and the domain is the area in which it is performed.

Knowledge engineering: the process of building an expert system, carried out by knowledge engineers.

Knowledge-based system (KBS): പ്രധാനമായും algorithmic അല്ലെങ്കിൽ statistical methods-ന് പകരം, knowledge-ന്റെ symbolic representation-ൽ rules of thumb പ്രയോഗിച്ച് ഒരു task ചെയ്യുന്ന system.

Task domain: ഒരു expert system ഉൾക്കൊള്ളുന്ന മേഖല: task എന്നത് goal-oriented problem-solving activity-യും, domain എന്നത് അത് നടക്കുന്ന മേഖലയും ആണ്.

Knowledge engineering: Knowledge engineers ചെയ്യുന്ന, ഒരു expert system നിർമ്മിക്കുന്ന പ്രക്രിയ.

A **knowledge-based system (KBS)** performs a task by applying rules of thumb to a symbolic representation of knowledge, rather than mainly using algorithmic or statistical methods. The term expert system is often reserved for a KBS whose knowledge came from human experts (not from textbooks or non-experts), but in practice the two terms are used interchangeably. The area captured in an expert system is the **task domain**: the **task** is the goal-oriented problem-solving activity (diagnosis, planning, scheduling, configuration, design) and the **domain** is the area in which it is done. Building an expert system is called **knowledge engineering**, done by **knowledge engineers**.

മലയാളത്തിൽ

Knowledge engineer ഒരു "പരിഭാഷകൻ" പോലെയാണ്. Expert doctor-ന് computer അറിയില്ല, programmer-ന് medicine അറിയില്ല. ഇവർക്കിടയിൽ നിന്ന് expert-ന്റെ അറിവ് ചോദിച്ചറിഞ്ഞ്, അത് rules പോലുള്ള ഒരു representation-ലേക്ക് മാറ്റുന്നതും, ഏത് reasoning method ഉപയോഗിക്കണമെന്ന് തീരുമാനിക്കുന്നതും knowledge engineer-ന്റെ പണിയാണ്. കാലാവസ്ഥയെക്കുറിച്ച് വെറുതെ സംസാരിക്കുന്ന program ഒരു KBS ആകാം, പക്ഷേ weather forecast ചെയ്യാൻ കഴിയുന്നതേ expert system ആകൂ.

Task = goal-oriented problem-solving activity (e.g. diagnosis)

Domain = area in which the task is performed (e.g. blood infections)

Knowledge engineer's jobs: (1) get the knowledge from the expert, (2) choose representation scheme(s): semantic nets, frames, predicate logic, rules, (3) choose efficient reasoning methods.

EXAMPLE

Identify task and domain: (i) MYCIN, (ii) XCON, (iii) a system that plans KSRTC bus crew duties.

(i) Task: diagnosis and treatment; domain: bacterial blood infections. (ii) Task: configuration; domain: VAX computer systems. (iii) Task: scheduling; domain: bus crew rostering.

Exam tip: The unit's example: a program that chats about weather is a knowledge-based system; only one that can forecast weather (needs meteorology expertise) is an expert system.

Planning and using domain knowledge

Planning: the process that exploits the structure of a problem to design a sequence of actions that solves it.

Classical planning environment: one that is fully observable, deterministic, finite, static and discrete.

Planning: ഒരു problem-ന്റെ ഘടന ഉപയോഗപ്പെടുത്തി അത് solve ചെയ്യുന്ന actions-ന്റെ ഒരു sequence രൂപകൽപ്പന ചെയ്യുന്ന പ്രക്രിയ.

Classical planning environment: Fully observable, deterministic, finite, static, discrete എന്നിവയായ environment.

To solve hard problems we plan. **Planning** is the process that exploits the structure of the problem to design a sequence of actions that solves it. Knowledge of the nature and structure of the problem domain is essential for

planning. Problem environments are of two kinds: **classical planning environments** are fully observable, deterministic, finite, static and discrete; **non-classical** ones may be only partially observable and/or stochastic.

മലയാളത്തിൽ

ഒരു കല്യാണം നടത്തുന്നത് ഓർക്കുക: hall book ചെയ്യണം, പിന്നെ catering, പിന്നെ ക്ഷണക്കത്ത്. ഏത് ക്രമത്തിൽ ചെയ്യണമെന്ന് തീരുമാനിക്കുന്നതാണ് planning. എല്ലാം അറിയാവുന്ന, ഒന്നും പെട്ടെന്ന് മാറാത്ത ഒരു ലോകം (chess board പോലെ) classical environment ആണ്. മഴ പെയ്യുമോ, ആളുകൾ വരുമോ എന്ന് ഉറപ്പില്ലാത്തത് non-classical.

Classical environment: fully observable, deterministic, finite, static, discrete

Non-classical: partially observable and/or stochastic

EXAMPLE

Classify: (a) the 8-puzzle, (b) driving an autorickshaw in Kochi traffic.

(a) Classical: the whole board is visible, each move has a fixed result, finitely many states, nothing changes while you think, moves are discrete. (b) Non-classical: you cannot see everything, other vehicles behave unpredictably (stochastic) and the world changes continuously.

Knowledge representation schemes

Definition: A knowledge representation scheme is a way of representing knowledge as symbol structures that supports inference; a good scheme is expressive, allows new knowledge to be inferred and has well-defined syntax and semantics.

Knowledge-നെ inference സാധ്യമാക്കുന്ന symbol structures ആയി പ്രതിനിധീകരിക്കുന്ന രീതിയാണ് knowledge representation scheme; നല്ല scheme expressive ആയിരിക്കും, പുതിയ knowledge infer ചെയ്യാൻ അനുവദിക്കും, well-defined syntax-ഉം semantics-ഉം ഉണ്ടായിരിക്കും.

AI assumes that intelligent behaviour can be achieved by manipulating **symbol structures** that represent pieces of knowledge. A key requirement of a representation scheme is that it supports **inference**, because we cannot store everything explicitly; some facts must be deduced when needed. A good scheme should (1) be able to express the knowledge we want, (2) allow new knowledge to be inferred from basic facts, and (3) have well-defined syntax and semantics.

മലയാളത്തിൽ

"എല്ലാ മനുഷ്യരും മരിക്കും", "സോക്രട്ടീസ് മനുഷ്യനാണ്" എന്ന് പറഞ്ഞാൽ "സോക്രട്ടീസ് മരിക്കും" എന്ന് നമ്മൾ സ്വയം കണ്ടെത്തും; അത് വേറെ എഴുതി വെക്കേണ്ടതില്ല. ഇങ്ങനെ പുതിയ അറിവ് infer ചെയ്യാൻ സാധിക്കുന്ന രീതിയിലായിരിക്കണം knowledge represent ചെയ്യുന്നത്. Semantic nets, frames, first order logic, rules എന്നിവയാണ് പ്രധാന schemes; ഇവ Unit 6-ൽ വിശദമായി പഠിച്ചതാണ്.

Popular schemes: semantic networks, frames, first order logic, rule-based systems
Good scheme: expressive + supports inference + well-defined syntax and semantics
This unit uses rule-based (IF-THEN) representation for expert systems.

EXAMPLE

Represent "Coconut is a palm; palms have no branches" as a rule and infer something new.

Facts: palm(coconut). Rule: IF palm(X) THEN no-branches(X). Applying the rule with X = coconut gives the new fact no-branches(coconut), which was never stored explicitly.

Exam tip: The unit's example of something FOL cannot express, "every symmetric and transitive relation need not be reflexive", talks about all relations. First order logic can quantify over objects only, not over relations, so this needs second order logic.

Propositional logic and its rules of inference

Proposition: a sentence that can be assigned a truth value, true or false.

Modus Ponens: from P and $P \rightarrow Q$, conclude Q.

Proposition: True അല്ലെങ്കിൽ false എന്ന truth value നൽകാൻ കഴിയുന്ന ഒരു sentence.

Modus Ponens: P-യും $P \rightarrow Q$ -ഉം ഉണ്ടെങ്കിൽ Q എന്ന് നിഗമനം ചെയ്യുക.

A **proposition** (statement) is a sentence that can be assigned a truth value, true or false. "The sun rises in the west" is a proposition (false). Commands, questions and exclamations are not propositions. To solve a problem with propositional logic (PL): (a) translate the English sentences into PL formulae, (b) apply rules of inference to derive the required formula.

മലയാളത്തിൽ

"ഇന്ന് ഞായറാഴ്ചയാണ്" എന്നത് ശരിയോ തെറ്റോ എന്ന് പറയാം, അതിനാൽ ഇത് proposition ആണ്. "വാതിൽ അടയ്ക്കൂ!" എന്നതിനോ "നീ എവിടെ പോകുന്നു?" എന്നതിനോ ശരി/തെറ്റ് ഇല്ല, അവ proposition അല്ല. Modus Ponens ലളിതമാണ്: "മഴ പെയ്താൽ റോഡ് നനയും", "മഴ പെയ്തു" എന്നറിഞ്ഞാൽ "റോഡ് നനഞ്ഞു" എന്ന് ഉറപ്പിക്കാം.

Modus Ponens: from P and $P \rightarrow Q$, conclude Q

Chain rule: from $P \rightarrow Q$ and $Q \rightarrow R$, conclude $P \rightarrow R$

Transposition: from $P \rightarrow Q$, conclude $\neg Q \rightarrow \neg P$

EXAMPLE (IDENTIFY PROPOSITIONS)

Which are propositions? (i) Ram must exercise regularly. (ii) Who was the first Prime Minister of India? (iii) 7 is a prime number. (iv) Hurry! We have won the trophy.

Only (iii) is a proposition (true). (i) is imperative, (ii) interrogative, (iv) exclamatory; none has a truth value.

EXAMPLE (THE UNIT'S EXAMPLE)

Given: (i) Matter always existed. (ii) If there is God, then God created the universe. (iii) If God created the universe, then matter did not always exist. Show: There is no God.

Symbols: M = matter always existed, TG = there is God, GU = God created the universe.

(i) M (ii) $TG \rightarrow GU$ (iii) $GU \rightarrow \neg M$ To show: $\neg TG$.

(v) Transposition on (iii): $M \rightarrow \neg GU$.

(vi) Modus Ponens on (i), (v): $\neg GU$.

(vii) Transposition on (ii): $\neg GU \rightarrow \neg TG$.

(viii) Modus Ponens on (vi), (vii): $\neg TG$. Proved.

EXAMPLE (THE UNIT'S EXERCISE 1)

Given: (i) There is a moral law. (ii) If there is a moral law, then someone gave it. (iii) If someone gave the moral law, then there is God. Show: (iv) There is God.

Symbols: ML , SG , TG . Formulae: (i) ML , (ii) $ML \rightarrow SG$, (iii) $SG \rightarrow TG$.

Modus Ponens on (i), (ii): (v) SG . Modus Ponens on (v), (iii): (vi) TG , which is (iv). Proved.

(Alternative: chain rule on (ii), (iii) gives $ML \rightarrow TG$, then Modus Ponens with (i) gives TG .)

Exam tip: Limitation of PL: each sentence is an unbreakable symbol. From $P = \text{"All children over 12 must exercise"}$ and $Q = \text{"Ram is over 12"}$ PL cannot conclude "Ram must exercise", because it cannot see inside P and Q . This is why we need predicate logic.

First order predicate logic (FOPL)

Definition: First order predicate logic (FOPL) extends propositional logic with predicates, functions, variables and the quantifiers $\forall$ and $\exists$, so it can express properties of objects, relations between them and general statements.

Predicates, functions, variables, $\forall$, $\exists$ എന്നീ quantifiers എന്നിവ ചേർത്ത് propositional logic-നെ വികസിപ്പിക്കുന്നതാണ് first order predicate logic (FOPL); objects-ന്റെ properties, അവ തമ്മിലുള്ള relations, പൊതു statements എന്നിവ ഇതിൽ പ്രകടിപ്പിക്കാം.

FOPL extends propositional logic with predicates, functions, variables and the quantifiers $\forall$ (for all) and $\exists$ (there exists), so it can describe properties of objects, relations between them and general statements about classes. The PL rules (Modus Ponens, chain rule, transposition) still hold with FOPL formulae. In addition there are quantifier rules Q1 to Q4 that have no PL counterpart. FOPL proofs are often done by **contradiction**: add the negation of the goal to the assumptions and derive a contradiction.

മലയാളത്തിൽ

Propositional logic-ൽ ഒരു വാക്യം മുഴുവൻ ഒരു അക്ഷരം (P) മാത്രമാണ്; അതിനകത്ത് എന്താണെന്ന് കാണാൻ പറ്റില്ല. FOPL-ൽ " x ഒരു കുട്ടിയാണ്" എന്നത് $\text{Child}(x)$ എന്ന predicate ആയി എഴുതാം, "എല്ലാ x -നും" എന്നതിന് $\forall x$ ഉപയോഗിക്കാം. അപ്പോൾ "12 വയസ്സിന് മുകളിലുള്ള എല്ലാ കുട്ടികളും exercise ചെയ്യണം" എന്ന പൊതുനിയമം Ram-ന് apply ചെയ്യാൻ കഴിയും.

Q1: $\neg(\exists x)F(x) \Rightarrow (\forall x)\neg F(x)$, and $(\forall x)\neg F(x) \Rightarrow \neg(\exists x)F(x)$

Q2: $\neg(\forall x)F(x) \Rightarrow (\exists x)\neg F(x)$, and $(\exists x)\neg F(x) \Rightarrow \neg(\forall x)F(x)$

Q3 (universal instantiation): $(\forall x)F(x) \Rightarrow F(a)$, a any arbitrary element

Q3' (universal generalisation): $F(a)$ for arbitrary $a \Rightarrow (\forall x)F(x)$

Q4 (existential instantiation): $(\exists x)F(x) \Rightarrow F(a)$, a a particular (new) constant

Q4' (existential generalisation): $F(a)$ for some $a \Rightarrow (\exists x)F(x)$

Steps: 1. Conceptualisation, 2. Nomenclature and translation, 3. Find a sequence of reasoning steps

EXAMPLE (RAM MUST EXERCISE)

(i) All children more than 12 years old must exercise regularly. (ii) Ram is more than 12 years old. Conclude: Ram must exercise regularly.

Predicates: $A(x)$ = x is a child more than 12 years old, $E(x)$ = x must exercise regularly.

(i) $(\forall x)(A(x) \rightarrow E(x))$ (ii) $A(\text{Ram})$

Q3 (universal instantiation) with $a = \text{Ram}$ on (i): $A(\text{Ram}) \rightarrow E(\text{Ram})$.

Modus Ponens with (ii): $E(\text{Ram})$.

EXAMPLE (THE UNIT'S EXAMPLE)

Given: (i) No feeling of pain is publicly observable. (ii) All chemical processes are publicly observable.

Prove: (iii) No feeling of pain is a chemical process.

$F(x)$: x is a feeling of pain; $O(x)$: x is publicly observable; $C(x)$: x is a chemical process.

(i) $(\forall x)(F(x) \rightarrow \neg O(x))$ (ii) $(\forall x)(C(x) \rightarrow O(x))$ To prove (iii) $(\forall x)(F(x) \rightarrow \neg C(x))$.

(iv) Universal instantiation on (i): $F(a) \rightarrow \neg O(a)$, a arbitrary.

(v) Universal instantiation on (ii): $C(a) \rightarrow O(a)$.

(vi) Transposition on (iv): $O(a) \rightarrow \neg F(a)$.

(vii) Chain rule on (v), (vi): $C(a) \rightarrow \neg F(a)$.

(viii) Transposition on (vii): $F(a) \rightarrow \neg C(a)$ (since $\neg\neg F(a) = F(a)$).

(ix) a is arbitrary, so universal generalisation: $(\forall x)(F(x) \rightarrow \neg C(x))$, which is (iii).

Problems with FOPL: it cannot easily represent

(i) properties of relations (e.g. "a symmetric and transitive relation may not be reflexive")

(ii) linguistic variables like hot, tall, sweet ("It is very cold today")

(iii) different belief systems ("she knows that he thinks India will win, but I think India will lose")

Exam tip: In the pain example the unit ends with $(\forall x)(C(x) \rightarrow \neg F(x))$ and calls it "the same as" (iii). It is only logically equivalent to (iii) = $(\forall x)(F(x) \rightarrow \neg C(x))$; one more transposition step (step viii above) is needed. The unit also says "generalized instantiation/quantification"; the correct names are universal instantiation and universal generalisation.

Shortcomings of predicate calculus

Sound: every conclusion the system infers is true.

Complete: every true conclusion can eventually be inferred.

Semi-decidable: if the goal is entailed, a proof is eventually found, but if it is not, the procedure may never terminate.

Sound: System infer ചെയ്യുന്ന എല്ലാ നിഗമനങ്ങളും true ആണ്.

Complete: True ആയ എല്ലാ നിഗമനങ്ങളും ഒടുവിൽ infer ചെയ്യാൻ കഴിയും.

Semi-decidable: Goal entailed ആണെങ്കിൽ ഒടുവിൽ proof കണ്ടെത്തും, പക്ഷേ അല്ലെങ്കിൽ procedure ഒരിക്കലും അവസാനിക്കണമെന്നില്ല.

Any logical reasoning system should ideally be **sound** (every inferred conclusion is true), **complete** (every true conclusion can eventually be inferred) and **tractable** (inference is feasible in reasonable time). Resolution refutation in predicate calculus is sound and complete, but if the goal is not entailed it may never terminate, so predicate calculus is only **semi-decidable**. Even when it terminates, it can need exponential time. AI researchers therefore relax these demands.

മലയാളത്തിൽ

Sound എന്നാൽ തെറ്റായ ഒന്നും "തെളിയിക്കില്ല". Complete എന്നാൽ ശരിയായ എല്ലാം ഒടുവിൽ തെളിയിക്കും. Tractable എന്നാൽ ന്യായമായ സമയത്തിനുള്ളിൽ ഉത്തരം കിട്ടും. Predicate logic-ൽ ഉത്തരം "ഇല്ല" ആണെങ്കിൽ program ചിലപ്പോൾ ഒരിക്കലും നിർത്തില്ല, അതാണ് semi-decidable. അതുകൊണ്ടാണ് expert systems കൂടുതലും ലളിതമായ IF-THEN rules (Horn clauses) ഉപയോഗിക്കുന്നത്.

Soundness → confidence that an inferred conclusion is true

Completeness → confidence that any true conclusion will eventually be produced

Tractability → confidence that inference is feasible

Remedies: (1) give up soundness, (2) give up completeness, (3) use a less expressive language, e.g. Horn clauses (at most one positive literal)

EXAMPLE

Which of these clauses are Horn clauses? (a) $\neg P \vee \neg Q \vee R$, (b) $P \vee Q$, (c) $\neg P \vee \neg Q$.

(a) One positive literal (R): Horn. It is the rule $P \wedge Q \rightarrow R$, exactly an IF-THEN rule. (b) Two positive literals: not Horn. (c) Zero positive literals: Horn (a goal/constraint clause).

MYCIN

Definition: MYCIN is an early rule-based expert system (Stanford, 1970s) that diagnoses bacterial blood infections and recommends antibiotics, using IF-THEN rules with certainty factors and backward chaining.

Bacterial blood infections diagnose ചെയ്യുകയും antibiotics നിർദ്ദേശിക്കുകയും ചെയ്യുന്ന ആദ്യകാല rule-based expert system ആണ് MYCIN (Stanford, 1970s); certainty factors ഉള്ള IF-THEN rules-ഉം backward chaining-ഉം ഇത് ഉപയോഗിക്കുന്നു.

MYCIN was one of the earliest expert systems, built at Stanford University in the 1970s (Edward Shortliffe, with Bruce Buchanan). Its job was to diagnose certain bacterial blood infections and recommend antibiotic treatment. Culture tests take a long time and the patient may be critical, so doctors must make quick guesses from partial data; MYCIN explored how experts make such judgements. Its knowledge was a set of IF-THEN rules (a few hundred) with **certainty factors**, written in Lisp. It is mainly **goal-directed** and uses **backward chaining**, plus heuristics to control the search.

മലയാളത്തിൽ

ICU-ൽ ഒരു രോഗിക്ക് രക്തത്തിൽ infection ഉണ്ട്; culture report വരാൻ രണ്ട് ദിവസം എടുക്കും, പക്ഷേ മരുന്ന് ഇപ്പോൾ തന്നെ തുടങ്ങണം. അപ്പോൾ ഒരു senior doctor ലഭ്യമായ ലക്ഷണങ്ങൾ വെച്ച് "ഏറ്റവും സാധ്യതയുള്ള bacteria ഇതാണ്" എന്ന് ഉറപ്പാക്കും. ആ ഉറപ്പാക്കൽ ചെയ്യാനാണ് MYCIN ഉണ്ടാക്കിയത്. ഓരോ rule-നും 0.8 പോലുള്ള ഒരു certainty factor ഉണ്ട്, "ഇത് ഏകദേശം 80% ഉറപ്പാണ്" എന്ന അർത്ഥത്തിൽ.

Sample MYCIN rule:

IF infection is primary-bacteria AND site of culture is one of the sterile sites AND suspected portal of entry is the gastrointestinal tract

THEN there is suggestive evidence (0.8) that the infection is bacteroid.

$CF(\text{conclusion}) = CF(\text{rule}) \times \min(CF \text{ of the conditions})$

Two rules supporting the same conclusion (both positive): $CF = CF_1 + CF_2(1 - CF_1)$

MYCIN drops a line of reasoning when certainty falls below a threshold, e.g. 0.2.

MYCIN's search heuristics:

1. First ask common, predefined questions to rule out unlikely diagnoses, then focus on specific ones and prove each by backward chaining.
2. Before using a rule, check whether any antecedent is already known to be false; if so, skip the rule.
3. Try rules with higher certainty first; abandon search when certainty drops below 0.2.

Three stages of a consultation: (1) gather initial data → broad diagnosis, (2) directed questions to test hypotheses → diagnosis, (3) questions to choose treatment → recommendation. At any stage the user may ask WHY or HOW, or ask for alternative treatments.

EXAMPLE (CERTAINTY FACTORS)

For the rule above (CF 0.8) the doctor is 0.9 sure the infection is primary-bacteria, 0.7 sure the site is sterile and 1.0 sure of the GI portal. A second rule (CF 0.5) whose conditions have certainties 0.8 and 1.0 also concludes "bacteroid". Find the combined certainty.

Rule 1: $0.8 \times \min(0.9, 0.7, 1.0) = 0.8 \times 0.7 = 0.56$.

Rule 2: $0.5 \times \min(0.8, 1.0) = 0.5 \times 0.8 = 0.40$.

Combined: $0.56 + 0.40 \times (1 - 0.56) = 0.56 + 0.176 = 0.736$. Both are above 0.2, so MYCIN keeps this hypothesis.

Exam tip: Remember the key MYCIN words: Stanford, 1970s, blood infections, IF-THEN rules with certainty factors, Lisp, backward chaining, WHY/HOW explanations. Its successors include NEOMYCIN, and its empty shell became EMYCIN. MYCIN performed well in evaluations but was never used routinely in hospitals. (Certainty factor theory is covered in Unit 7.)

COMPASS

Definition: COMPASS is an expert system that analyses error messages from a telephone switch's self-test routines to find the likely fault and suggest further tests or component replacement.

ഒരു telephone switch-ന്റെ self-test routines-ൽ നിന്നുള്ള error messages വിശകലനം ചെയ്ത്, സാധ്യമായ fault കണ്ടെത്തുകയും കൂടുതൽ tests അല്ലെങ്കിൽ component മാറ്റൽ നിർദ്ദേശിക്കുകയും ചെയ്യുന്ന expert system ആണ് COMPASS.

A telephone company's **switch** is a huge device whose circuits can fill a large part of a building. Maintenance aims to minimise calls rerouted due to bad connections and to repair faults quickly. **COMPASS** is an expert system that reads the error messages from the switch's self-test routines, looks for open circuits and reduces component down-time. Looking at a series of such messages, it finds the likely cause and suggests running more

tests or replacing a component such as a relay or printed circuit board. It was built because such expertise was scarce.

മലയാളത്തിൽ

BSNL exchange-ൽ ആയിരക്കണക്കിന് lines ഉണ്ട്. Machine തന്നെ self-test ചെയ്ത് കുറെ error messages print ചെയ്യും, പക്ഷേ അവ വായിച്ച് "ഈ relay ആണ് കേട്" എന്ന് പറയാൻ കുറച്ചു പരിചയസമ്പന്നർക്ക് മാത്രമേ കഴിയൂ. ആ വിദഗ്ധരുടെ അറിവ് program-ലാക്കിയതാണ് COMPASS. Knowledge acquisition എങ്ങനെ ചെയ്തു എന്നതിന് unit ഇതിനെ example ആയി എടുക്കുന്നു (topic 14).

COMPASS (Central Office Maintenance Printout Analysis and Suggestion System): built at GTE Laboratories in the 1980s for the GTE No. 2 EAX telephone switch.

Input: switch self-test error messages → Output: likely fault + suggested tests or component replacement

EXAMPLE (A COMPASS-STYLE RULE)

Write one plausible rule of the kind COMPASS uses.

IF several error messages in the last hour point to the same line card AND the card's self-test reports an open circuit THEN suggest replacing that printed circuit board. The rule turns a pattern across many messages into one action, which is exactly the expert's skill.

Other well-known expert systems

DENDRAL: usually called the first expert system (Stanford, 1965); it infers the molecular structure of organic compounds from mass-spectrometry data.

XCON (R1): a rule-based expert system built for DEC that configures VAX computer orders; the first big commercial success.

DENDRAL: സാധാരണയായി ആദ്യത്തെ expert system എന്ന് വിളിക്കപ്പെടുന്നു (Stanford, 1965); mass-spectrometry data-യിൽ നിന്ന് organic compounds-ന്റെ molecular structure infer ചെയ്യുന്നു.

XCON (R1): VAX computer orders configure ചെയ്യാൻ DEC-നായി നിർമ്മിച്ച rule-based expert system; ആദ്യത്തെ വലിയ commercial വിജയം.

The unit names MYCIN and COMPASS; these others are also commonly asked about. DENDRAL is usually called the first expert system, and XCON was the first big commercial success.

മലയാളത്തിൽ

Expert systems-ന്റെ ചരിത്രം 1965-ലെ DENDRAL-ൽ നിന്നാണ് തുടങ്ങുന്നത്; രാസവസ്തുക്കളുടെ ഘടന കണ്ടെത്താനായിരുന്നു അത്. പിന്നീട് DEC കമ്പനി computer orders ശരിയായി configure ചെയ്യാൻ XCON ഉപയോഗിച്ച് കോടിക്കണക്കിന് dollar ലാഭിച്ചു. ഇവയുടെ പേര്, വർഷം, domain എന്നിവ ഒരു table ആയി ഓർത്തുവെച്ചാൽ മതി.

System	Where / when	Task and domain	Notes
DENDRAL	Stanford, from 1965 (Feigenbaum, Lederberg, Buchanan, Djerassi)	Infers molecular structure of organic compounds from mass-spectrometry data	Usually called the first expert system
MYCIN	Stanford, 1970s (Shortliffe)	Diagnosis and therapy for bacterial blood infections	Certainty factors, backward chaining, Lisp
PROSPECTOR	SRI International, late 1970s (Duda, Hart)	Mineral exploration (geology)	Helped locate a molybdenum deposit (Mount Tolman, 1982)
XCON (R1)	Carnegie Mellon (John McDermott) for DEC, in use from 1980	Configures VAX computer orders	Rule-based in OPS; estimated savings about \$25 million a year by the mid-1980s
COMPASS	GTE Laboratories, 1980s	Telephone switch maintenance	Unit's knowledge-acquisition example
EMYCIN	Stanford, about 1980 (van Melle)	Shell: MYCIN without medical knowledge	Used to build PUFF (lung function) and SACON (structural analysis)
NEOMYCIN	Stanford, early 1980s (Clancey)	Improved MYCIN for diagnosis and teaching	Separates strategy knowledge from domain rules

EXAMPLE

Match: (1) DENDRAL (2) XCON (3) PROSPECTOR (4) MYCIN with (a) configuration (b) chemistry (c) medicine (d) geology.

1-b, 2-a, 3-d, 4-c.

Expert system building tools

Definition: Expert system building tools are software environments for building expert systems, mainly by prototyping; they are expert system shells, high-level programming languages and multiple programming environments.

പ്രധാനമായും prototyping വഴി expert systems നിർമ്മിക്കാനുള്ള software environments ആണ് expert system building tools; expert system shells, high-level programming languages, multiple programming environments എന്നിവയാണ് ഇവ.

Expert system tools give an environment for building expert systems, mainly by **prototyping**. Development is usually a mix of prototyping and incremental development, not the classical waterfall model. The problem of fitting new features into an earlier version is reduced by keeping knowledge **modular** (rules, frames). Tools fall into three categories.

മലയാളത്തിൽ

ഒരു expert system ഒറ്റയടിക്ക് പൂർത്തിയാക്കാൻ പറ്റില്ല. ആദ്യം ചെറിയ ഒരു prototype ഉണ്ടാക്കി expert-നെ കാണിക്കും, അവർ തിരുത്തും, പിന്നെ rules കൂട്ടും. Rules ഓരോന്നും വേറിട്ട കഷണങ്ങളായതിനാൽ (modular) പുതിയ rule ചേർക്കുന്നത് എളുപ്പമാണ്. ഇതിന് സഹായിക്കുന്ന software tools മൂന്ന് തരം: shells, high-level languages, multiple programming environments.

Category	What it is	Example
Expert system shells	A set of programs abstracted from existing applications: inference engine + empty knowledge base + tools	EMYCIN (rule interpreter of MYCIN); M.4 (backward chaining of EMYCIN + frame-like structures)
High-level programming languages	Hide implementation details, so the programmer need not worry about efficient storage and access of data; many are research tools	OPS5 rule language
Multiple programming environments	Modules that let you mix several AI programming styles	LOOPS (rule-based + object-oriented)

EXAMPLE

A Krishi Bhavan wants a pest-advice system quickly, with no AI programmers. Which tool category fits?

An expert system shell. The inference engine, explanation facility and user interface already exist; the team only has to enter the agricultural officer's rules. This saves both time and cost compared with building from scratch.

Expert system shell and its architecture

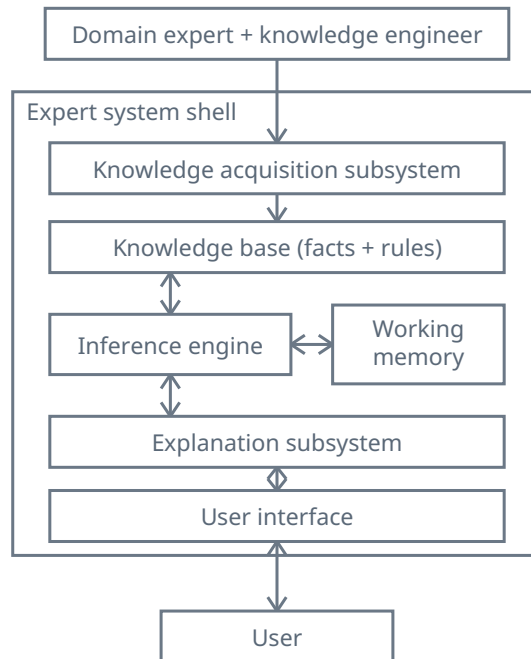
Definition: An expert system shell is a software development environment containing the basic components of an expert system, with a prescribed method for building applications by configuring them; expert system = shell + domain knowledge.

ഒരു expert system-ന്റെ അടിസ്ഥാന components ഉൾക്കൊള്ളുന്നതും, അവ configure ചെയ്ത് applications നിർമ്മിക്കാൻ നിശ്ചിത രീതിയുള്ളതുമായ software development environment ആണ് expert system shell; expert system = shell + domain knowledge.

An **expert system shell** is a software development environment containing the basic components of an expert system, with a prescribed method for building applications by configuring these components. Shells let non-programmers reuse templates built by earlier developers who solved similar problems. For a new problem you pick a suitable shell and add only the expert's knowledge. The core components are the **knowledge base** and the **reasoning (inference) engine**.

മലയാളത്തിൽ

Shell എന്നാൽ "ഒഴിഞ്ഞ ചിരട്ട" പോലെ; അകത്ത് അറിവില്ല, ബാക്കി എല്ലാ സംവിധാനങ്ങളും ഉണ്ട്. ഒരു readymade shop counter-ഉം billing machine-ഉം വാങ്ങി, അതിൽ നിങ്ങളുടെ സാധനങ്ങൾ മാത്രം നിറച്ചാൽ കട തുടങ്ങാം എന്നതുപോലെ. Knowledge base മാറ്റിയാൽ അതേ shell കൊണ്ട് medicine-നോ agriculture-നോ ഉള്ള expert system ഉണ്ടാക്കാം.



Components of an expert system shell (the unit's Fig. 9.2, drawn vertically). Working memory holds the case-specific facts of the current consultation.

Components: knowledge base, knowledge acquisition subsystem, inference engine, explanation subsystem, user interface

Expert system = shell + domain knowledge

Benefit: much lower development cost and time than building from scratch

EXAMPLE (FOLLOWING ONE QUESTION THROUGH THE ARCHITECTURE)

A farmer types "my paddy leaves have spots". Trace the flow.

1. User interface takes the answer and stores "leaf-spots = yes" in working memory.
2. Inference engine matches working memory against rules in the knowledge base and decides the next thing to ask (spot shape).
3. If the farmer types WHY, the explanation subsystem shows the rule being tried.
4. The rules themselves were entered earlier by the agricultural officer through the knowledge acquisition subsystem.

Exam tip: "Draw the architecture of an expert system and explain each component" is a standard 10-mark question. Draw the boxes in this order and write 2-3 lines on each of the five components (topics 13-17).

Knowledge base

Definition: The knowledge base is the component of an expert system that contains the facts and heuristic knowledge of the domain, usually as IF-THEN rules and/or frames.

Domain-ലെ facts-ഉം heuristic knowledge-ഉം, സാധാരണയായി IF-THEN rules ആയോ frames ആയോ, ഉൾക്കൊള്ളുന്ന expert system component ആണ് knowledge base.

The **knowledge base** contains facts and heuristic knowledge about the domain. Developers try to keep one uniform representation as far as possible. Many schemes are possible; advanced shells use both **frames (objects)** and **IF-THEN rules**, while in PROLOG knowledge is written as logical statements.

മലയാളത്തിൽ

Knowledge base ആണ് expert system-ന്റെ "തലച്ചോറിലെ അറിവ്". "നെല്ലിന്റെ ഇലയിൽ ചാരനിറത്തിലുള്ള നടുവോടെ spindle ആകൃതിയിലുള്ള പാടുകൾ ഉണ്ടെങ്കിൽ അത് blast ആണ്" എന്ന പോലുള്ള rules, "Kuttanad-ൽ പുഞ്ച കൃഷി ഡിസംബറിൽ തുടങ്ങും" എന്ന പോലുള്ള facts എല്ലാം ഇവിടെയാണ്. ഇതിൽ രണ്ട് തരം അറിവുണ്ട്: പുസ്തകത്തിലുള്ള facts-ഉം, പരിചയത്തിൽ നിന്ന് കിട്ടിയ heuristics-ഉം.

Knowledge base = facts + heuristic knowledge (rules of thumb)

Rule form: IF condition(s) THEN conclusion/action

Same idea in PROLOG: disease(blast) :- leaf_spots, spindle_spots.

EXAMPLE

Write the fact "Jyothi is a paddy variety" and the rule "IF the crop is paddy AND the field is waterlogged for weeks THEN risk of bacterial leaf blight is high" in rule form.

Fact: variety(jyothi, paddy). Rule: IF crop = paddy AND waterlogged = long THEN blb-risk = high. The fact is textbook knowledge; the rule is heuristic knowledge from experience.

Knowledge acquisition subsystem (COMPASS)

Definition: Knowledge acquisition is capturing useful information for a problem from a knowledge source (often a human expert) and transforming it into the format the program needs.

ഒരു knowledge source-ൽ നിന്ന് (പലപ്പോഴും ഒരു human expert) ഒരു problem-ന് ഉപകാരപ്രദമായ വിവരങ്ങൾ ശേഖരിച്ച്, program-ന് ആവശ്യമായ format-ലേക്ക് മാറ്റുന്നതാണ് knowledge acquisition.

Knowledge acquisition is capturing useful information for a problem from a knowledge source (often a human expert) and transforming it into the format the program needs. The knowledge acquisition subsystem helps experts build knowledge bases. Because experts may not be computer-literate, it involves interviews and questionnaires, which are slow, so knowledge acquisition has always been the **biggest bottleneck** in building expert systems. Machine learning is one route to automating it.

മലയാളത്തിൽ

ഒരു പഴയ കർഷകനോട് "എപ്പോഴാണ് വിതയ്ക്കേണ്ടത്?" എന്ന് ചോദിച്ചാൽ "മണ്ണ് കണ്ടാൽ അറിയാം" എന്നാകും ഉത്തരം. ആ അറിവ് rules ആയി എഴുതിയെടുക്കാൻ വളരെ പ്രയാസമാണ്. അതുകൊണ്ടാണ് knowledge acquisition-നെ bottleneck എന്ന് വിളിക്കുന്നത്. COMPASS-ൽ expert പറയുന്നത് knowledge engineer rule ആക്കി, expert പരിശോധിച്ച്, തെറ്റുണ്ടെങ്കിൽ വീണ്ടും തിരുത്തി, ഈ ചക്രം ആവർത്തിച്ചു.

Why knowledge acquisition is difficult:

1. Domain facts and principles do not easily convert into a mathematical or deterministic model (e.g. how a teacher motivates students).
2. Each domain has its own jargon; experts find it hard to express knowledge in plain language.
3. Facts alone are not enough; experience-based judgements (which sources are reliable, how to simplify a problem) are hard to capture.
4. Commonsense knowledge is very hard to capture.

COMPASS knowledge acquisition cycle:

1. Extract knowledge from the human expert (the expert explains, the knowledge engineer writes an IF-THEN rule, the expert checks it).
2. Document the extracted knowledge.
3. Test it: (a) the expert analyses new data; (b) the same data is analysed with pencil and paper using the documented rules; (c) compare the two results; (d) if they differ, find the rules causing the discrepancy and go back to step 1; (e) otherwise exit.

EXAMPLE (ONE TURN OF THE CYCLE)

An agricultural officer says: "If the leaves yellow from the tips, it is bacterial leaf blight." The engineer writes R: IF yellowing-from-tips THEN blb. Test: on a new field report (yellow tips, no wavy margins, recent heavy rain but plants also short of nitrogen) the officer says "nitrogen deficiency", but hand simulation of R says "blb".

The results differ, so R is the culprit. Back to step 1: the officer adds that blight yellowing has wavy margins. New rule: IF yellowing-from-tips AND wavy-margins THEN blb. Re-test: hand simulation no longer wrongly says "blb". Since the system now gives no conclusion where the officer said "nitrogen deficiency", the next turn of the cycle adds a rule for that disorder.

Exam tip: Learn the COMPASS cycle as a numbered list; it is asked as "describe the knowledge acquisition process with an example". Hand (pencil-and-paper) simulation is used first because it is cheaper than implementing and re-implementing rules.

Inference engine

Inference engine: the component that reasons with the expert knowledge (rules) and the case-specific data to reach conclusions.

Forward chaining: data-driven reasoning: start from known facts and fire rules whose IF parts match, adding their conclusions, until no rule can fire.

Backward chaining: goal-driven reasoning: start from a goal, find rules that conclude it and try to prove their conditions as sub-goals.

Inference engine: Expert knowledge (rules), case-specific data എന്നിവ ഉപയോഗിച്ച് reason ചെയ്ത് നിഗമനങ്ങളിൽ എത്തുന്ന component.

Forward chaining: Data-driven reasoning: അറിയാവുന്ന facts-ൽ നിന്ന് തുടങ്ങി, IF part match ചെയ്യുന്ന rules fire ചെയ്ത് conclusions ചേർക്കുന്നു, ഒരു rule-ഉം fire ചെയ്യാൻ കഴിയാത്തതുവരെ.

Backward chaining: Goal-driven reasoning: ഒരു goal-ൽ നിന്ന് തുടങ്ങി, അത് conclude ചെയ്യുന്ന rules കണ്ടെത്തി, അവയുടെ conditions sub-goals ആയി prove ചെയ്യാൻ ശ്രമിക്കുന്നു.

The **inference engine** reasons with two kinds of input: the **expert knowledge** (mostly IF-THEN rules) and the **case-specific data** (user's answers plus partial conclusions with their certainty factors). In a forward-chaining rule-based system the case data are the elements of **working memory**. Knowledge about what is known and when and how to use it is called **meta-knowledge**; it controls which questions to ask and when. The control strategy strongly affects performance.

മലയാളത്തിൽ

Knowledge base പുസ്തകം പോലെയാണെങ്കിൽ inference engine അത് വായിച്ച് ചിന്തിക്കുന്ന ആളാണ്. രണ്ട് രീതികളുണ്ട്. Forward chaining: കയ്യിലുള്ള facts-ൽ നിന്ന് തുടങ്ങി rules fire ചെയ്ത് മുന്നോട്ട് പോകുന്നു, detective തെളിവുകളിൽ നിന്ന് കുറ്റവാളിയിലേക്ക് എത്തുന്നതുപോലെ. Backward chaining: "ഇത് blast ആണോ?" എന്ന goal-ൽ നിന്ന് തുടങ്ങി അതിന് വേണ്ട തെളിവുകൾ അന്വേഷിക്കുന്നു, doctor ഒരു സംശയം ഉറപ്പിക്കാൻ tests ചോദിക്കുന്നതുപോലെ. MYCIN backward chaining ഉപയോഗിക്കുന്നു.

Forward chaining	Backward chaining
Data-driven: starts from known facts	Goal-driven: starts from a hypothesis (goal)
Fires every rule whose IF part matches working memory, adds THEN part as new fact	Finds rules whose THEN part is the goal and tries to prove their IF parts as sub-goals
Good when there are few facts and many possible conclusions (monitoring, configuration: XCON)	Good when there are few possible goals to check (diagnosis: MYCIN)
May derive many irrelevant facts	Asks only questions relevant to the current goal
Natural question: "What follows?"	Natural questions: WHY (this question)? HOW (this conclusion)?

Forward chaining cycle: match (find rules whose conditions are all in working memory = conflict set) → select one (conflict resolution) → fire (add conclusion to working memory) → repeat until no rule can fire
 Backward chaining: goal → rule with that conclusion → prove each condition (from memory, from another rule, or by asking the user)

EXAMPLE (FORWARD CHAINING, PADDY RULES OF TOPIC 20)

Rules: R3: IF yellowing-from-tips AND wavy-margins THEN blb. R5: IF blb THEN advice-B. Initial working memory: {yellowing-from-tips, wavy-margins}.

Cycle 1: conflict set = {R3}. Fire R3; working memory gets blb.

Cycle 2: conflict set = {R5}. Fire R5; working memory gets advice-B.

Cycle 3: no unfired rule matches; stop. Conclusion: bacterial leaf blight, give advice B. (Checked by a small simulation.)

EXAMPLE (META-KNOWLEDGE)

Give one piece of meta-knowledge a paddy-advice system might use.

"Ask about visible leaf symptoms before asking about fertiliser history, because symptoms rule out most diseases at once." This is knowledge about how to use the domain knowledge, not about paddy itself, like MYCIN's heuristic of asking common questions first.

Explanation subsystem (car example, MYCIN)

Definition: The explanation subsystem is the component that explains the system's reasoning: HOW a conclusion was reached and WHY a question is being asked.

System-ന്റെ reasoning വിശദീകരിക്കുന്ന component ആണ് explanation subsystem: ഒരു conclusion-ൽ HOW എത്തി, ഒരു ചോദ്യം WHY ചോദിക്കുന്നു എന്നിവ.

The **explanation subsystem** lets the program explain its reasoning: how it reached final or intermediate conclusions, and why it needs extra data. Users can ask **WHY** (why is this question asked?) and **HOW** (how was this conclusion reached?); answers refer to the system's goals, the rules in use and the problem data. It also helps evaluation: from the trace we can judge whether the system gives the right answer for the right reasons.

മലയാളത്തിൽ

ഒരു നല്ല doctor "ഈ test എന്തിനാണ്?" എന്ന് ചോദിച്ചാൽ കാരണം പറഞ്ഞുതരും. അതുപോലെ expert system-നോട് WHY ചോദിച്ചാൽ "ഞാൻ battery problem ആണോ എന്ന് നോക്കുകയാണ്" എന്ന് പറയും. HOW ചോദിച്ചാൽ "Rule 2 പ്രകാരം, നിങ്ങൾ തന്ന ഉത്തരങ്ങൾ വെച്ച്" എന്ന് പറയും. ഇങ്ങനെ വിശദീകരിക്കുമ്പോഴാണ് ആളുകൾ system-നെ വിശ്വസിക്കുന്നത്.

Why explanation subsystems are important:

- (i) Proper use of knowledge: assures knowledge engineers the knowledge is applied properly, even in a prototype
- (ii) Correctness of conclusions: users can satisfy themselves the conclusions are right
- (iii) Execution trace: shows whether knowledge elicitation is going well
- (iv) Knowledge of program behaviour: needed for maintenance and debugging
- (v) Suitability of reasoning approach: shows whether the reasoning technique suits the domain

EXAMPLE (THE UNIT'S CAR DIAGNOSIS SYSTEM)

R1: IF engine-gets-petrol AND engine-turns-over THEN spark-plug-fault

R2: IF NOT engine-turns-over AND NOT lights-come-on THEN battery-problem

R3: IF NOT engine-turns-over AND lights-come-on THEN starter-not-working

R4: IF fuel-tank-has-petrol THEN engine-gets-petrol

No initial facts. Backward chaining tries the goals in the order R1, R2, R3.

```
System: Does the fuel tank have petrol?      (for R4, to get R1's first condition)
User:   Yes.                                R4 fires: engine-gets-petrol
System: Does the engine turn over?          (R1's second condition)
User:   No.                                R1 fails; try R2
System: Do the lights come on?
User:   Why?
System: I am trying to prove there is a battery problem (R2).
        It is established that the engine does not turn over.
        If the lights do not come on, there is a battery problem.
        Do the lights come on?
User:   No.
System: I conclude that there is a battery problem.
User:   How?
System: By Rule 2: NOT engine-turns-over and NOT lights-come-on
        were both given by the user.
```

Checked by simulation: backward chaining over R1-R4 with these answers asks exactly these three questions in this order and concludes battery-problem.

Explanation in MYCIN (backward chaining = goal tree):

WHY → look UP the tree: which goal is this question serving?

HOW → look DOWN the tree: which sub-goals (rules, answers) established this conclusion?

So explanation is a search problem: tree traversal. It needs a record of rule invocations and which question each belongs to.

E.g. "Why did you ask whether the organism's stain is gram-negative?" → quotes the rule using gram-negative staining and the goal it serves.

"How do you say Organism-2 might be proteus?" → lists the rules applied, their certainties, and the last question asked.

Why explanations are sometimes not accepted:

1. They cite only "surface" knowledge (the rule), not the "deep" domain knowledge behind it (e.g. R2 exists because both the lights and the starter use the battery).

2. If the user does not understand, the system cannot re-explain in a different way, as a person can.

Exam tip: In the unit's MYCIN paragraph both sentences say "Why" because of a typo; the second must be "How": WHY goes up the goal tree, HOW goes down it. Long reasoning chains make these explanations hard to follow.

User interface and EMYCIN

User interface: the component that handles communication between the expert system and the user.

EMYCIN: "Empty MYCIN": an expert system shell formed by removing MYCIN's medical knowledge, giving a domain-independent framework for building consultation programs.

User interface: Expert system-ഉം user-ഉം തമ്മിലുള്ള ആശയവിനിമയം കൈകാര്യം ചെയ്യുന്ന component.

EMYCIN: "Empty MYCIN": MYCIN-ന്റെ medical knowledge നീക്കം ചെയ്ത് ഉണ്ടാക്കിയ expert system shell; consultation programs നിർമ്മിക്കാനുള്ള domain-independent framework.

The **user interface** handles communication with the user. It was once ignored as "not part of expert system technology", but it is now accepted that the interface can make a critical difference to a system's usefulness, whatever its reasoning power. **EMYCIN** ("Empty MYCIN" or "Essential MYCIN") is a shell: MYCIN minus its medical knowledge, giving a domain-independent framework for building and running consultation programs, plus tools for building and debugging them.

മലയാളത്തിൽ

എത്ര നല്ല system ആണെങ്കിലും കർഷകന് മനസ്സിലാക്കാത്ത English-ൽ ചോദ്യം ചോദിച്ചാൽ ആരും ഉപയോഗിക്കില്ല; മലയാളത്തിൽ, ലളിതമായ ചോദ്യങ്ങളോടെയുള്ള interface ആണ് വിജയം നിശ്ചയിക്കുന്നത്. EMYCIN എന്നത് MYCIN-ൽ നിന്ന് medical rules എടുത്തുമാറ്റിയ ഒഴിഞ്ഞ ചട്ടക്കൂടാണ്. അതിൽ lung രോഗങ്ങളുടെ rules നിറച്ചപ്പോൾ PUFF എന്ന പുതിയ expert system ഉണ്ടായി.

Characteristics of EMYCIN:

1. Abbreviated rule language in ALGOL-like notation: easier than LISP, more concise than MYCIN's English subset.
2. Backward chaining, like MYCIN.
3. Indexes rules and groups them by the parameters they refer to.
4. System-designer interface: display, edit and partition rules, edit knowledge tables, run rule sets on sets of problems; includes the knowledge editor TEIRESIAS to help build and maintain large knowledge bases.
5. A user interface for smooth consultation.

EXAMPLE

Explain why EMYCIN is "more than an empty MYCIN".

Removing MYCIN's rules leaves only the inference engine. EMYCIN adds tools for the designer: a simpler rule language, rule indexing, rule editing and partitioning, table editing, batch testing of rule sets, and TEIRESIAS for knowledge-base maintenance. These make building a new system (such as PUFF) much faster.

Application areas of expert systems

Diagnosis and troubleshooting: systems that deduce faults and suggest corrective action.

Process monitoring and control: systems that analyse real-time sensor data, notice errors, predict trends and control the process.

Knowledge publishing: systems that deliver knowledge relevant to the user's problem, in its context.

Diagnosis and troubleshooting: തകരാറുകൾ കണ്ടെത്തി പരിഹാര നടപടികൾ നിർദ്ദേശിക്കുന്ന systems.

Process monitoring and control: Real-time sensor data വിശകലനം ചെയ്ത് errors കണ്ടെത്തുകയും trends പ്രവചിക്കുകയും process നിയന്ത്രിക്കുകയും ചെയ്യുന്ന systems.

Knowledge publishing: User-ന്റെ problem-ന് പ്രസക്തമായ knowledge അതിന്റെ സന്ദർഭത്തിൽ എത്തിച്ചുകൊടുക്കുന്ന systems.

Expert systems appear in most areas of knowledge work. The unit lists six main categories.

മലയാളത്തിൽ

ആശുപത്രി മുതൽ bank വരെ, factory മുതൽ airport വരെ expert systems ഉപയോഗിക്കുന്നു. തകരാർ കണ്ടുപിടിക്കൽ (diagnosis), ജോലിക്രമം തയ്യാറാക്കൽ (scheduling), യന്ത്രങ്ങൾ നിരീക്ഷിക്കൽ (monitoring), loan തീരുമാനങ്ങൾ, ശരിയായ അറിവ് ശരിയായ സമയത്ത് എത്തിക്കൽ, design എന്നിവയാണ് പ്രധാന മേഖലകൾ.

Category	What the system does	Example
Diagnosis and troubleshooting	Deduces faults and suggests corrective action	MYCIN (medicine), COMPASS (telephone switch), car diagnosis
Planning and scheduling	Analyses interacting goals to find a set of actions	Flight and crew scheduling, manufacturing process planning
Process monitoring and control	Analyses real-time sensor data, notices errors, predicts trends, controls	Steel-making, oil refining
Financial decision making	Acts as advisor or risk analyser	Loan approval, fraud alerts
Knowledge publishing	Delivers knowledge relevant to the user's problem, in context	Advice systems for tax rules or crop practices
Design and manufacturing	Helps design devices and processes, from concept to factory floor	XCON-style configuration

EXAMPLE

Classify: (a) a system watching temperature sensors in a rubber factory boiler, (b) a bank's system deciding whether a small-business loan is risky, (c) a system that suggests which part of a lift is faulty.

(a) Process monitoring and control. (b) Financial decision making. (c) Diagnosis and troubleshooting.

Advantages and limitations

Definition: Brittleness is the limitation that an expert system works only within its narrow domain and fails badly on problems just outside it, because it lacks common sense.

Commonsense ഇല്ലാത്തതിനാൽ ഒരു expert system അതിന്റെ ഇടുങ്ങിയ domain-ൽ മാത്രം പ്രവർത്തിക്കുകയും തൊട്ടുപുറത്തുള്ള problems-ൽ മോശമായി പരാജയപ്പെടുകയും ചെയ്യുന്ന limitation ആണ് brittleness.

The unit's discussion implies clear benefits and weaknesses; examiners often ask for them as a list. Expert systems make scarce expertise available anywhere, at any time, consistently and with explanations. But they are narrow, costly to build (knowledge acquisition bottleneck), lack common sense, and only give surface explanations.

മലയാളത്തിൽ

ഒരു expert system-ന് ക്ഷീണമില്ല, ലീവ് ഇല്ല, ഒരേ ചോദ്യത്തിന് എന്നും ഒരേ ഉത്തരം. വിദൂര ഗ്രാമത്തിലും അത് ലഭ്യമാക്കാം. പക്ഷേ തന്റെ domain-ന് പുറത്തുള്ള ഒരു കാര്യം വന്നാൽ അതിന് ഒന്നും അറിയില്ല, സാമാന്യബുദ്ധിയും ഇല്ല. പുതിയ അറിവ് ഉണ്ടാകുമ്പോൾ ആരെങ്കിലും rules update ചെയ്യുകയും വേണം.

Advantages	Limitations
Makes scarce expertise available at many places and at all times	Works only in a narrow domain; fails badly just outside it
Consistent: no fatigue, mood or forgetting	Knowledge acquisition is slow and costly (the bottleneck)
Fast decisions; helps experts decide quickly	No commonsense knowledge
Explains its reasoning (WHY/HOW); useful for training	Explanations are "surface" (rule-level), cannot re-explain differently
Preserves the knowledge of retiring experts	Needs manual updating; does not learn by itself
Knowledge is modular: easy to add or change rules	Heuristic: may give wrong answers; legal responsibility is unclear

EXAMPLE

A paddy-disease system is asked about yellowing in a banana plant. What happens and which limitation is this?

No rule concludes anything about banana, or worse, a paddy rule matches the symptom words and gives a paddy diagnosis. This is the narrow-domain (brittleness) limitation; a human expert would simply say "that is not my field".

Case study: a paddy-disease expert system for Kerala farmers

This small system ties together the knowledge base, backward-chaining inference engine, working memory and WHY/HOW explanations. The goal is to reach an **advice**. The rules below are simplified for teaching, not agronomic advice. Both consultations were checked by a small backward- and forward-chaining simulation.

മലയാളത്തിൽ

Kuttanad-ലെ ഒരു കർഷകൻ Krishi Bhavan-ൽ പോകാതെ തന്നെ phone-ൽ ചില ചോദ്യങ്ങൾക്ക് ഉത്തരം നൽകി തന്റെ നെല്ലിന്റെ രോഗം അറിയുന്നു എന്ന് കരുതുക. System goal-ൽ നിന്ന് പിന്നോട്ട് ചിന്തിച്ച് (backward chaining) ആവശ്യമുള്ള ചോദ്യങ്ങൾ മാത്രം ചോദിക്കും. കർഷകൻ WHY എന്ന് ചോദിച്ചാൽ കാരണം പറയും, HOW എന്ന് ചോദിച്ചാൽ ഏത് rules ഉപയോഗിച്ചു എന്ന് കാണിക്കും.

R1: IF leaf-spots AND spots-spindle-shaped-with-grey-centre THEN disease = blast

R2: IF leaf-spots AND spots-oval-brown THEN disease = brown-spot

R3: IF yellowing-from-leaf-tips AND wavy-leaf-margins THEN disease = bacterial-leaf-blight

R4: IF disease = blast AND heavy-urea-applied THEN advice A: stop extra urea top-dressing; take a leaf sample to Krishi Bhavan for fungicide advice

R5: IF disease = bacterial-leaf-blight THEN advice B: drain standing water, avoid extra nitrogen, report to Krishi Bhavan

Goals tried in order: advice A (R4), then advice B (R5). Working memory starts empty.

EXAMPLE (CONSULTATION 1: BACKWARD CHAINING WITH WHY AND HOW)

```
Goal: advice A → R4 needs "disease = blast" → R1 needs leaf-spots
System: Are there spots on the leaves? Farmer: Yes
System: Are the spots spindle-shaped with a grey centre? Farmer: Yes
R1 fires → working memory: disease = blast
System: Did you apply a heavy dose of urea recently? Farmer: Why?
System: I am trying to find advice A (Rule 4). It has been
established that the disease is blast (Rule 1). If heavy
urea was applied, advice A applies. Did you apply it? Farmer: Yes
R4 fires → advice A
System: Stop extra urea; take a leaf sample to Krishi Bhavan. Farmer: How?
System: Rule 4: disease = blast (from Rule 1: leaf spots and
spindle spots with grey centre, both given by you) and
heavy urea applied (given by you).
```

Only 3 questions were needed; R2, R3 and R5 were never touched because advice A succeeded first. The WHY answer looks up the goal tree (R4 is the parent goal); the HOW answer looks down it (R1 and the farmer's answers).

EXAMPLE (CONSULTATION 2: SAME FARMER ANSWERS "NO" TO UREA)

Questions 1-2 as before, so R1 fires (disease = blast). Question 3 (urea): No, so R4 fails. Next goal advice B: R5 needs bacterial-leaf-blight, so R3 is tried: "Are leaves yellowing from the tips?" No, so R3 fails, R5 fails.

Result: the system knows the disease is blast but has no advice to give. This is a knowledge gap, found by testing, exactly as in the COMPASS cycle. Fix: add R6: IF disease = blast AND NOT heavy-urea-applied THEN advice C (take a leaf sample to Krishi Bhavan for fungicide advice). Brown spot (R2) similarly has no advice rule yet.

EXAMPLE (CONSULTATION 3: FORWARD CHAINING FROM REPORTED SYMPTOMS)

A farmer sends a message: "leaves yellowing from the tips, margins wavy". Working memory = {yellowing-from-leaf-tips, wavy-leaf-margins}. R3 fires → bacterial-leaf-blight; then R5 fires → advice B; then nothing else matches, stop. Forward chaining suits this case because the facts arrive first, with no question-answer dialogue.

Exam tip: If asked to "design a small rule-based expert system", write 4-6 IF-THEN rules, state the goal and chaining method, and show one traced consultation with at least one WHY and one HOW. That covers knowledge base, inference engine and explanation in one answer.

Quick recap

- Expert system: a program holding domain knowledge of an expert and reasoning with it to solve problems; narrow domain, heuristic, explains itself.
- Knowledge-based system uses rules of thumb on symbolic knowledge; "expert system" stresses knowledge from human experts. Building one is knowledge engineering.
- Task = goal-oriented activity (diagnosis, planning, configuration, design); domain = area where it is done.
- Classical planning environments: fully observable, deterministic, finite, static, discrete.
- Good representation: expressive, supports inference, well-defined syntax and semantics. Schemes: semantic nets, frames, FOL, rules.
- PL rules: Modus Ponens, chain rule, transposition ($P \rightarrow Q$ gives $\neg Q \rightarrow \neg P$). FOPL adds quantifier rules Q1-Q4 (instantiation, generalisation).
- Predicate calculus is sound and complete but semi-decidable and intractable; remedy: Horn clauses (at most one positive literal) = IF-THEN rules.
- MYCIN: Stanford 1970s, blood infections, rules with certainty factors, Lisp, backward chaining, threshold 0.2, WHY/HOW.
- COMPASS: GTE telephone-switch maintenance; example of the knowledge acquisition cycle (extract, document, test by hand simulation, repeat).
- DENDRAL (chemistry, 1965, first ES), XCON/R1 (VAX configuration, DEC), PROSPECTOR (geology).

- Tools: shells (EMYCIN, M.4), high-level languages (OPS5), multiple programming environments (LOOPS).
- Shell components: knowledge base, knowledge acquisition subsystem, inference engine, explanation subsystem, user interface.
- Knowledge acquisition is the biggest bottleneck. Meta-knowledge = knowledge about when and how to use knowledge.
- WHY goes up the goal tree, HOW goes down it. EMYCIN = "Empty/Essential MYCIN"; includes TEIRESIAS knowledge editor.

Check yourself

Q. 1. Define an expert system. List its basic properties.

An expert system is a computer program that possesses or represents knowledge in a particular domain and can process or reason with this knowledge to solve a problem or achieve a specific goal.

Properties: (1) it simulates human reasoning about a specific domain rather than the domain itself; (2) it reasons over acquired knowledge rather than only calculating or retrieving data; (3) it uses heuristic or approximate models that, unlike algorithms, are not guaranteed to succeed.

Q. 2. Differentiate between a knowledge-based system and an expert system. What is knowledge engineering?

A KBS performs a task by applying rules of thumb to a symbolic representation of knowledge. An expert system is a KBS whose knowledge comes from human experts and which achieves expert-level performance on tasks needing domain expertise (a weather-chat program is a KBS; a weather-forecasting program is an expert system). In practice the terms are often used interchangeably.

Knowledge engineering is the process of building an expert system: acquiring knowledge from experts, choosing representation schemes and reasoning methods. Its practitioners are knowledge engineers.

Q. 3. Using propositional logic, show that from "There is a moral law", "If there is a moral law, someone gave it" and "If someone gave the moral law, there is God", it follows that "There is God".

Let ML = there is a moral law, SG = someone gave it, TG = there is God. Premises: ML, $ML \rightarrow SG$, $SG \rightarrow TG$.

Modus Ponens on ML and $ML \rightarrow SG$ gives SG. Modus Ponens on SG and $SG \rightarrow TG$ gives TG. Hence proved.

Q. 4. Draw the architecture of an expert system shell and explain its components.

Draw the diagram of topic 12: expert → knowledge acquisition subsystem → knowledge base ↔ inference engine (with working memory) ↔ explanation subsystem ↔ user interface ↔ user.

Knowledge base: facts and heuristic rules. Knowledge acquisition subsystem: captures expert knowledge and converts it into the program's format. Inference engine: reasons with rules and case data (forward/backward chaining), guided by meta-knowledge. Explanation subsystem: answers WHY and HOW. User interface: communicates with the user; strongly affects usefulness.

Q. 5. Why is knowledge acquisition called the bottleneck of expert system development? Describe the knowledge acquisition cycle used in COMPASS.

Experts are often not computer-literate, so knowledge is gathered by slow interviews and questionnaires. Also: domain knowledge does not convert easily into a deterministic model; domains have their own jargon; experience-based judgement is hard to put into words; commonsense is very hard to capture.

COMPASS cycle: (1) extract knowledge from the expert and write it as IF-THEN rules checked by the expert; (2) document it; (3) test: let the expert analyse new data, analyse the same data by pencil-and-paper using the rules, compare; if they differ, find the responsible rules and go back to step 1; otherwise exit.

Q. 6. What is an explanation subsystem? Why is it important? Illustrate WHY and HOW with the car example.

It lets the system explain how it reached a conclusion and why it needs some data. Importance: proper use of knowledge, correctness of conclusions, execution trace, knowledge of program behaviour for maintenance, suitability of the reasoning approach.

With rules R1-R4 of topic 16, after the user says the tank has petrol and the engine does not turn over, the system asks "Do the lights come on?". WHY → "I am trying to prove a battery problem (R2); the engine does not turn over; if the lights don't come on it is a battery problem." After "No" it concludes battery problem; HOW → "By R2: NOT engine-turns-over and NOT lights-come-on were both given by you."

Q. 7. Write short notes on MYCIN.

Developed at Stanford in the 1970s to diagnose bacterial blood infections and recommend treatment when culture results are not yet available. Knowledge: IF-THEN rules with certainty factors, in Lisp. Reasoning: goal-directed backward chaining plus heuristics (ask common questions first, skip rules with a known-false antecedent, prefer high-certainty rules, drop lines below 0.2 certainty). Three stages: initial data, directed questions leading to diagnosis, treatment questions leading to recommendation. The user can ask WHY/HOW at any time. Derivatives: NEOMYCIN; its shell became EMYCIN.

Q. 8. What are the categories of expert system building tools? What is EMYCIN? List its characteristics.

Categories: expert system shells (EMYCIN, M.4), high-level programming languages (OPS5), multiple programming environments (LOOPS).

EMYCIN (Empty/Essential MYCIN) is MYCIN without its medical knowledge: a domain-independent framework for consultation programs. Characteristics: ALGOL-like abbreviated rule language; backward chaining; rules indexed and grouped by parameters; designer interface to display, edit and partition rules, edit tables and test rule sets, including the TEIRESIAS knowledge editor; smooth user interface.

Q. 9. Differentiate forward chaining and backward chaining. Which does MYCIN use and why?

Forward chaining is data-driven: start with known facts, fire every rule whose conditions match, add conclusions until nothing new follows. Backward chaining is goal-driven: start from a hypothesis, find rules concluding it, and prove their conditions as sub-goals (asking the user when needed).

MYCIN uses backward chaining because diagnosis means testing a limited set of hypotheses, and it lets the system ask only relevant questions and answer WHY/HOW by walking the goal tree.

Q. 10. In the paddy system of topic 20, a farmer answers: leaf spots Yes, spindle spots with grey centre Yes, heavy urea No, yellowing from tips No. Trace the consultation and suggest a fix.

Goal advice A: R4 needs blast; R1 asks leaf spots (Yes) and spindle spots (Yes), so R1 fires: disease = blast. R4 then asks urea: No, so R4 fails.

Goal advice B: R5 needs bacterial leaf blight; R3 asks yellowing from tips: No, so R3 and R5 fail. The system identifies blast but gives no advice: a knowledge gap. Fix: add R6: IF disease = blast AND NOT heavy-urea-applied THEN advice C (leaf sample to Krishi Bhavan for fungicide advice). This is found and fixed by the test step of the knowledge acquisition cycle.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.